

100 days of deep learning:

Day 2 deep learning vs ml

Day 3: Type of neural network:

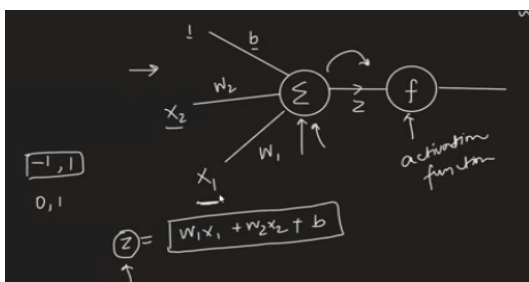
1. **multi-layer perceptron:** input layer, hidden layer and then output layer , used mostly in the supervised learning
2. **Convolutional neural network:** used mostly in image and video processing
3. **Recurrent neural network:** MLP AND CNN ARE FEED FORWARD NETWORK (INFO MOVE FROM START TO END LINEARLY) WHILE RNN SOMEWHERE IN HIDDEN LAYERS THE INFO GOES BACK TO IMPROVE THE PREDICTION, RNN ARE USED IN TIME SERIES, SEQUENTIAL DATA, ORDERED DATA
4. **AUTO ENCODERS:** WE HAVE INPUT DATA WHICH WE WANNA COMPRESS WITHOUT LOSSING QUALITY,
5. **GENERATIVE ADVERSERIAL NETWORKS:**

Day:4 Perceptron

Perceptron cannot learn a XOR function, perceptron work on linear models while It cant work on nonlinear data and xor is nonlinear function

Perceptron is the building block of everything

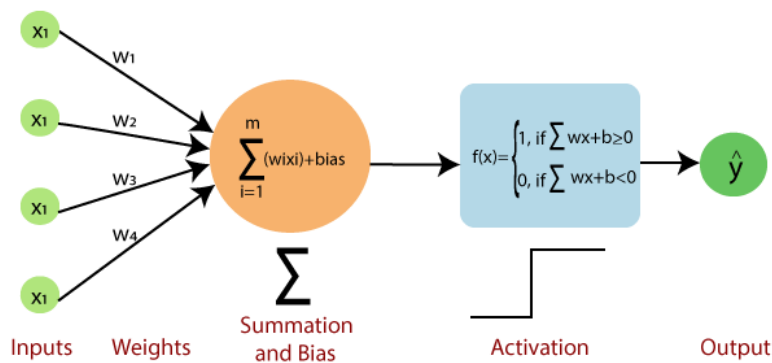
Perceptron is an algorithm (mathematical function/model) used for supervised ml, because of it design it automatically became the building block for deep learning



x_1, x_2 are input, w are weight for each input that tell us about feature strength, which feature matter the most in specific case and then there is b bias which all got summed into z , z is then passed through activation function $f(z)$

Activation function is to bring z value into some range like from -1 to 1 or 0 to 1 something like these

So we can say that y (target) is the function of z where $z = w_1x_1 + w_2x_2 + b$



Neuron vs perceptron

Neurons are so complex billion of input

We don't know what's happening in a neuron, how they process input in the nucleus while we know in the case of a perceptron that we have to multiply it by weight and take their sum along with bias and pass it through an activation function so processing is different

Neuroplasticity: Neuron input thickness can change over time while in the case of p they never change

Perceptron is inspired from a neuron but it's not a true copy of it

Perceptron is nothing but a line

Perceptron problem/limitation is that it can classify only linear or sort of linear data, (data that can be separated through a line)

If we have nonlinear data our perceptron will fail

So, the general rule is:

- ☒ Hidden neurons: each has an activation function.
- ☒ Output neurons: each has an activation function appropriate for the task (or linear for regression).
- ☒ Input neurons: typically do **not** have an activation function; they just pass the input features forward.

Yes. In a standard neural network, **every neuron (except usually the input neurons) has an activation function.**

Here's what happens inside a single neuron:

1. **Receive inputs** from the previous layer.
2. **Multiply** each input by its corresponding weight.
3. **Add** all the weighted inputs together.
4. **Add a bias.**
5. **Apply the activation function.**
6. **Output** the result to the next layer.

Mathematically:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + bz = w_1x_1 + w_2x_2 + \dots + w_nx_n + bz = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Then,

$$a = f(z)$$

where:

- x_i = inputs
- w_i = weights
- b = bias
- z = weighted sum
- f = activation function
- a = neuron's output

Day:5

Perceptron trick

Desmos is a tool to visualize line in 2d plane for different range for values for better understanding

To see whether the point is in positive region or negative region, just put values of that point into the line equation and if the value came below <0 then that is in negative region and vice versa

If we have to move line towards the negative region then we add the values
And same if we move line towards positive region then we subtract the values why is so?

How can we find weights through perceptron?

Problem with perceptron trick:

1. We can't be sure that weight and intercept value is best for this line, because if some points are misclassified then it will change the line towards itself by changing values of w and b .
2. We can't quantify our result that how better it is
3. Sometime it shows a problem in convergence sometime line does not converge

Day:6

Perceptron loss function

To solve the problem of perceptron trick we use loss function to quantify our result

Loss function is a function that tells us how good or bad our model is performing

Perceptron error calculation is different than logistic and linear regression
How to calculate that?

$L = 1/n \sum \max(0, -y f(x))$

What does it tell us $-y f(x)$?

Where n is no. of rows

For perceptron activation like step size we use perceptron hinge loss function

Multiclass classification we use SoftMax activation function and categorical cross entropy as a loss function

For probability (logistic regression)

we use sigmoid activation function and binary cross entropy (log loss) loss function

For regression we would use Linear activation function and mean square error as loss function

Loss function	Activation	Output
Hinge loss	Step	Perceptron
Log loss (binary cross entropy)	Sigmoid	Logistic regression
Categorical cross entropy	Softmax	Multiclass classification
Mse	Linear	Linear regression

Day:7

Problem with Perceptron

It can work only on linear data and cannot work on the nonlinear data. It cannot converge

A great tool for playing with neural networks

Playground.tensorflow

[https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-](https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2,2&seed=0.30539&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=true&xSquared=true&ySquared=true&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=regression&initZero=false&hideText=false)

[plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2,2&seed=0.30539&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=true&xSquared=true&ySquared=true&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=regression&initZero=false&hideText=false](https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2,2&seed=0.30539&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=true&xSquared=true&ySquared=true&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=regression&initZero=false&hideText=false)

Multilayer perceptron:

the hectic things in understanding the multi-layer perceptron is its training algorithm which the backpropagation

Day:8 MLP Notation:

Scalar Notation: $w_{ij}^{[l]}$

- **l**: The index of the **current layer** (e.g., $[1]$ for the first hidden layer, $[2]$ for the second, etc.).
-
- **i**: The index of the neuron in the **previous layer** that the weight originates from.
-
- **j**: The index of the neuron in the **current layer** $[l]$ that the weight goes to.
-
- **Example**: $w_{23}^{[1]}$ represents the weight on the connection from the (2^{nd}) neuron in the input layer (or previous layer) to the (3^{rd}) neuron in the (1^{st}) hidden layer. [1, 2, 3, 4, 5]
-

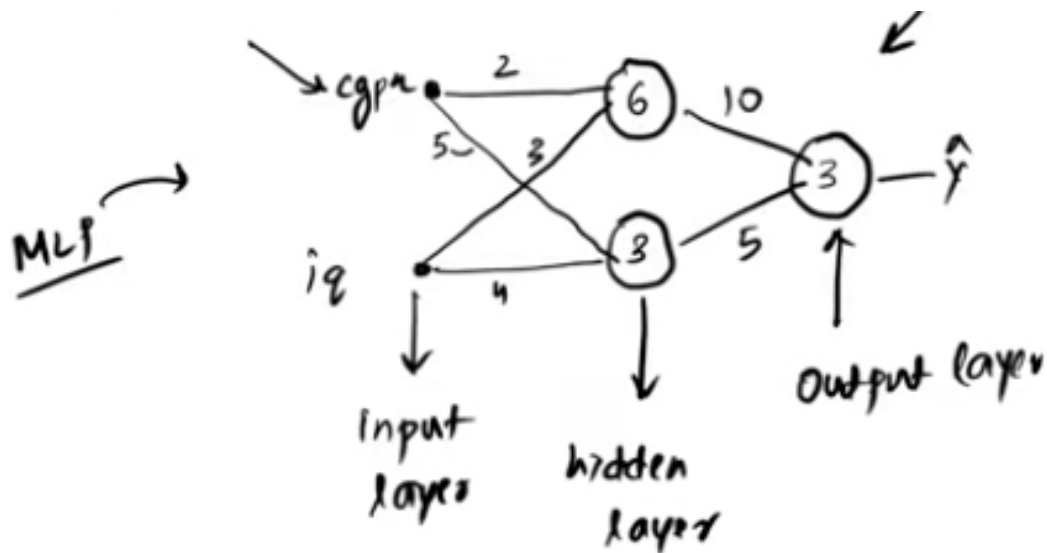
Day:9

Multi Layer Perceptron | MLP Intuition

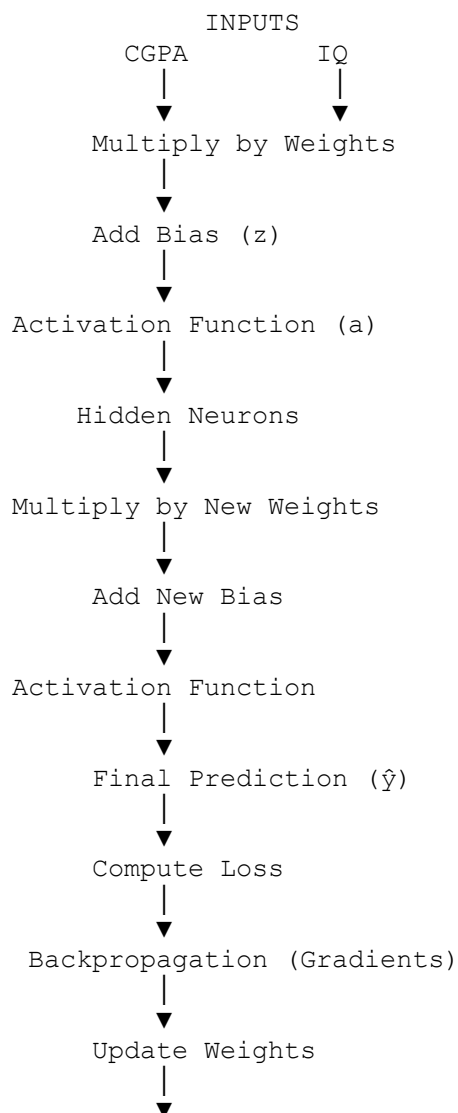
Single perceptron cannot create non linear decision boundary because it a linear model so the solution is combination of multiple perceptron to make a neural network which will be able to create non linear boundary

How MLP can create nonlinear decision boundary?

In mlp we make linear combinations of multiple perceptron



Complete Pipeline (Core Intuition)



Repeat Thousands of Times

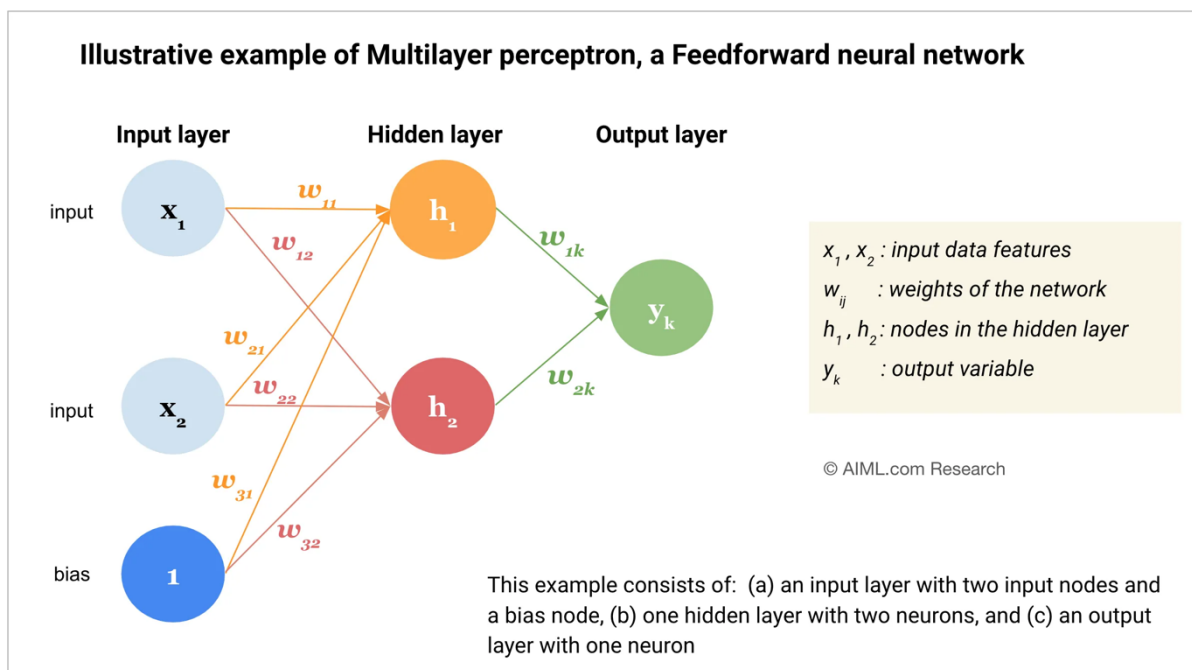
The one equation to remember

For **every neuron** in an MLP, regardless of the layer, the computation is always:

$$a = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

where:

- x_i : inputs to the neuron,
- w_i : learnable weights,
- b : learnable bias,
- $\sum w_i x_i + b = z$: pre-activation,
- $f(\cdot)$: activation function,
- a : output of the neuron.



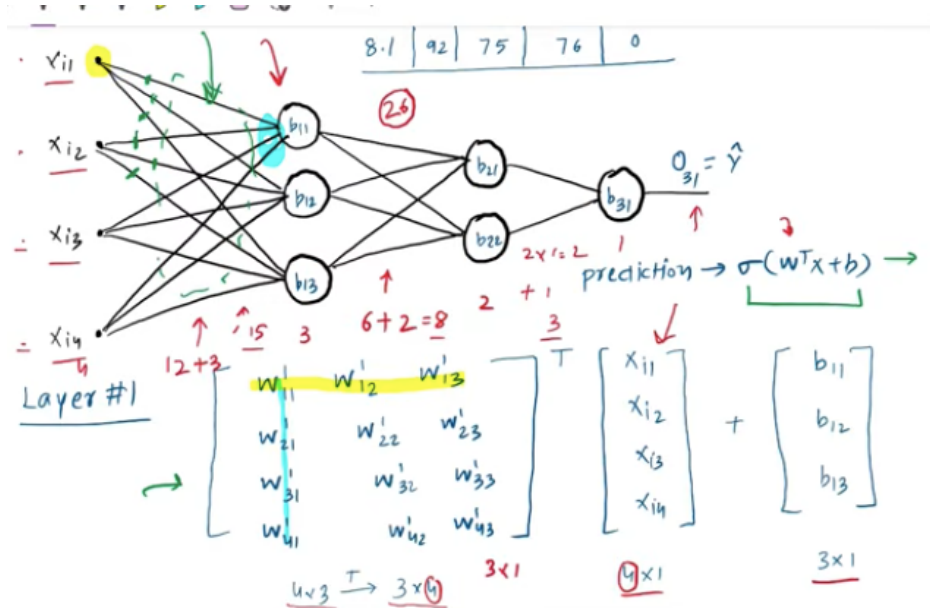
Adding nodes in the output node:

In Multiclass classification we keep multiple nodes in output layer where the perceptron/node with highest probability tell us which class it belong to

When we have very complex linear data then we have to add more and more layers to make the decision boundary

That why neural networks are called universal functional approximator: given enough hidden layers and enough training time we can make any mathematical function with neural networks

Day:10 Forward propagation:



$$= \begin{bmatrix} w_{11}^1 x_{i1} + w_{21}^1 x_{i2} + w_{31}^1 x_{i3} + w_{41}^1 x_{i4} \\ w_{12}^1 x_{i1} + w_{22}^1 x_{i2} + w_{32}^1 x_{i3} + w_{42}^1 x_{i4} \\ w_{13}^1 x_{i1} + w_{23}^1 x_{i2} + w_{33}^1 x_{i3} + w_{43}^1 x_{i4} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}$$

$$= \sigma \left(\begin{bmatrix} w_{11}^1 x_{i1} + w_{21}^1 x_{i2} + w_{31}^1 x_{i3} + w_{41}^1 x_{i4} + b_{11} \\ w_{12}^1 x_{i1} + w_{22}^1 x_{i2} + w_{32}^1 x_{i3} + w_{42}^1 x_{i4} + b_{12} \\ w_{13}^1 x_{i1} + w_{23}^1 x_{i2} + w_{33}^1 x_{i3} + w_{43}^1 x_{i4} + b_{13} \end{bmatrix} \right)$$

Output: 0_{11}

$$= \begin{bmatrix} 0_{11} \\ 0_{12} \\ 0_{13} \end{bmatrix}$$

Layer #2

$$\begin{bmatrix} w_{11}^2 & w_{12}^2 \\ w_{21}^2 & w_{22}^2 \\ w_{31}^2 & w_{32}^2 \end{bmatrix}^T \begin{bmatrix} 0_{11} \\ 0_{12} \\ 0_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}$$

Dimensions: $3 \times 2 \xrightarrow{T} 2 \times 3$ (3x1), 3×1 (2x1).

$$= \sigma \left(\begin{bmatrix} w_{11}^2 o_{11} + w_{21}^2 o_{12} + w_{31}^2 o_{13} + b_{21} \\ w_{12}^2 o_{11} + w_{22}^2 o_{12} + w_{32}^2 o_{13} + b_{22} \end{bmatrix} \right) = \begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix}$$

Layer #3

$$\begin{bmatrix} w_{11}^3 \\ w_{21}^3 \end{bmatrix}^T \begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix} + \begin{bmatrix} b_{31} \end{bmatrix}$$

$2 \times 1 \xrightarrow{T} 1 \times 2 \quad 2 \times 1 \quad 1 \times 1$

$$= \sigma \left(\begin{bmatrix} w_{11}^3 o_{21} + w_{21}^3 o_{22} + b_{31} \end{bmatrix} \right) = \hat{y}_i$$

DO EXAMPLE OF FORWARD PROPAGATION IN MATRIC NODE AND LAYER WISE ?

DAY 11: Loss Functions:

Loss function is a method of evaluating how well your algorithm is modelling your dataset.

High value of loss function -> poor algorithm and same vice versa

Loss function importance : you cant improve what u cant measure

Loss function is eye of ml algorithm which tell algorithm which way to go improve

Type of loss function in deep learning:

Regression : mse , mae, huber loss

Classification: binary cross entropy, categorical cross entropy(for multiclass classification task),hinge loss (SVM)

Autoencoders: kl divergence

GAN: discriminator loss, minmax GAN loss

Object detection: focal loss

EMBEDDINGS: triplet loss

Lost function vs cost function

Loss function : single training error checking

Cost function: of all batch , sum of all loss (error)

Mean Squared Error (MSE) – L2 LOSS

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Meaning

- Squares the error and average it
- Large errors become very large

Intuition: “Punish large mistakes more”

Example: **Errors: 2, 3, 4, 100**

MSE: $4 + 9 + 16 + 10000 =$ huge impact from 100

So: One outlier dominates everything

Advantages

- Differentiable (Gradient Descent)
- Easy for optimization
- One local minima

Why square helps

- Removes negative sign
- Makes function smooth
- Easy to optimize using gradient descent

Disadvantages

- Unit becomes squared so hard to interpret back
- **Penalize outliers: not robust to outliers** : as big errors become extremely big and outliers dominate result

Rule

- More outliers → use MAE
- Clean data → use MSE

In order to apply MSE we have to make sure that the last neuron activation function is linear

2. MEANS ABSOLUTE ERROR :

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

3. Huber loss:

Mathematical Formula for Huber Loss

$$\text{Huber Loss} = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2 & \text{if } |y_i - \hat{y}_i| \leq \delta \\ \delta \cdot |y_i - \hat{y}_i| - \frac{1}{2}\delta^2 & \text{if } |y_i - \hat{y}_i| > \delta \end{cases}$$

Where:

- y_i is the actual value.
- $\hat{y}_i$ is the predicted value.
- δ is a hyperparameter (threshold) that determines where the loss function switches from quadratic to linear behavior.

4. Binary cross entropy/ log loss

We use binary cross entropy when we have two classes in the classification problem

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n (Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

Differentiable so gd can be applied

Problem is that we can have multiple local minima

5. Categorical cross entropy:

Used in softmax regression

Used for multi class classification like if there are more categories/ classes in target features like (yes, no , maybe)

$$\text{Loss} = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

where k is number of classes in the data

Note in multiclass categorical classification we have to keep the number of neuron in outputs same as no of classes , if three classes then we have to keep 3 neuron in output layer

Sparse categorical entropy: ? it fast , we don't have to do one hot encoding here

DAY 12: Back Propagation (The what)

Backpropagation is an algorithm that is used to train neural networks using gradient descent.

Back Propagation follow a series of step

Steps:

0. Initialize w,b either randomly or make w=1 and b=0
1. You select a point (row)
2. Predict required thing using forward propagation (dot product)
3. Choose a loss function $(Y - Y^{\wedge})^{**2}$
4. Update weights and bias using gradient descent

Derivative part in gd formula means when we change w how much change will occur in the predicted value that will tell us the loss and based on that we update the gd weight

The derivative in Gradient Descent **does not measure how much the prediction changes**. It measures **how much the loss changes when the weight changes**.

A better way to say it is:

The derivative tells us how much the loss function changes for a small change in the weight. Based on this information, Gradient Descent updates the weight in the direction that reduces the loss.

Step-by-step

1. We make a prediction:

$$y^{\wedge} = wx + b$$

2. We calculate the loss:

$$L = (y - \hat{y})^2$$

3. We compute the derivative:

$$\partial L / \partial w$$

This answers the question:

"If I change the weight w a tiny bit, how much will the loss change?"

4. We update the weight: using gd

$$w_{\text{new}} = w_{\text{old}} - \alpha \partial L / \partial w$$

where α is the learning rate.

Back propagation algorithm

For I in range (epochs):

 Select 1 row random

 Predict using forward prop

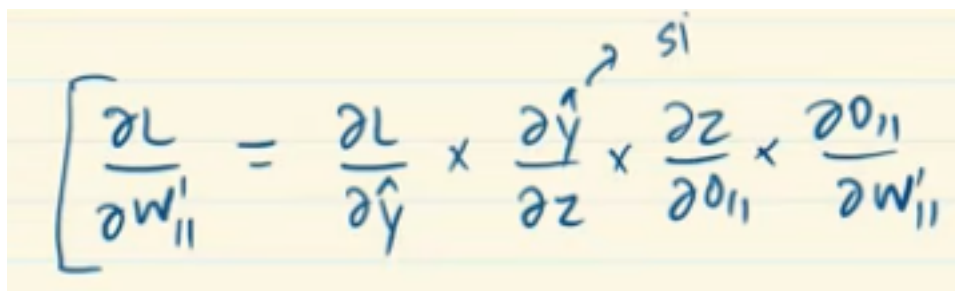
 Calculate loss using loss function

 Update weight and bias using gd

Calculate avg loss for the epoch

Make me understand how we back propagate with some example , take some dataset and show everything happen in back propagation

The whole backpropagation intuition



The image shows a handwritten mathematical formula on a yellow background with blue horizontal lines. The formula is:
$$\left[\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}} \right]$$
 There is a handwritten 'is' with an arrow pointing to the $\frac{\partial \hat{y}}{\partial z}$ term.

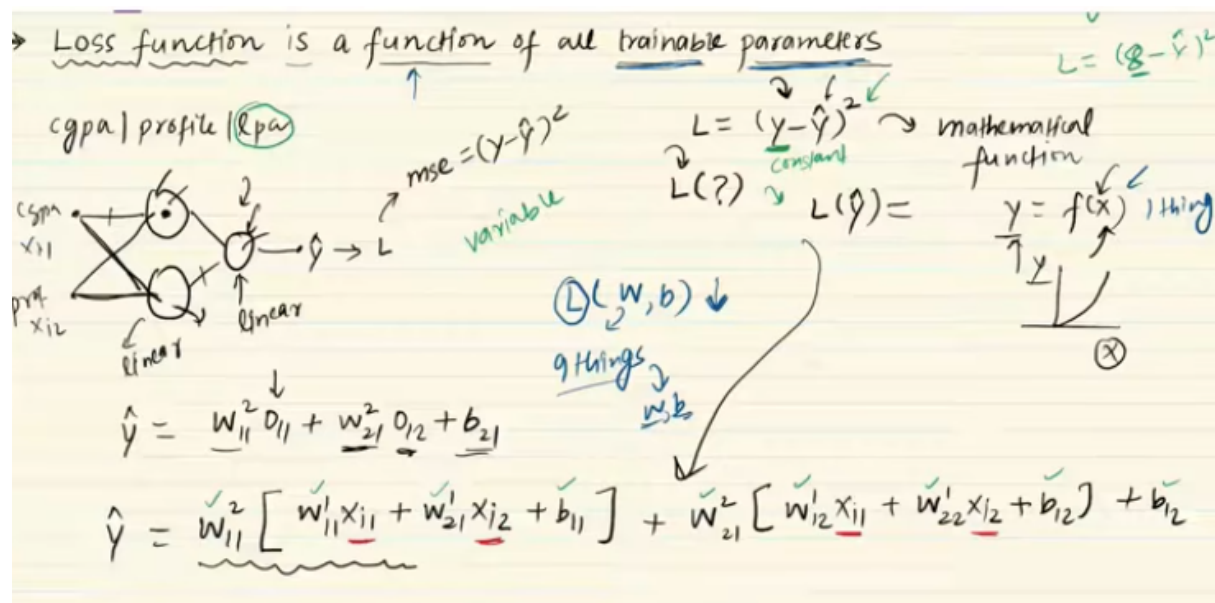
DAY 13: Back Propagation (the how)

Do mathematical how we update each previous weight and bias and update loss etc and everything

A tool link for animation of neural network forward propagation backward , activation function etc

<https://developers-dot-devsite-v2-prod.appspot.com/machine-learning/crash-course/backprop-scroll>

DAY 14: Back Propagation (the why)



Sometime when our function depend on just one paramet then we call it derivatice d/dx
 And when our function depend on multiple parameters then we call it gradient,
 we write $\text{del}z/dx$ and $\text{del}z/dy$

so here our loss function is complex function which depend on multiple weights and bias so we had to take partial derivative of multiple functions

how loss function depend on y hat which inturn focus on z which in turn depend on some o_{11} and which inturn depend on some weight values

Feature	Epoch	Iteration
Definition	One full cycle through the entire dataset.	One update step using one batch of data.
Scope	Outer loop of training.	Inner loop of training.

Weight Updates	Occurs multiple times per epoch.	Occurs exactly once per iteration.
Impact of Count	Higher counts can lead to overfitting.	Higher counts directly increase training time.

Concrete Example

Imagine you have a dataset of **1,000 images** and you set your **batch size to 200**. [\[1\]](#)

- **1 Batch** = 200 images.
- **1 Iteration** = The model processes 1 batch (200 images) and updates its weights.
- **1 Epoch** = The model must see all 1,000 images. [\[1, 2, 3, 4, 5\]](#)

To complete **1 epoch**, the model needs to complete **5 iterations** ($1,000 \div 200 = 5$). If you decide to train your model for 10 epochs, the system will run for a total of 50 iterations. [\[1, 2, 3\]](#)

The Role of Batch Size

The relationship is governed by the formula:

$$\text{Iterations per Epoch} = \frac{\text{Total Training Samples}}{\text{Batch Size}}$$

- **Batch Gradient Descent:** If your batch size equals your total dataset size, then **1 iteration = 1 epoch**.
- **Stochastic Gradient Descent (SGD):** If your batch size is 1, you will run as many iterations per epoch as you have individual data rows.
- **Mini-batch Gradient Descent:** The dataset is split into reasonably sized blocks (e.g., 32, 64, 128), resulting in multiple iterations per epoch

Convergence in Gradient Descent (GD) is the point where the model stops learning because it has found the optimal weights. It means the algorithm has successfully minimized the loss function and reached a stable bottom (a local or global minimum). As a model trains, its error drops quickly at first, then flattens out. **Once the changes in slope(error) become tiny or zero, the model has converged.**

DAY 14: Vanishing Gradient Problem

When we have to multiply small no like $0.1 * 0.1 * 0.1$ which will result in more small no 0.0001
Only in deep learning where we have multiple hidden layers

Often in sigmoid and tanh activation function because sigmoid bring each output in the range of 0 to 1

$$\left[\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}} \right]$$

What if each derivative give us a smaller value which will become more small on the multiplication between them

How to recognize that there is vanishing gradient problem

Focus on loss → it changing or not ,if no change then it vanishing gradient problem
Plot the graph of weights

How to handle VGP

1. Reduce model complexity, use shallow neural network instead of deep with many layers, not applicable because we use deep network to find complex things if we reduce it then it wont work
2. Using RELU activation functions formula is $\max(0, z)$ if -ve then zero on +ve z value would be the output, it derivative is either 0 or 1 , if -ve then zero if 1 then +ve, it has has problem of dying relu in which if relu got zero value then weight would not update in that case
3. Proper weight initialization like xavier and glorat
4. Batch normalization it has some layer
5. Residual network -> building block → use in ANN , in cnn resnet is build using residual network

Exploding Gradient problem : it is same as VGP but most commonly occur in RNN
It opposite of VGP like if we have big value on multiplication it will give us a very huge value

DAY 15: MLP MEMOIZATION

DAY 16: GRADIENT DESCENT IN NEURAL NETWORKS

DAY 17: HOW TO IMPROVE THE PERFORMANCE OF NN

DAY 18: EARLY STOPPING IN NN

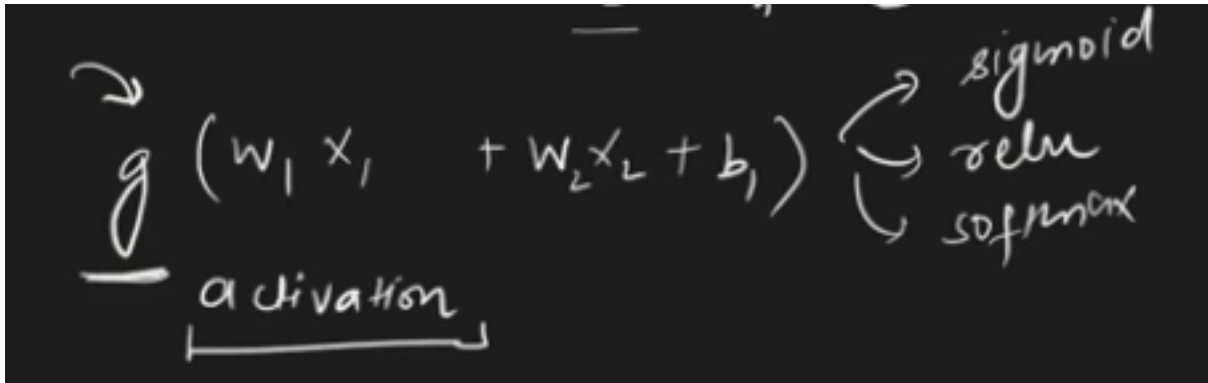
DAY 19: DATA SCALING IN NN

DAY 20: DROPOUT LAYER IN DEEP LEARNING

DAY 21: REGULARIZATION IN DL

DAY 22: ACTIVATION FUNCTION IN DL

In ann each neuron form a weighted sum of its input and passes the resulting scaler value through a function called as activation function or transfer function



Activation function decides whether any neuron would be activated or not.

If we wont use af in nn then it wont be able to capture non linear data

If the function g is not taken or taken as linear function then the neuron perform linear regression or classification. In general g is to be non linear function to do non linear regression and solve classification problem that are not linearly separable

Why activation functions are needed (mathematically):

Ideal activation function

1. Should be non linear
2. Ac should be differentiable , we should be able to calculate it derivative
3. Should be computationally inexpensive
4. Zero centered: zero center mean that next layer mean would be zero , or wither should be normalized, nn are easy/faster to converge when normalized given is provided, tanh is zero centered
5. Non saturating: saturating function squeeze input, sigmoid squeeze input into 0 to 1 , relu is non saturating, VGP happens due to saturating function

Sigmoid Activation function:

[0,1] we use this in output layer in binary classification , it is non linear function, is differentiable

Disadvantages is that it is saturating function which could lead to VGP.

Nowadays no one use it in hidden layers , used only in output layer for classification

It is non zero centered: not normalized , mean is not zero , either they +ve or -ve , which make training slow and create problem. In convergence

Also computationally expensive

Tanh Activation function / hyperbolic ac:

Formula is

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

It is between 1 and -1 where sigmoid was between 0 to 1

Non linear differentiable , zero centered

Still saturating problem, computationally slow

(Rectified linear unit) RELU ACTIVATION FUNCTION

Formula is

$$f(x) = \max(0, x)$$

-ve value it is zero while for positive value it is x input

Advantage : non-linear, not saturated in positive region , computationally inexpensive, convergence is faster

Disadvantage is not completely differentiable , not zero centred, so for that we use batch normalization which normalize the output of hidden layers, dying relu problem

DAY 23: RELU VARIANT EXPLAINED

$A = \max(0, x)$

Relu has problem: relu dying problem when some neuron got dead permanently, if more than 50 % neuron are dead then we wont be able to find pattern in the data

Why dying relu problem happens:

If we have negative value then our ac value would be zero in that case weight would not get updated

High learning rate

High -ve bias

What does it mean neuron is dead?

Solution:

set low learning rate

Set bias value positive $\rightarrow 0.01$

Don't use relu instead use its variants

It has linear variant and non linear variants? What does it mean linear variant and non

Linear : leaky relu $= \max(0.01z, z)$, parametric relu

Non linear variant : elu (its performance is better than relu, it's always continuous) , selu

DAY : Optimizers

Deep layers take most of the time in training so we speed up this process of this training

Speeding up depends on weight initialization, batch normalization , activation function and optimizers

Role of optimizer:

It's an optimization problem, in which we minimize the loss between the actual and predicted one

Gradient descent in which we update the weight

Adam: adaptive moment estimation

Most powerful for ann, cnn , rnn

Momentum concept came because gradient descent slow to converge to the global minima

Momentum had problem of oscillations, we crossing over and over again and coming back then came **NAG** momentum optimizer which damped the oscillation. THEN CAME THE problem of sparse data in case of which gd and momentum first move through non sparse data and then through sparse data , for to solve this problem we had **ADAGRAD** optimizer but it has problem of missing minima so for that came RMSPROP

IDEA OF MOMENTUM

LEARNING RATE DECAY

ADAM COMBINE BOTH THE IDEAS

IN MOST CASES ADAM IS FINE , IT'S A GOOD STARTING POINT

Day: weights initialization techniques

Day: xavier / glorat and he weight initialization in dl

Day: batch normalization in dl

Day: optimizers in dl

Day: exponentially weighted moving average or exponential weighted average

Day: sgd with momentum explained with animation optimizer in dl

Day: nesterov accelerated gradient explained

Day: adagrad explained

Day: rmsprop

Day: adam optimizer

Day: keras tuner

Convolutional Neural Network

Day: CNN intuition

Cnn are for processing data that has a known grid like topology like time series data (1d) or images (2d)

Layer in cnn

Convolutional

Pool layer

Fully connected dense layer

Picture is 2D grid picture

Why not use ann?

We can use ann as we used it for mnist but the problem is result wont be better as cnn.

Also will result in high computation cost in terms of converting that 2d grid to 1d for ann and then connected it to fully layer if we take 40x40 image and we take a small hidden layer with hundred units then we had to train 160000 parameters that result in high computation cost. Overfitting: connecting each pixel to each node then we capture minute thing so it will result in overfitting

Loss of important info like spatial arrangement of pixel ? what does it mean

Filter start filtering primitive features (edges) in the picture. then we pass it through other area from where it filter more complex features and goes on and thus we get our output

Popular cnn architecture

Lenet

Ailxnet

Vggnet

Resnet

Xception

Day Convolutional operation

The start layer (convolutional layer) in cnn detect primitive features (edges) , then moving deeper it slightly look for more complex features

What are edges(intensity change)? And then with combination of edges we make some pic or etc?, do explain the full working of convolutional layers

And do tell me everything I should about images for cnn what are they , grid pixels etc , collection of pixels

Give me example of above line how we do it in each phase

In black and white we have single channel

Collection of pixels is stored in 2d array

In colored picture we have 3 channels red green and blue

Every sheet will act as a single gray scale image

What are channels in colored piture? And why are 3 channels in coloredd ones and each channel has 1-255

228x228x3

For convolutional operation we need filter/ kernel which is generally a matrix of 3x3

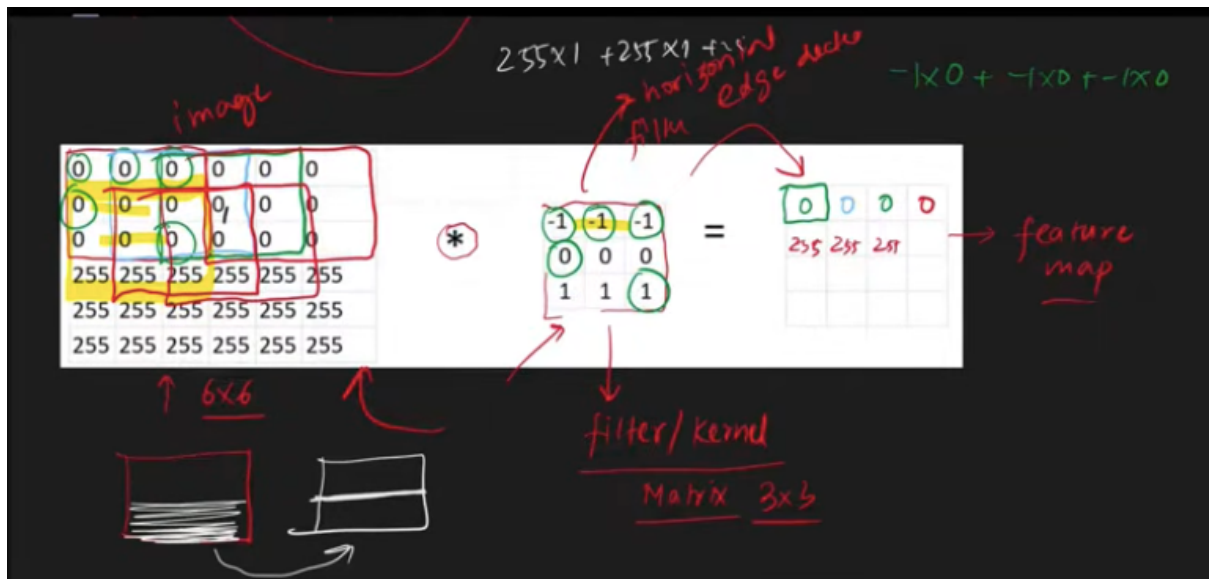
Image pixel grid is then element wise multiplied with this kernel/filter and thus we get output feature map and then added

do any small example on this above element wise and then addition etc upto how we get some feature map

What value would be there in filter generally ? what in case of horizontal and what in vertical and generally we use what

Do we initiate filter with random values and then update it through back propagation, filter values behave as weights in ann?

We convolve this filter onto the image

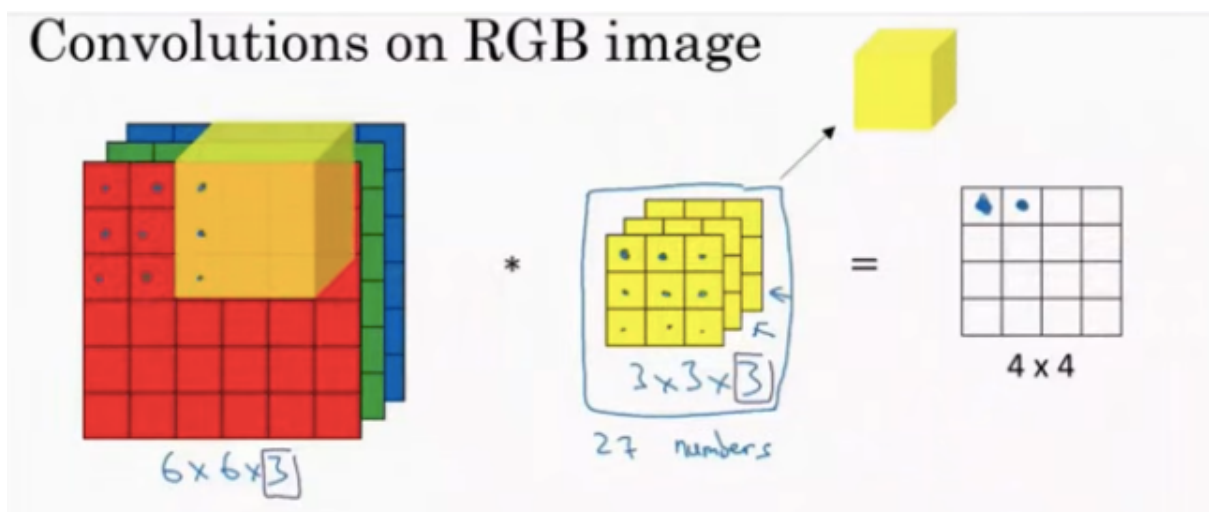


Formula for feature map size ?

If we have 6x6 pic and 3x3 filter then our feature map would be 4x4

Same way (28x28 and 3x3) then feature map would be (26x26)

If picture is $n \times n$ and filter is $m \times m$ then formula would be $(n-m+1) \times (n-m+1)$



What would be the size of feature pic in colored picture

Now if we have colored picture having three channels like 6x6x3 then our filter also become 3x3x3 one filter for each channel

Pixel $m \times m \times c$ filter $n \times n \times c$ $\rightarrow (m-n+1) \times (m-n+1) / \text{single channel}$

Generally, we apply multiple filter on image depend we can use horizontal vertical and other
How to apply multiple filter to the pic and do multiplication of it along with giving size , do some small example on it

If there is anything else in this concept do clear that for me too

Day Padding and Strides in CNN:

Give better working and explanation etc and make these topics notes that clear my concept about all these topics

Problem 1: we get features map that has less size than the actual image
now when we apply another filter on the feature map its size will decrease more
problem 2: the border pixels become part of convolutional operation less than the center ones pixels, filter move through center again and again so it means center pixels will have more importance, and side pixels information will get lost

we want image size not to get reduced

In padding we add one row and one column (top bottom left right) on each side of the picture thus padding increases the size of the picture and thus making feature map size equal to that of the before padding picture size, this boundary is called padding it generally has 0 value,

Formula : $(n+2p-f+1)$. What p here

In keras we can apply valid(no padding) and same padding(?)

Strides?

Let say we have 7x7 images and 3x3 filter, we then start doing convolutional operation over images and thus our filter moving over image (element wise multiplication and addition), when we move/shift to the right during convolutional operation then we call it stride=1 and then we move downward one pixel then we call it (1,1) stride

We can change stride (2,2) in this we will move jump next to two pixels right and down

In stride our feature map size $(n-f+1)$ will decrease $[(n-f)/s + 1]$ while in case of padding its formula would be $[(n + 2p - f)/2 + 1]$

When stride is greater than 1 it is called strided convolutional

Sometimes when using stride we can get decimal value on changing stride value, which will be then floored to previous digit then

Why we use stride ?

Two reasons:

when we just want to have high level features from your images, doesn't want low level features

second reason is computing power which makes our training faster

nowadays stride are not that much important in asense that it get rarely used

how will we apply padding and strides on the colored rgb images? Do give me concept of them

day Pooling:

228x228 Is common in cnn, used mostly in great architecture

Why we need pooling in cnn?

The problem with convolution?

1. Memory issues with convolutions operation ()
 - Suppose we have 228x228 images and we want to apply 100 filter then feature map would (226x226)x100 one map for each filter
 - We had to store this much feature maps which take a lot of memory
 - So we had to reduce the size of the fmap
 - Although we can use stride to reduce the size but pooling is much better for solving problem 2 too.
2. Translation variance:
 - During convolutional operation we find identify features so it become location dependent where they are picture they will shown as it is which Is not desired thing , where in image classification like task it doesnot matter where the object is in the picture

Pooling is a technique to down sample(reduce the femap size) your feature map , due to which feature become location independent

We have max pooling min pooling average pooling global pooling

How polling works

Apply relu on feature map first to add some non linearlity then Let say I have feature map of 4x4 , we had to tell 3 things to pooling : size (window on which it will operate), stride, type of pooling

Let say we pick size 2 so now we take 2x2 window into consideration in feature map

We will pick max value(dominant feature) in each window

Max pooling : pick dominant feature in the window , eliminate low level details

Min polling : will pick small no

Average : average of area

L2 :

Global pooling : we convert the matrix into 1x1 scalar value

Two types : global max pooling(will give global one max value in fmap) , global average pooling

Pooling on columns:

Colored images :

Advantages of pooling :

Reduce size of feature map

Translation in variance

Enhanced features only in max pooling

No need of training ? how so

Disadvantages : we lose much of the information

Day CNN Architecture

Lenet :

Convolutional operation

Padding and stride

Pooling

Make this below architecture pipeline more accurately

IMAGE -> convolutional layer -> flatten -> fully connected layer -> output

LeNET yann lekun father of cnn

ALEXNET

GOOGLENET

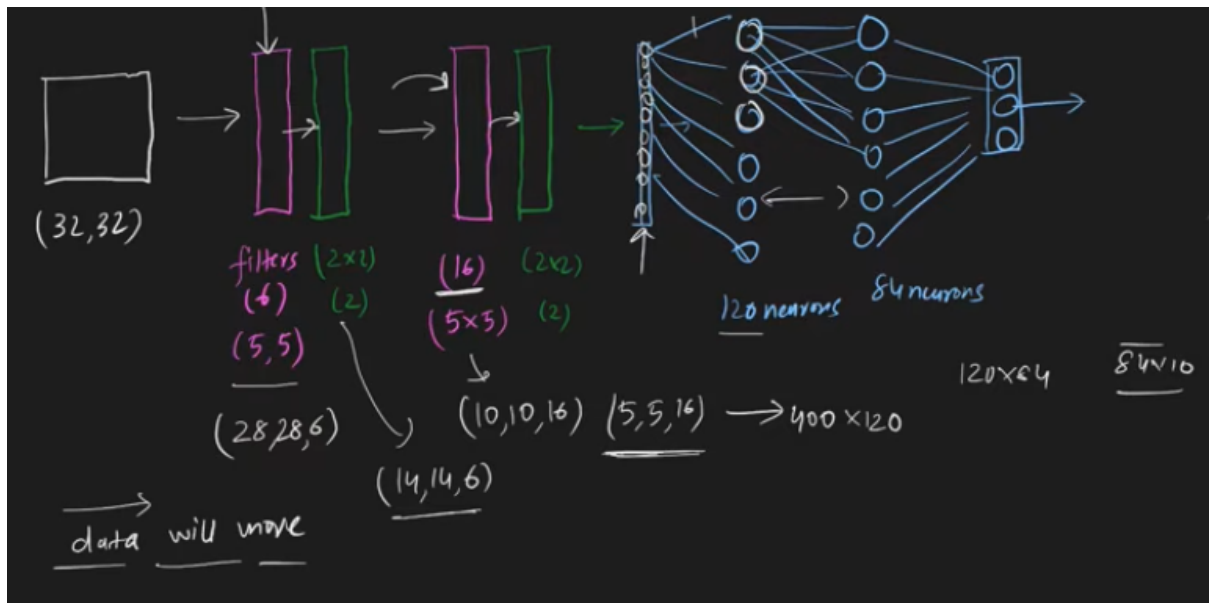
VGGNET

RESNET

INCEPTION

Lenet architecture:

Give me its overview?



Day CNN vs ANN

Computation cost
Overfitting
Loss of important features

Similarity is that in Ann weights are trained and in cnn filter values are trained
Ann ke nodes and cnn ke filter operate the same

Learnable parameter in the filter of $3 \times 3 \times 3$ if we have 50 filters = 1400 which has 50 bias and 1350 weights values

Day Backpropagation In Cnn

Had to cover these topics

Day Data augmentation technique:

Smart technique to reduce overfitting.
We create more photos from just one photo by applying various transformations.

Ek image ko leke uspe transformation applying krke abhot sara sasta data bana dete ha

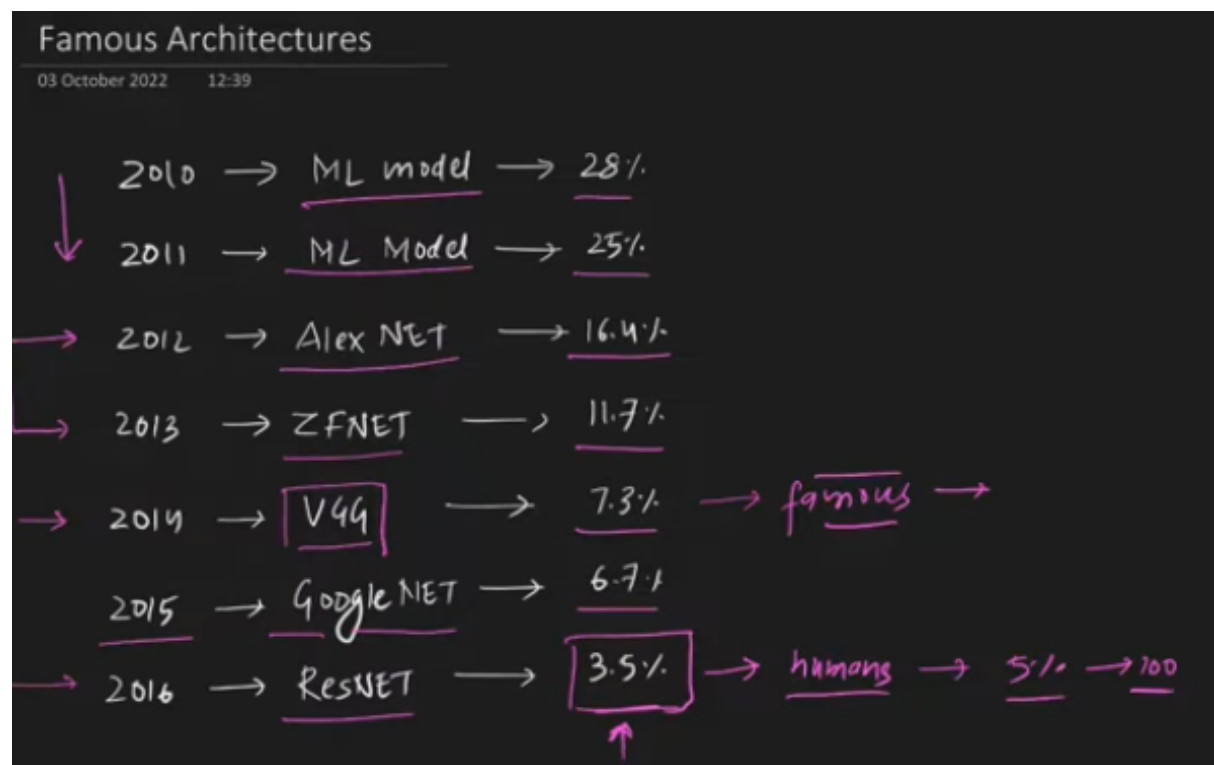
It is best for increasing accuracy of the model . using same data no of data normal and augmented but augmented will result in more accuracy

Day Pre trained models in CNN

Trained on dataset by someone else but we can use it for our own problem

ImageNet dataset: visual database of images
ImageNet challenge:

Idea of pretrained model: we need more data to train new model and why build everything from scratch when we have already trained model , so no need of training, and would not use any data



Day:

DAY: Transfer Learning

1. Why Transfer Learning Exists (The Core Problem)

Deep learning models are **data hungry** and **compute hungry**. A CNN that gets really good at image recognition (like ResNet, VGG, InceptionNet) is typically trained on **ImageNet** — 1.2 million images across 1000 classes — for days, sometimes weeks, on powerful GPU clusters.

Most of us don't have:

- Millions of labeled images
- Weeks of GPU time
- A budget for massive compute

The insight that saves us: A model trained on a huge, diverse dataset doesn't just memorize those 1000 classes. In the process of learning to tell a "cat" from a "truck," it is forced to learn *general-purpose visual building blocks* — edges, curves, textures, shapes, patterns of light and shadow. These building blocks are useful for almost **any** visual task, not just the one it was trained on.

Analogy: Think of someone who spent 10 years mastering violin. If they now want to learn viola, they don't start from zero — they already know how to read music, how bowing works, how to control pitch with their fingers. They just need to adapt to a slightly bigger instrument. They "transfer" their musical knowledge to a related but different problem.

That's transfer learning: **take the knowledge (weights) learned while solving Problem A, and reuse it for Problem B, which is different but related.**

2. Why Not Just Use the Pretrained Model As-Is?

Because the pretrained model's final job was to output probabilities for its **original 1000 classes** (dog breeds, vehicles, furniture, etc. — whatever ImageNet has). If your task is "classify cat breeds" or "detect tumors in X-rays," your desired output classes **don't exist** in the original model's output layer.

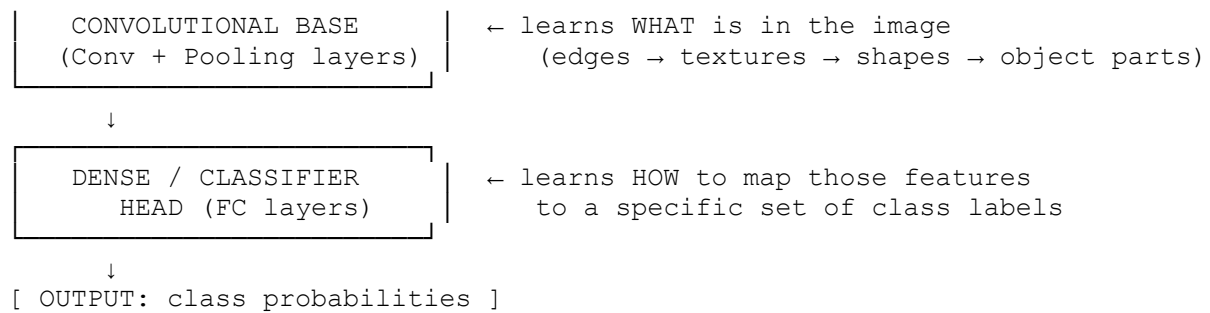
So we can't just deploy the pretrained model directly. We need to **surgically modify** it.

3. Anatomy of a CNN (Why This Split Makes Sense)

Every CNN used for image classification has two conceptual parts:

[INPUT IMAGE]

↓



- **Convolutional base:** Learns hierarchical, *generic* visual features.
 - Early layers → edges, colors, gradients (very generic, useful for almost everything)
 - Middle layers → textures, patterns, simple shapes
 - Later conv layers → more complex, semi-task-specific shapes (eyes, wheels, fur patterns)
- **Dense head:** Takes those extracted features and learns the *specific decision boundary* — "these feature combinations = Persian cat," "those = Siamese cat." This part is **entirely tied to the original dataset's classes**.

Analogy: Imagine a chef (the conv base) who has mastered chopping, dicing, sautéing, seasoning — universal cooking skills. Then imagine a recipe card (the dense head) that says "combine these prepped ingredients in this exact way to make Dish X." If you want to cook Dish Y instead, you don't need to re-teach the chef how to chop — you just need a new recipe card. The chef's *skills* transfer; only the *final recipe* changes.

This is exactly why transfer learning **freezes the conv base** and **replaces the dense head**.

4. The General Recipe

1. Take a pretrained model (e.g., VGG16, ResNet50, MobileNet, InceptionV3), trained on ImageNet.
2. **Remove** its original dense/classifier head.
3. **Freeze** the convolutional base (make weights non-trainable) — "don't reinvent the wheel," we already paid the cost of learning good generic features.
4. **Attach your own new dense layers** on top, ending in an output layer that matches *your* number of classes.
5. Train — but now you're only training the small new head, which needs far less data and compute.

This is why it's called "**don't reinvent the wheel**" — the expensive, generic feature-learning has already been done for you by researchers with far more data/compute than you have. You just build the small custom part on top.

5. The Two Flavors of Transfer Learning

This is the heart of your question — how do **Feature Extraction** and **Fine-Tuning** differ, and when do you use each?

5.1 Feature Extraction

What happens:

- Convolutional base = **completely frozen** (zero training, weights untouched).
- Only the new dense layers you add on top are trained.
- The conv base acts purely as a **fixed feature extractor** — you're just using it to convert images into meaningful numeric feature vectors, and then training a simple classifier on those vectors.

Intuition: You're saying "I trust that this base already extracts great generic features (edges, shapes, textures) — I just need to teach a new 'reader' (dense layers) how to interpret those features for my specific classes."

Analogy: You hire a professional photographer (frozen conv base) who is amazing at taking high-quality, detailed photos of anything. You don't retrain their photography skills. You just take their photos and hand them to a new analyst (dense head) who has to learn to sort the photos into *your* custom categories.

When to use Feature Extraction:

- Your dataset is **small** (few hundred to few thousand images).
- Your new classes are **visually similar** to what the original dataset (e.g. ImageNet) contained — e.g., classifying different cat breeds when the base was trained on many animals including cats.
- You have **limited compute** (training a full CNN is expensive; training only a small dense head is cheap).
- You want fast results with low risk of overfitting (fewer trainable parameters = less chance of memorizing a small dataset).

Risk if misused: If your images are *very* different from the original dataset (e.g., medical X-rays vs. ImageNet's everyday photos), the frozen features may not be relevant enough, and accuracy suffers.

5.2 Fine-Tuning (Detailed — for your project use)

What happens:

- You still start with a pretrained conv base and a new dense head.
- But instead of freezing the *entire* conv base, you **unfreeze some of the later (top) convolutional layers** and let them retrain, along with the new dense head.
- Early layers (which learn very generic features like edges/colors) usually **stay frozen** — no point retraining them, they're universal.

- Later layers (which learned more dataset-specific, complex patterns) are **unfrozen and retrained slightly**, so they can adapt from "ImageNet-specific shapes" to "my-dataset-specific shapes."

Intuition: You're not just reusing the photographer's photos anymore — you're sending the photographer back to a short training course so their eye adapts slightly to your specific subject matter, while keeping their fundamental photography skills intact.

Analogy: The violinist learning viola (from earlier) doesn't just apply violin technique blindly — they take a few lessons to adjust finger spacing and bow pressure for the bigger instrument (fine-tuning the "later," more specialized skills), while never having to relearn how to read music or count rhythm (the frozen "early," generic skills).

Why fine-tune only the top conv layers, not all of them?

- Early layers = generic (edges, colors, gradients) → useful for virtually every vision task → no reason to touch, and retraining them risks *destroying* good generic knowledge ("catastrophic forgetting").
- Later layers = more specialized to the original dataset's specific object types → these benefit from adapting to your new domain.

Why use a LOW learning rate during fine-tuning? This is critical and easy to get wrong.

- The pretrained weights are already very good — they're sitting in a "good neighborhood" of the loss landscape.
- If you fine-tune with a large learning rate, the big gradient updates will **violently disturb these well-tuned weights**, effectively destroying the useful pretrained knowledge — like slamming on the brakes so hard you fly through the windshield.
- A **low learning rate** (e.g., $1e-5$ instead of the default $1e-3$) makes small, gentle adjustments — nudging the weights just enough to adapt to the new task without erasing what was already learned.
- RMSprop (or Adam with a low LR) is commonly recommended because it adapts the learning rate per parameter, giving smoother, more stable updates — good for this delicate nudging process.

Analogy: Fine-tuning is like adjusting a finely-tuned guitar that's *already* mostly in tune. You turn the tuning pegs very slightly (low learning rate). If you crank them hard (high learning rate), you'll snap a string or throw it wildly out of tune — undoing all the careful setup that was already there.

Order of operations (important, standard practice):

1. First, do feature extraction: freeze the whole conv base, train only the new dense head until it converges reasonably.
 - *Why first?* If you unfreeze conv layers while your new dense head still has random, untrained weights, the large gradient signals from the messy dense head will backpropagate into the conv base and wreck the pretrained weights.
2. Once the dense head is reasonably trained, unfreeze the top N conv layers.
3. Re-compile the model with a **very low learning rate**.
4. Continue training (fine-tuning) all unfrozen layers + dense head together.

When to use Fine-Tuning:

- Your dataset is **moderately large** (thousands of images at least — fine-tuning more layers means more trainable parameters, which needs more data to avoid overfitting).
- Your new classes/domain are **fairly different** from the original dataset (e.g., ImageNet was trained on everyday photos, but you're classifying medical scans, satellite images, or something visually distinct) — so generic features alone (feature extraction) aren't quite enough; the model needs to *adapt* its higher-level feature detectors.
- You have **more compute/time budget** than feature extraction requires (more layers = more computation).
- You've already tried feature extraction and want to squeeze out more accuracy.

6. Feature Extraction vs Fine-Tuning — Quick Comparison Table

Aspect	Feature Extraction	Fine-Tuning
Conv base	Fully frozen	Partially unfrozen (top layers)
What's trained	Only new dense head	Dense head + some conv layers
Data needed	Small dataset OK	Needs more data
Compute needed	Low	Higher
Risk of overfitting	Lower	Higher (more params trainable)
Learning rate	Normal (e.g. 1e-3)	Very low (e.g. 1e-5)
Best when new data is...	Similar to original dataset	Different from original dataset
Speed	Fast	Slower

7. Code — Both Approaches (Keras/TensorFlow)

Step 0: Setup

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

Step 1: Load pretrained conv base (no top/dense layers)

```
conv_base = VGG16(
    weights='imagenet',          # load ImageNet-pretrained weights
    include_top=False,           # exclude original dense classifier
    input_shape=(224, 224, 3)
)
```


Step 2A: FEATURE EXTRACTION — freeze everything, add your own head

```
conv_base.trainable = False    # freeze the entire convolutional base

model = models.Sequential([
    conv_base,
    layers.GlobalAveragePooling2D(),    # or layers.Flatten()
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax') # your own output layer
])

model.compile(
    optimizer=optimizers.RMSprop(learning_rate=1e-3), # normal LR is fine here
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_generator, validation_data=val_generator, epochs=10)
```

At this stage, only the new Dense layers get trained — the millions of VGG16 conv parameters stay untouched. This is fast.

Step 2B: FINE-TUNING — unfreeze top conv layers, retrain with low LR

```
# Step 1: Do feature extraction first (as above) until the dense head converges.

# Step 2: Unfreeze the base, but only the TOP layers
conv_base.trainable = True

# freeze all layers except the last few (e.g., last 4 layers of VGG16's block5)
fine_tune_at = len(conv_base.layers) - 4

for layer in conv_base.layers[:fine_tune_at]:
    layer.trainable = False    # keep early layers frozen (generic features)
for layer in conv_base.layers[fine_tune_at:]:
    layer.trainable = True    # unfreeze the last few conv layers

# Step 3: Recompile with a VERY LOW learning rate — critical step!
model.compile(
    optimizer=optimizers.RMSprop(learning_rate=1e-5), # note: much lower than before
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Step 4: Continue training (fine-tuning)
history_finetune = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10
)
```

Key gotchas to remember in code:

- Always `compile()` the model again after changing `.trainable` — Keras won't apply the freeze/unfreeze change until you recompile.
 - Do feature extraction *before* fine-tuning, never fine-tune from scratch with an untrained head.
 - Use a much smaller learning rate for the fine-tuning phase (typically 10x–100x smaller).
 - Watch validation loss closely during fine-tuning — if it starts increasing, you're overfitting or your LR is still too high; consider early stopping.
-

8. Mental Model to Remember Forever

- **Pretrained model** = someone else already did the expensive homework (learned to see generically).
 - **Conv base** = "eyes" — knows how to perceive the world generically.
 - **Dense head** = "brain" — makes the actual decision/classification.
 - **Feature Extraction** = reuse the eyes as-is, train a new brain.
 - **Fine-Tuning** = reuse the eyes mostly as-is, but let them adjust *slightly*, and train a new brain — used when your visual domain is different enough that the eyes need a bit of retraining too.
 - **Low LR in fine-tuning** = don't shout instructions at someone who already mostly knows what they're doing — whisper small corrections instead.
-

9. Suggested Mini-Project to Practice This

Build a "Cat Breed Classifier":

1. Collect ~200–500 images per breed for 5 cat breeds (small dataset — perfect for transfer learning).
2. Load VGG16 (or MobileNetV2 for speed) with `include_top=False`.
3. Do **feature extraction** first: freeze conv base, train a small dense head, check validation accuracy.
4. Then do **fine-tuning**: unfreeze the last conv block, drop the learning rate to 1e-5, train a few more epochs.
5. Compare accuracy/loss curves between the two phases — you should visibly see fine-tuning squeeze out a bit more accuracy (if your dataset is different enough from ImageNet's typical cat photos) or plateau (if feature extraction was already sufficient).
6. Bonus: Try this with 2 different base models (VGG16 vs MobileNetV2) and compare speed/accuracy trade-offs.

This project alone will make feature extraction vs fine-tuning, layer freezing, and learning rate sensitivity all click through hands-on experience.

DAY: HOW TO BUILD NON LINEAR NEURAL NETWORK

Non-Linear Neural Networks (Keras Functional API)

Problem with Sequential API: Sequential model = a straight stack of layers, one input → one output, strictly linear flow. It can't handle branching, merging, multiple inputs, or multiple outputs.

Why linear topology fails sometimes: Example — predict **age** and **emotion** from a face image. Naive approach: train two separate models (wasteful, no shared learning). But if you want **one model, one shared conv base, two separate output heads** (multi-output), Sequential simply can't express this — it only allows one path in, one path out.

Analogy: Sequential API is like a single hallway — you walk straight through one room after another. Functional API is like a building with multiple hallways, junctions, and rooms that connect back to each other — you can branch, merge, and reuse rooms.

Solution: Functional API

- Lets you build models as a **DAG (Directed Acyclic Graph) of layers** instead of a stack.
- Supports:
 - **Non-linear topology** (branches, merges)
 - **Shared layers** (same layer reused on different inputs)
 - **Multiple inputs** (e.g., image + text together)
 - **Multiple outputs** (e.g., age + emotion from same face)

Core idea: Every layer is treated as a function — you call it on a tensor, and it returns a tensor. You explicitly wire together how data flows, instead of Keras assuming a straight stack.

```
from tensorflow.keras import layers, Model, Input

inputs = Input(shape=(224, 224, 3))

# shared conv base
x = layers.Conv2D(32, 3, activation='relu')(inputs)
x = layers.MaxPooling2D()(x)
x = layers.Flatten()(x)

# branch 1: age (regression)
age_output = layers.Dense(1, name='age')(x)

# branch 2: emotion (classification)
emotion_output = layers.Dense(7, activation='softmax', name='emotion')(x)

model = Model(inputs=inputs, outputs=[age_output, emotion_output])

model.compile(
    optimizer='adam',
    loss={'age': 'mse', 'emotion': 'categorical_crossentropy'},
    metrics={'age': 'mae', 'emotion': 'accuracy'})
```

)

Remember: Sequential = one path. Functional = graph — use it whenever you need branching, merging, shared layers, or multiple inputs/outputs.

DEEP LEARNING RNNs

Day Why rnn are needed?

- ANN -> tabular data
- CNN -> images
- RNN -> is type of sequential model to work on sequential data

In RNN order matter

Rnn are mostly used in nlp as rnn are well suited for textual data

Unnecessary computation in case of ann using for textual data

While using ann

Text input -> varying size so we can't adjust

Zero padding

Prediction problem

Totally disregarding the sequence

Sequence contains same meaning so we loss information

Day Forward propagation in RNN:

RNN has memory of previous input that why they are great for sequential data,

(Timesteps, input-features) that how we input data in rnn (3,5) 3 is no of timesteps 3 words
Keras -> simple RNN -> (batchsize , timesteps, input_features) here we send multiple ,
(3,4,5) 3 mean we sending 3 documents or 3 sentences

How RNN works ?

In ann we have input then hidden layer and then output layer

But in rnn there are big differences

1. How we feed data, we don't do it all at once

2. ANN is feed forward nn but rnn are not, they have concept of states, they send back feedback

How input is send back to the hidden layer? is there anything like this in rnn

Unfolding through time in forward propagation in rnn ?

How output of one timestep is send to another next timestep ?and moreover

With first input we give 0 to keep consistency across all steps

Why is it called recurrent nn

Because our hidden layer reoccur again and again with new input and same weights

At each step or whatever we add sum of input $x*w$ and previous output $O_{t-1}*w$

Types of RNN:

FOUR TYPE OF RNN ARCHITECTURES

1. One to One

Image

↓

Class

Not really RNN.

2. One to Many

Image

↓

Caption

Example

Photo

↓

"A dog is running."

USED IN IMAGE CAPTIONING

3. Many to One

Words

↓

Positive

Example

"I love this phone."

↓

Positive Review

4. Many to Many (Variable Length)

English

↓

French

Example

I love AI

↓

J'aime l'IA

5. Many to Many (Same Length)

Word1 → Tag1

Word2 → Tag2

Word3 → Tag3

Example

John

lives

in

Paris

↓

Name

Verb

Prep

Location

This is used in Named Entity Recognition Or POS Tagging.

Day : Backpropagation in RNN

Backpropagation through time
How is it done

Problems with RNN

RNN are suitable for sequential data but they are not used that much because they suffer from two problems.

1. Problem of long term dependency :

Not having memory capability to store long term memory, this problem come because of vanishing gradient

Cure is to use Relu activation function, better weights initialization like identity matrix, skip RNNS, move to LSTM

2. Problem of unstable gradients: exploding gradient problem

Stagnated gradients: long term dependency become so large that it dominates everything

Cure is to gradient clipping, controlled learning rate, LSTM



Day : LSTM

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Ltm is also called cell state while the stm is called hidden state , lstm make possible the communication between ltm and stm

Rnn has just stm short test memory while in lstm we got ltm memory cells too

In lstm

There are three inputs

Previous cell state (t_0) long term memory

Previous hiddens state (ct_0) short term meory

Input for the current time x

In output we get current hidden state (ht) and current cell state (ct)

Update cell state from $ct_0 \rightarrow ct_1$

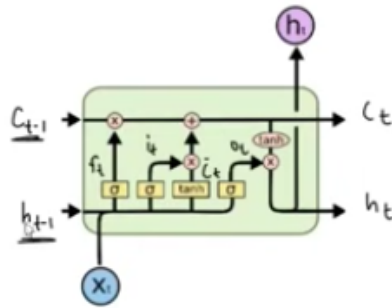
3 steps are involved in its processing

The gates

Forget gate: to remove something from cell state

Input gate: to add something to the cell state

Output gate: to calculate hidden state h_t for the next step



What are h_t and c_t mathematically? they are 3d vectors, both of their dimension will be same
Input x_t is also vector

Other vectors:

f_t : is in forget gate

i_t : is in input gate

c_t also in input gate, also called candidate cell state

o_t : in output gate

All these vectors shape, dimensions would be same to the c_t and h_t (rule)

Pointwise operation: between two vector whose shape and dimension would be the same and in result we will also get vector.

Multiplu X, addition +, taking tanh of each element in vector and making new vector having those values tanh

In the above picture the yellow boxes are neural network layers containing nodes and each nodes has activation function, and we decide how much nodes it must have, but whatever it is, they should same across all nueral network layers

Forget gate gets three things in the input

x_t

h_{t-1}

c_{t-1}

Based on these we remove certial things from cell state that deemed irrelevant or unnecessary

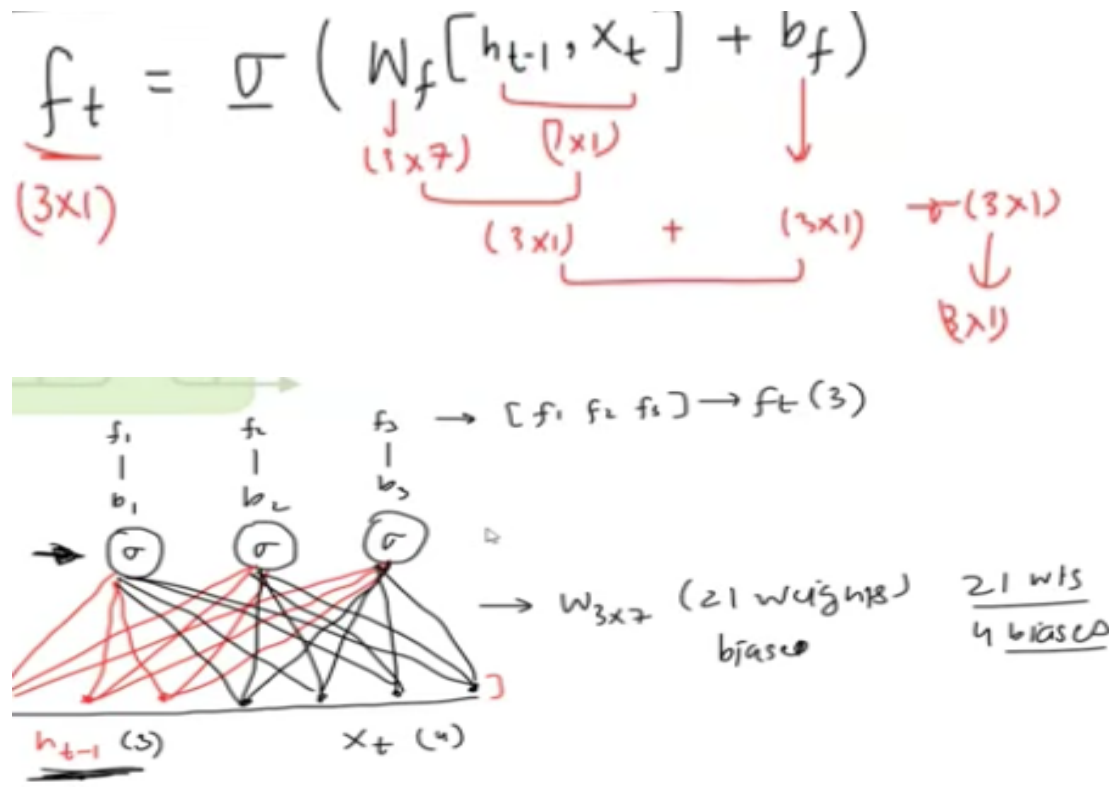
Cell state and hidden state dimension shape would be the same as number of nodes in our neural network layer

Work is done in two stages in the forget gate

1. Calculate F_t :
2. $C_{t-1} * F_t$: and this quantity mean we removing something from the cell state

In the hidden state we have 3d vector while in input X_t we have 4d vector and we have 3 nodes in the neural network , and all this is fully conncted which will give us 21 weights and 4 baieses , and when all these will pass through those nodes we will get f in the output which collectively give us F_t vector (3d)

Equation for calculating F_t



Why $F_t * C_{t-1}$ mean removal ?

F_t has the power how much information from C_{t-1} will be allowed to keep and passed next

Based on the input state our lstm will add those new information into the C_t

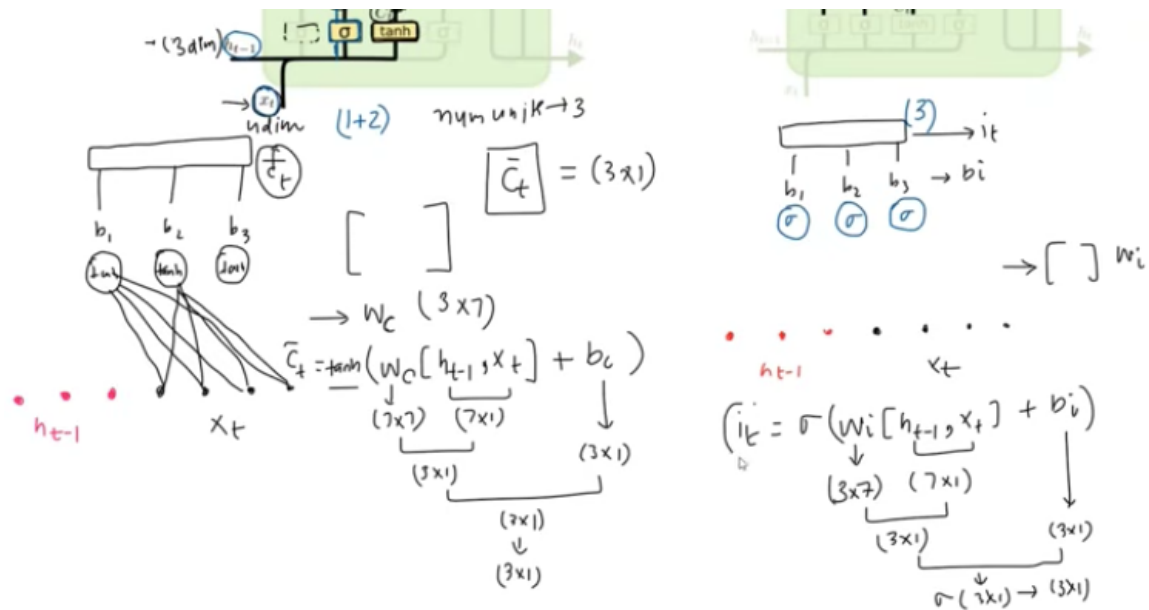
Stage 1: C_t candidate cell state : based on current input and previous hidden state we look for new candidate value to be added to ?

Stage 2: It input cell state :decides which value from the candidate cell state will be actually added to the Cell state

Stage 3: we finally C_t , which is out current cell state

How candidate cell state is calculated : done through the neural network

CANDIDATE CELL STATE IS POTENTIAL IMPORTANT INFORMTAION THAT SHOULD BE ADDED TO THE LONG TERM CONTEXT



It is like a filter which decides which information from candidate cell state should be added

Pointwise

$i_t * C_t \rightarrow$ filtered candidate cell state

And then this filtered candidate cell state value are added to the C_{t-1} and become part of long terms memory which will go the next timestamp

Output gate: current timestamp keliye output or hidden state deicde krna

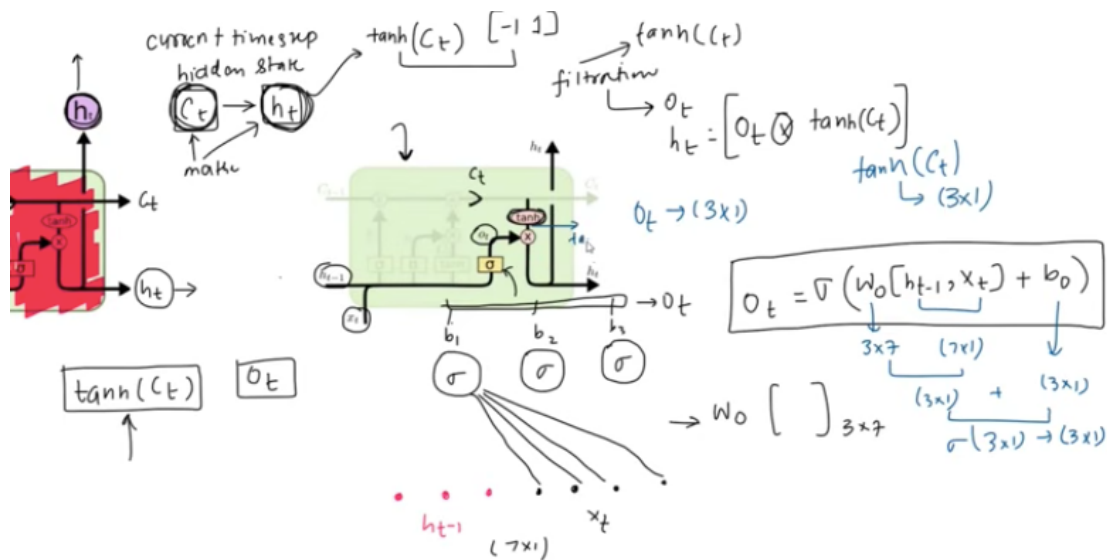
We get C_t from previous stage which we put in the tanh operation so we get $\tanh(C_t)$

Then we have to do filtration in it for which we have to find O_t

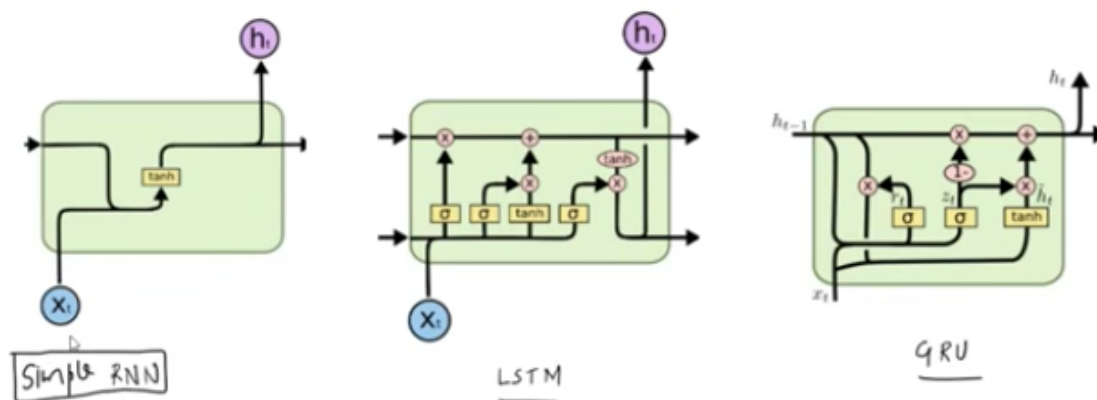
O_t is calculated through the neural network layer from the input of current timestamp input and previous hidden state input

Then O_t is multiplied with the $\tanh(C_t)$ which give us our output H_t for the current timestamp

Mathematical formula of each gate is in screenshot



Day : Gated Recurrent Unit (GRU)



When sequence become long in Simple RNN then it can't retain long term context, so it result in vanishing gradient and exploding gradient

Lstm solve this rpobelm we keep short memory and long memory in this

Os if lstm solve that problem why we need gru

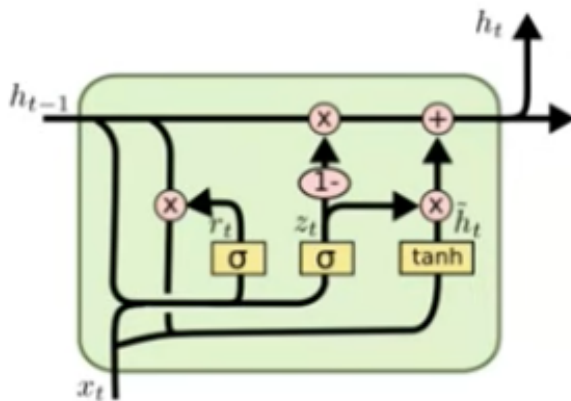
Lstm architexture is comeplex so it training time would be lot

So gru solve this problem as it give simple architecture

Here we have only two gates to carry both long term context and short term context

Reset gate

Update gate



$$\frac{\partial \mathcal{L}}{\partial x_t} \rightarrow \frac{\partial \mathcal{L}}{\partial h_t}$$

$h_{t-1}, h_t, x_t, r_t, z_t, \tilde{h}_t$?
 σ $\tanh$?

It takes two input

Current input and hidden state

h_{t-1} : represent hidden state which take our information forward

h_t : represent current hidden state

x_t : current input

r_t : reset gate

z_t : update gate

$\tilde{h}_t$ with approximate symbol=candidate hidden gate

Yellow boxes are neural network

$\otimes, +$ are element wise operation multiplication and addition etc

All these are vector mathematically of whatever dimensions

Except x_t , all other vectors dimension would be same if h_{t-1} is 4 D then all other would be 4d

Architecture :

In any timestamp we get x_t and h_{t-1} to get h_t

Steps

Calculate r_t reset gate

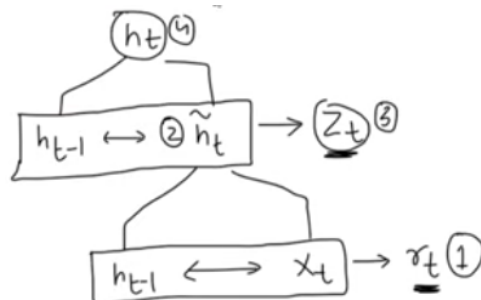
Calculate $\tilde{h}_t$ candidate hidden state

Calculate z_t update gate

Calculate h_t current hidden state

We get out h_t from combination of h_{t-1} and $\tilde{h}_t$ (candidate one) that is decide by z_t update gate

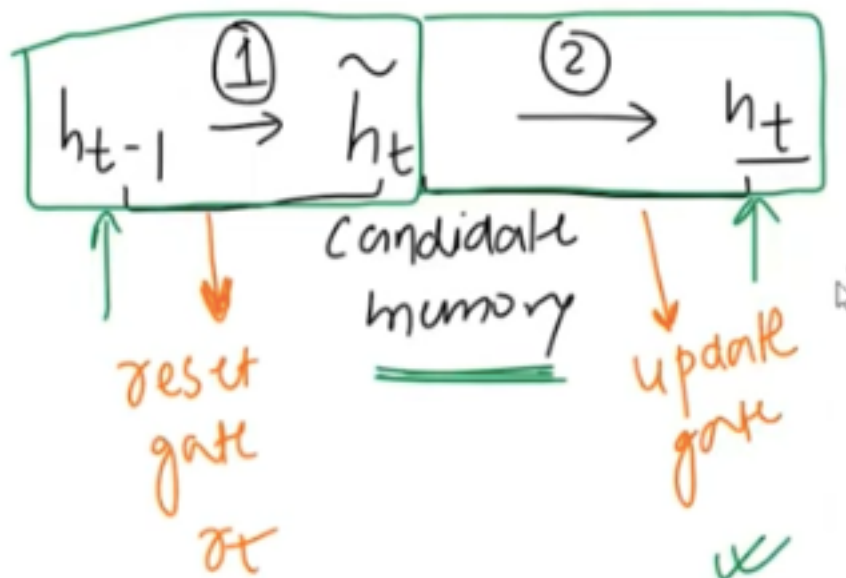
While our candidate hidden state is made of combination of h_{t-1} and x_t with help of reset gate



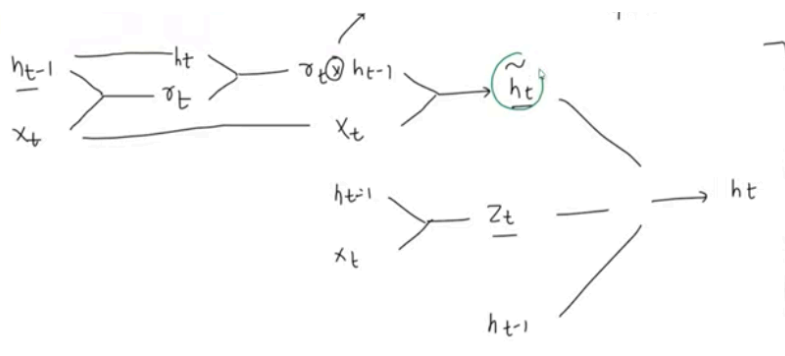
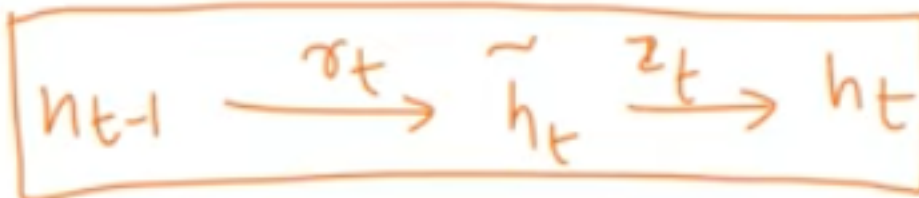
What are hidden state (h_{t-1} , h_t) what do they represent?

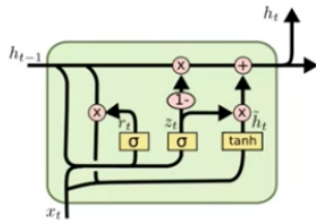
How they get updated in each timestamp do it through some example by passing one by one and updating h_{t-1} and h_t

h_{t-1} previous hidden state become candidate hidden state with the help of current input which then become our current hidden state explain this ?



This above conversion is done through help of gates





Reset gate based on our current input reset some of the past dimension of the H_{t-1}
 $R_t = \text{sigmoid}(\text{dot product of } (H_{t-1}, X_t) + b_r)$

H_t candidate state = mathematical expression? $\text{TANH}(\text{dot product of } (h_{t-1} * r_t, X_t) + b_r)$

$Z_t = \text{sigmoid}(\text{dot product of } h_{t-1}, X_t + b_z)$

Now to calculate H_t we do these calculation

$$H_t = ((1 - Z_t) * H_{t-1}) + (Z_t * h_t \text{ (candidate one)})$$

Lstm VS GRU :

Number of gates difference

Memory unit difference gru only use single hidden state

Lstm has more parameter count

Lstm has more computational complexity

Lstm has better empirical performance

Day : Bi Directional RNN

Why we need them?

Everything flow in forward Direction, mean output depend on all past input

What if there was situation where some past outputs depend on future inputs in that case unidirectional architecture of rnn fails

Assume we doing NER named entity recognition , here we extract entities from sentence , used mostly in chatbots

I love amazon , here we don't know whether this amazon is river or website so we cant tell what it is unless we read the next phrase , like it a great website , then we would know we talking about organization , here the future input affect our past output meaning , in this case we use bidirectional RNN

other examples?

How bidirectional rnn solve this challenge?

It use two diff rnn

forward rnn move from read from left to right

and backward rnn right to left

so here we make two rnns

HOW DO BOTH FORWARD AND BACKWARD RNN WORK SIDE BY SIDE

Forward and backward rnn equation and their explanations

We concatenate their output of both

Same bidirectional concept can be applied bilstm and bidirectional gru

We mostly use bilstm

Applications: NER, POS, Machine translation, sentiment analysis , time series forecasting (stock price)

Drawbacks : complexity, sometime we don't have full data so we cant do bi directional traversing

architectural variants in transformers all T5 models

Gpt

History of LLMs:

Day : Evolution of Sequence-to-Sequence Models: From Seq2Seq to LLMs

1. Background: Where Seq2Seq Comes From

Seq2Seq models originate from RNNs used on time-series/sequential data (e.g., video frames, text). RNNs are commonly categorized by input-output length relationships:

- **Many-to-Many (equal length):** input and output sequences have the same length, aligned position by position.
- **Many-to-Many (variable length):** input and output sequences can differ in length (e.g., machine translation — a 5-word English sentence may translate to a 7-word French sentence).

Why Seq2Seq exists: Standard RNN architectures assume aligned, fixed-length mappings. Real-world tasks like translation, summarization, and Q&A need variable-length input/output pairs. Seq2Seq models were designed specifically to solve this variable-length many-to-many problem.

Applications: text summarization, question-answering systems, chatbots/conversational AI, speech-to-text.

2. The Historical Progression (Overview)

Stage	Development
1	Encoder-Decoder architecture
2	Attention mechanism

Stage	Development
3	Transformers
4	Transfer learning using Transformers
5	LLMs (e.g., ChatGPT)

Each stage exists **because the previous stage had a specific, identifiable limitation**. This is the core logic to remember.

Stage 1: Encoder–Decoder Architecture

Reference paper: *Sequence to Sequence Learning with Neural Networks* (Sutskever et al.)

How it works:

- Both the encoder and decoder are built using LSTMs.
- The encoder reads the input word by word, progressively summarizing/compressing the information.
- At the end of the input sequence, all the accumulated information is compressed into a single fixed-size vector called the **context vector**.
- The decoder takes this context vector and generates the output sequence step by step.

Limitation:

- This worked well for short sentences, but for **long sentences**, compressing an entire sequence's meaning into one fixed-size context vector caused **information loss**. Early words in a long sentence would get diluted or forgotten by the time encoding finished.

Why the next stage was needed: A single vector is an information bottleneck — it cannot scale to represent long, detailed sequences. This motivated the **attention mechanism**.

Stage 2: Attention Mechanism

Core idea: Instead of relying on a single final context vector, the decoder gets access to **all** the encoder's hidden states (from every input step), not just the last one.

How it works:

- At each decoder step, attention automatically searches across all encoder hidden states to determine which ones are most relevant to the word currently being generated.
- This produces a **dynamically computed context vector specific to each decoder step**, rather than one static vector for the whole sequence.

Why it was needed: With many hidden states available (one per input word), the model needs a way to know *which* hidden state matters for the current output step. Attention solves this by learning a relevance/similarity score between the current decoder state and every encoder state.

Limitation:

- **Computational complexity:** For every decoder step, the model must compute similarity scores against *every* encoder step. This means training time grows significantly, since a full comparison is done repeatedly across all word pairs.
- The architecture was still fundamentally sequential (LSTM-based), so words were still processed **one at a time**, limiting parallelization.

Why the next stage was needed: Researchers wanted to remove the sequential bottleneck of RNNs/LSTMs entirely and enable **parallel processing** — this led to the Transformer.

Stage 3: Transformers

Core idea: Remove RNNs and LSTMs entirely. Introduce **self-attention** as the primary mechanism.

How it works:

- Still follows an encoder–decoder structure, but both the encoder and decoder use **self-attention** internally.
- Self-attention allows the model to look at **all words in a sequence simultaneously** (in parallel), rather than one at a time as in RNN-based models.
- Combines attention layers with fully connected (dense) feed-forward neural network layers.

Why it was needed: RNN/LSTM-based attention models processed one word at a time, making training slow. Self-attention removes this constraint, allowing parallel computation across the whole sequence — dramatically speeding up training.

Limitation:

- Transformers are powerful but **expensive and hard to train from scratch:** they require large-scale GPU hardware, long training times, and massive datasets. Not every team or task has access to this scale.

Why the next stage was needed: This limitation motivated **transfer learning** — reusing a large pre-trained model instead of training from scratch for every new task.

Stage 4: Transfer Learning Using Transformers

Definition: Transfer learning is a technique where knowledge learned from one task is reused to boost performance on a related task.

Example (non-NLP): Knowledge gained by a model learning to recognize *cars* can be partially reused when learning to recognize *trucks*.

Two-step process:

1. **Pre-training:** Train the model on a large, general/universal dataset to learn broad, general features of the data (not task-specific).
2. **Fine-tuning:** Take the pre-trained model, keep the weights of the earlier layers (general features), and retrain/adjust the later layers on a smaller, task-specific dataset.

Key paper: ULMFiT Before this paper, common misconceptions limited the adoption of transfer learning in NLP:

- The belief that transfer learning is too **task-specific** to generalize across NLP tasks.
- The belief that NLP transfer learning would fail due to **lack of sufficient data**.

ULMFiT's contribution:

- Instead of pre-training on machine translation (a supervised, task-specific objective), it used **language modeling** as the pre-training task.
- **Language modeling** = an unsupervised NLP task where the model learns to predict the next word in a sentence.

Why language modeling as pre-training works well:

- **Rich feature learning:** predicting the next word forces the model to learn deep semantic and syntactic structure of language, not just surface patterns.
- **Huge data availability:** since it's unsupervised (no labels needed), virtually unlimited raw text can be used for pre-training.

This combination (general pre-training via language modeling + task-specific fine-tuning) became the blueprint for the next stage.

Stage 5: Large Language Models (LLMs)

Origin: Following ULMFiT, in 2018 two influential transformer-based language models were released:

- **BERT** — Encoder-only architecture
- **GPT** — Decoder-only architecture

Both were trained as language models (next-word/masked-word prediction tasks) using the Transformer architecture, and both could be fine-tuned by others for downstream tasks.

Why they're called "Large" Language Models: These models were trained on such massive datasets that they earned the "large" designation. This scale requires:

- **Data:** billions of data points, sourced from diverse origins (books, websites, etc.) — diversity matters to reduce bias.
- **Hardware:** clusters of GPUs.
- **Training time:** ranging from days to weeks.
- **Cost:** significant investment in infrastructure, electricity, hardware, and expert personnel.

From GPT-3 to ChatGPT

Important distinction:

- **GPT** is the underlying language *model*.
- **ChatGPT** is a *chatbot application* built on top of a GPT model, adapted specifically for conversational use.

Steps taken to turn a raw GPT model into ChatGPT:

1. **RLHF (Reinforcement Learning from Human Feedback)**
 - Supervised fine-tuning was done first using curated conversational data.
 - Human evaluators then ranked multiple model responses, and this ranking feedback was used to reinforce better response behavior.
2. **Safety and ethical guidelines**
 - Additional training/filtering to minimize biased, harmful, or inappropriate outputs.
3. **Improved contextual understanding**
 - Enhanced ability to maintain context across multiple turns, remembering earlier parts of a conversation.
4. **Dialogue-specific training**
 - Training data and objectives shifted to focus specifically on natural, coherent conversation rather than generic text completion.
5. **Continuous improvement**
 - Ongoing updates and refinement based on user interaction and feedback over time.

Summary: The Logical Thread

Each stage exists to fix a specific flaw in the one before it:

- **Encoder-Decoder** → solved variable-length seq2seq problems, but **lost information on long sequences** (single bottleneck vector).
- **Attention** → solved information loss by referencing all encoder states, but was **computationally expensive and still sequential**.
- **Transformers** → solved the sequential bottleneck via self-attention and parallelization, but were **expensive/hard to train from scratch**.
- **Transfer Learning (ULMFiT)** → solved the cost/data problem via pre-training + fine-tuning, using language modeling as an efficient unsupervised pre-training task.
- **LLMs (BERT/GPT)** → scaled this approach massively using huge diverse datasets and compute.
- **ChatGPT** → adapted a raw LLM (GPT) into a safe, context-aware, conversational product via RLHF and dialogue-specific training.

Day: Encoder–Decoder Architecture for Seq2Seq Data

Reference: "Sequence to Sequence Learning with Neural Networks" — Sutskever, Vinyals, Le (2014)

1. The Journey — Why This Architecture Even Exists

Deep learning architectures weren't invented randomly — each one exists because the previous one *failed* at a specific kind of data. Understanding this chain is the fastest way to actually understand *why* Seq2Seq looks the way it does.

Step 1: Tabular data → ANN (Artificial Neural Network)

A plain ANN (just stacked dense/fully-connected layers) works great for tabular data — rows of independent features like `age`, `salary`, `city` → `buys_product?`. Each row is independent of every other row; there's no "order" between rows that matters.

Why ANN fails on sequential data: An ANN takes a *fixed-size* input vector and has no built-in concept of "what came before." If you feed it a sentence word by word, it has no memory — it treats each word in isolation and has no way to know that word 5 depends on word 2. It also can't handle variable-length inputs (a sentence with 5 words vs. one with 20 words) without hacks like padding, and even then it still won't capture *order/sequence relationships*.

Step 2: Image data → CNN (Convolutional Neural Network)

Images have **spatial** structure — a pixel's meaning depends on its neighboring pixels (edges, textures, shapes). CNNs use filters/kernels that slide over the image to detect local patterns (edges → shapes → objects), and this works because that spatial relationship is fixed and local.

Why CNN isn't naturally suited for text: Text has **sequential/temporal** structure, not spatial structure. The relationship that matters in "I did not like the movie" is the *order* of words, not their 2D spatial neighborhood. CNNs can be forced onto text (1D convolutions exist), but they don't naturally model "what happened 10 words ago affects what this word means now" the way sequence models do.

Step 3: Sequential data → RNN, then LSTM/GRU

Text, time series, speech — all of these have **order that carries meaning**. "Dog bites man" ≠ "Man bites dog," same words, different order, different meaning.

RNNs introduced the idea of a **hidden state** that gets passed from one timestep to the next — like a running memory. But plain RNNs suffer from the *vanishing gradient problem*: over long sequences, information from early timesteps fades out before it reaches later ones. LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) were built to fix this using gates that control what to remember, forget, and output — letting them retain long-range dependencies far better than plain RNNs.

Step 4: Seq2Seq data → Encoder–Decoder

LSTM/GRU solved "understand a sequence." But a new problem appeared: **machine translation, text summarization, chatbots** — tasks where **both the input AND the output are sequences**, and their lengths don't match.

- Input: "How are you?" (3 words)
- Output: "Comment ça va?" (3 words, but not always — translations often differ in length)
- Input: "I love deep learning" (4 words) → Output: "J'adore l'apprentissage profond" (different word count entirely)

A single LSTM reads a sequence and produces *one* output per input timestep, or a single final vector — it isn't designed to *generate* an entirely new, independently- lengthed sequence. This mismatch is exactly what the Encoder–Decoder architecture was built to solve.

2. Why Variable-Length Seq2Seq Is Genuinely Hard

Your question: "*We handle variable length data in LSTM/GRU but only in the input phase, not in output. What does this mean?*"

Short answer: An LSTM naturally handles variable-length **input** because it just loops through the sequence one timestep at a time — it doesn't care if there are 5 words or 50, it processes them one by one and updates its hidden state. That's the easy part.

The **output** side is a different problem entirely. When generating an output sequence:

- You don't know its length in advance (a translation might need 4 words or 10 — the model has to decide *when to stop*).
- Each output word depends on the words *already generated by the model itself*, not just on the input — this is a fundamentally different, generative process.
- A plain LSTM has no built-in mechanism to say "stop generating now" or "here's my next guess based on what I just said."

That's why the Decoder is architecturally different from a simple LSTM pass — it needs a `<START>` signal to know when to begin generating, a loop that feeds its own previous output back in as the next input, and an `<END>` token to know when to stop.

3. The Encoder–Decoder Architecture

The Encoder

- Takes the input sequence **word by word** (e.g., "I am a student").
- Each word is fed into an LSTM cell **one timestep at a time**.
- The LSTM updates its **hidden state** and **cell state** at every timestep, carrying forward everything it has "understood" so far.
- After the *last* word is processed, the encoder's final hidden state + cell state becomes the **context vector** — a compressed numerical summary of the entire input sentence's meaning.

Think of it like reading a whole sentence and then closing your eyes and holding just the *gist* of it in your head — that gist is the context vector.

Your question: *"How is context vector different from embedding?"*

This trips up almost everyone, so here's the clean distinction:

	Embedding	Context Vector
What it represents	A single word or token's meaning	The meaning of the entire input sequence
When it's created	Before/as input enters the network (a lookup)	After the encoder has processed the <i>whole</i> sequence
Fixed or contextual	Static per word (in classic embeddings like Word2Vec)	Built dynamically depending on everything seen so far
Analogy	A dictionary definition of one word	A one-paragraph summary of an entire book

So: embeddings go **in** (per word), the context vector comes **out** (per sentence, after the encoder finishes reading everything).

The Decoder

- Its **initial hidden state and cell state are set equal to the encoder's final hidden/cell state** — i.e., the decoder "inherits" the encoder's summarized understanding as its starting memory.

Your question: *"Initial state of first LSTM of decoder ko encoder ke final state ke barabar set kia jata hai — what does it mean?"*

It literally means: `decoder_initial_state = encoder_final_state`. The decoder doesn't start "blank" — it starts already "knowing" everything the encoder compressed from the input sentence. This is *how* the context vector actually gets used — not as a separate input fed at every step (in the original 2014 paper), but as the decoder's starting memory.

- A special <START> token is fed in first, telling the decoder "begin generating now."

- At each timestep, the decoder predicts the next word, and (during training, via teacher forcing — explained below) the *correct* next word is fed back in as input for the following timestep.
- This continues until the decoder outputs a special <END> token, telling the system to stop.

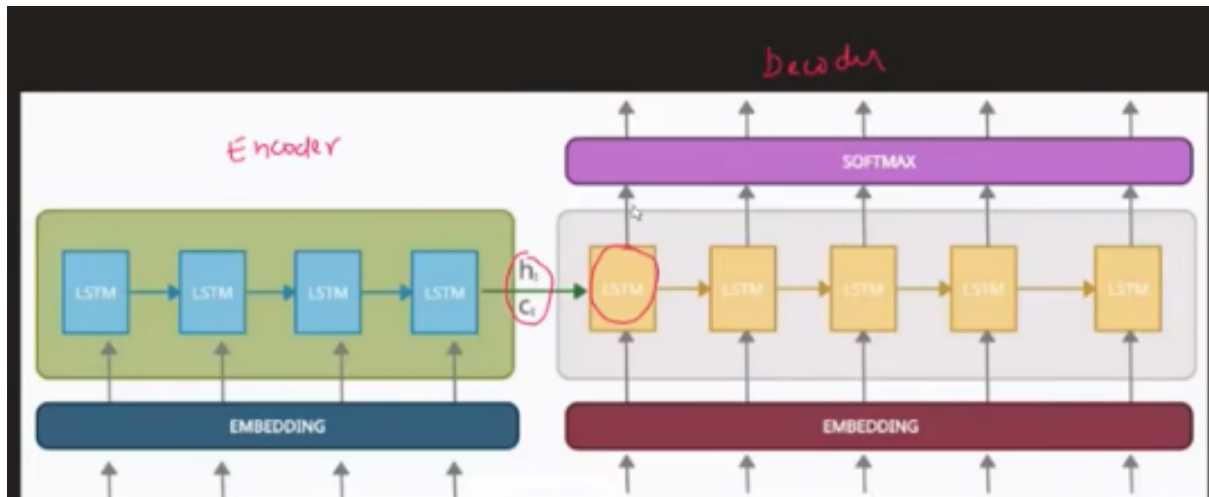


Image 2 (the clean, labeled one) — Encoder/Decoder side by side

- **Left green block = Encoder.** Bottom dark-blue bar = **Embedding layer** — every input word is first converted into its embedding vector before entering the LSTM. Above it, 4 blue boxes labeled **LSTM** connected left→right by arrows — this is a single layer of LSTM cells, each one processing one word (one **timestep**) and passing its hidden state (h) and cell state (c) to the *next* LSTM cell in the same row.
- The **red circle** at the boundary between encoder and decoder = exactly the handoff point described above: the encoder's *final* h and c (after processing the last input word) become the decoder's *initial* h and c .
- **Right yellow/orange block = Decoder.** Same structure mirrored: an Embedding layer at the bottom (for the decoder's own input words — either <START>/previous predictions during inference, or ground-truth words during teacher forcing), a row of LSTM cells left→right (one per output timestep), and a **SOFTMAX** layer on top that converts each timestep's LSTM output into a probability distribution over the vocabulary.

<START> and <END> — why we need them at all

Without them, the decoder has no way to know:

- **When to start generating** — <START> is essentially "GO" signal, an artificial first input since there's no "real" previous word to feed at timestep 1.
- **When to stop generating** — <END> is the model's way of saying "I'm done," since output length is variable and unknown ahead of time. Without it, the model would

keep generating words indefinitely (or you'd need to hardcode a max length, which is a much cruder solution).

The Softmax Output Layer

Your question: "We add a softmax layer in the decoder having nodes equal to total vocabulary — is that right? And why?"

Yes, that's correct. At every decoder timestep:

1. The LSTM cell processes the current input and produces a hidden state.
2. That hidden state is passed through a **Dense (fully connected) layer** whose output size = **size of the vocabulary** (e.g., 50,000 if your vocabulary has 50,000 possible words).
3. A **Softmax activation** converts those raw scores into a **probability distribution over the entire vocabulary** — i.e., "here's how likely each possible word is to be the correct next word."
4. The word with the **highest probability** is picked as the predicted output for that timestep (this is called *greedy decoding* — more advanced systems use *beam search* instead, which we can cover separately if useful).

Why the whole vocabulary? Because the decoder isn't picking from a small fixed set — it needs to be able to output *any* word in the target language, so the output layer must have exactly one "slot" (probability) per possible word.

5. Training the Encoder–Decoder — Full Worked Example

Let's translate: "I am fine" → "Je vais bien" (a toy English→French example).

Step 1 — Add boundary tokens to target: Target becomes: <START> Je vais bien <END>

Step 2 — Encoder forward pass (word by word):

Timestep	Input word	What happens
t1	"I"	embedding("I") → LSTM cell → updates h1, c1
t2	"am"	embedding("am") → LSTM cell (using h1, c1) → updates h2, c2
t3	"fine"	embedding("fine") → LSTM cell (using h2, c2) → updates h3, c3 (final)

After t3, h3, c3 = the **context vector**. The encoder has now "read and understood" the whole sentence.

Step 3 — Decoder forward pass (training, with teacher forcing): Decoder's LSTM starts with h3, c3 as its initial state.

Timestep	Decoder input	LSTM output → Softmax predicts	Actual target
d1	<START>	"Je" (hopefully)	"Je"
d2	"Je" (<i>true word, not model's guess</i>)	"vais"	"vais"
d3	"vais" (<i>true word</i>)	"bien"	"bien"
d4	"bien" (<i>true word</i>)	<END>	<END>

Step 4 — Loss calculation: At each decoder timestep, compare the *predicted probability distribution* against the *actual correct word* (one-hot encoded) using **categorical cross-entropy** (explained fully below). Sum/average the loss across all 4 timesteps to get the total loss for this training example.

Step 5 — Backpropagation (through time): Compute the gradient of the total loss with respect to every parameter in the network (all LSTM weights, embedding weights, softmax weights) — this tells us how much each parameter contributed to the error and in which direction to nudge it.

Step 6 — Parameter update: An optimizer (e.g., Adam, SGD) uses those gradients and the learning rate to update every weight slightly, so next time the model sees a similar sentence, its predictions get a little closer to correct.

This entire 6-step process repeats for thousands/millions of sentence pairs until the model generalizes well.

6. Teacher Forcing — Explained Properly

Your question: *"Even if there's a mistake in the output of one timestep, we will send the correct input in the next timestep? Explain properly."*

Yes — exactly right, and here's why it exists.

During training, the decoder generates one word at a time, and normally the *next* input should be "whatever the decoder just predicted" (since that's what happens at inference/real usage time). But early in training, the model is bad — it might predict garbage at timestep 2, and if you feed *that* garbage into timestep 3, the error compounds and the model spends the rest of the sequence trying to recover from a bad start rather than learning the actual next-word patterns.

Teacher forcing's fix: During training, *regardless of what the decoder actually predicted*, we feed the **ground-truth (correct) word** as the input to the next timestep.

Example: Suppose at d2, the true word is "vais" but the model incorrectly predicted "suis." With teacher forcing, we still feed **"vais"** (the correct word) into d3 — not the model's wrong guess "suis." This way, every timestep gets trained on a "clean" signal, learning is faster and more stable, and the loss at each step directly reflects that step's own prediction quality rather than being polluted by an earlier mistake.

Trade-off: This creates a mismatch between training (always sees correct previous words) and inference (only has its own, possibly wrong, previous predictions to work with) — known as **exposure bias**. This is why techniques like *scheduled sampling* (gradually mixing in the model's own predictions during training) exist as refinements, though the vanilla Sutskever et al. approach uses full teacher forcing.

7. Loss Function — Categorical Cross-Entropy

Your point: *"Categorical cross-entropy because it's multiclass classification — since we predict each word at each timestep."*

Correct. Here's the full picture: at every single decoder timestep, the model isn't doing regression or binary classification — it's choosing **one word out of the entire vocabulary** (e.g., "which of these 50,000 words comes next?"). That is precisely a **multi-class classification problem**, and categorical cross-entropy is the standard loss for exactly that setup.

For one timestep:

$$\text{Loss} = -\sum (\text{true_probability_of_word_i} \times \log(\text{predicted_probability_of_word_i}))$$

Since the true label is one-hot (the correct word has probability 1, everything else 0), this simplifies to just: $\text{Loss} = -\log(\text{predicted probability assigned to the correct word})$. If the model assigned high probability to the correct word, loss is small (close to 0). If it assigned low probability to the correct word, loss is large — punishing confident wrong predictions much more heavily than uncertain ones.

The **total sequence loss** is the sum (or average) of this cross-entropy loss across every decoder timestep in that training example.

8. Improvement #1 — Using Embeddings Properly

Your point: *"Embeddings are low-dimensional and dense (no zeros), a summary of a word."*

Correct — and it's worth contrasting this with what came before: **one-hot encoding**, where each word is represented as a huge vector (size = vocabulary) with a single 1 and thousands of 0s. That's *sparse* (mostly zeros), *high-dimensional* (as big as your vocabulary), and — critically — carries **no meaning**: "cat" and "dog" are just as "different" as "cat" and "car" under one-hot encoding, mathematically speaking.

Embeddings fix this by representing each word as a small, dense vector (e.g., 100 or 300 numbers, no forced zeros) learned such that semantically similar words end up close together in that vector space.

Worked example (simplified, illustrative numbers):

"I" → [0.21, -0.05, 0.88, ...] (100-dim vector)
"am" → [0.11, 0.42, -0.09, ...]
"fine" → [0.67, -0.31, 0.15, ...]

Each of these vectors is fed into the encoder's LSTM one at a time, *instead of* raw one-hot vectors — this is what happens right at the purple "Embedding Layer" box in both diagrams, **before** anything touches the LSTM cells.

Two ways to get embeddings:

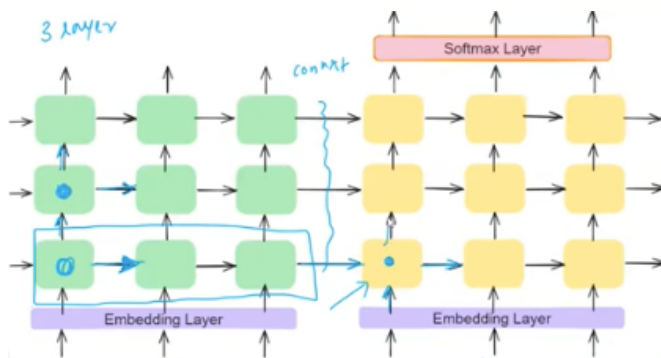
1. **Pretrained embeddings** (Word2Vec, GloVe) — trained separately on huge text corpora beforehand, capturing general language patterns (e.g., "king – man + woman ≈ queen"). You just plug these in as fixed or fine-tunable starting weights. Good when you have limited training data.
2. **Train your own embedding layer** — let the embedding matrix be a trainable part of your Seq2Seq network, learned end-to-end for your specific task/dataset. Good when you have plenty of task-specific data, since the embeddings become tailored to your exact use case.

9. Improvement #2 — Deep (Stacked) LSTMs

Your point: *"Whenever we increase parameters in deep learning, capacity increases to find more minute patterns."*

Exactly right, and here's the fuller picture connecting to the diagram explanation above:

- **Single-layer LSTM:** one row of cells. It can capture reasonably good sequence patterns, but has limited capacity to represent very complex or hierarchical structure in language.
- **Deep/stacked LSTM (e.g., 3–4 layers):** stacking layers lets the network build a **hierarchical representation** — similar to how in CNNs, early layers learn edges and later layers learn complex shapes/objects. In stacked LSTMs:
 - Lower layers tend to pick up on more local/surface patterns (word-level relationships, basic grammar).
 - Higher layers build on that to capture more abstract, longer-range, sentence-level meaning.
- This is *why* the original Sutskever et al. paper reported using **4 LSTM layers** in both the encoder and decoder — they found depth measurably improved translation quality, especially on longer, more complex sentences.
- The trade-off: more layers = more parameters = more capacity to model minute patterns, but also more computation, more risk of overfitting on small datasets, and harder/slower training.



This diagram is showing the **same encoder-decoder idea but stacked in multiple layers**, which is the "deep LSTM" concept you asked about. Here's how to read it:

- **Horizontal axis (left → right) = TIME.** Each column of boxes represents one **timestep** — i.e., one word position in the sequence. Word 1 goes through column 1, word 2 through column 2, and so on. The arrows going left-to-right *within a row* represent the hidden state being carried forward in time from one word to the next.
- **Vertical axis (bottom → top) = LAYERS (depth).** The bottom row of boxes takes the raw embeddings as input. Its output (hidden state at each timestep) doesn't just move forward in time — it *also* moves **upward** to feed into the next LSTM layer directly above it. That second layer processes the *first layer's output* the same way the first layer processed the raw embeddings.
- The **purple "Embedding Layer"** at the very bottom is the entry point — raw words converted to vectors.
- The **orange "Softmax Layer"** at the very top is the exit point — after passing through all the stacked LSTM layers, the final layer's output goes to softmax to produce word probabilities.
- The **"context"** annotation and the arrow connecting the two blocks marks exactly the same handoff as in Image 2 — encoder's final states → decoder's initial states — just now happening **at every stacked layer**, not only once. So if you have 3 LSTM layers, you're actually passing 3 separate (h, c) pairs from encoder to decoder, one per layer (layer 1's final state → decoder layer 1's initial state, and so on).

So what does data movement actually look like?

Picture a grid: **rows = layers, columns = timesteps.**

```

Layer 3: [LSTM]→[LSTM]→[LSTM]→[LSTM]   ← final row feeds Softmax
      ↑   ↑   ↑   ↑
Layer 2: [LSTM]→[LSTM]→[LSTM]→[LSTM]
      ↑   ↑   ↑   ↑
Layer 1: [LSTM]→[LSTM]→[LSTM]→[LSTM]
      ↑   ↑   ↑   ↑
Embedding: word1 word2 word3 word4
  
```

- **Left-to-right arrows** = information flowing *through time* (this word's processing depends on the memory carried from the previous word in the *same* layer).
- **Bottom-to-top arrows** = information flowing *through depth* (each layer refines what the layer below it extracted — layer 1 might capture basic word-level patterns, layer 2 might capture phrase-level patterns, layer 3 might capture sentence-level meaning).

A **single-layer LSTM** is just that bottom row alone — words go in, hidden state passes left to right, and the *last* box's output goes straight to softmax (no vertical movement at all).

A **deep/stacked LSTM (3–4 layers)** adds more rows on top. At *each timestep*, the box in layer 1 finishes its computation, and its output becomes the **input** for the box directly above it in layer 2 at the *same* timestep — which then also receives its own left-to-right memory from layer 2's previous timestep. Every box therefore receives **two inputs**: one arriving from the left (its own layer's memory of earlier words) and one arriving from below (the more "raw" representation of the current word from the layer beneath it).

10. Improvement #3 — Reversing the Encoder Input

Your question: *"Reverse the encoder input, another improvement — how?"*

One of the more surprising tricks from the original paper: instead of feeding the input sentence in normal order, they fed it **reversed**.

Example: Instead of feeding "I am a student" as $I \rightarrow am \rightarrow a \rightarrow student$, they fed it as $student \rightarrow a \rightarrow am \rightarrow I$.

Why this helps: Remember, the decoder's first predicted word depends most heavily on the *most recent* information the encoder processed (since LSTM hidden state is most strongly influenced by recent inputs due to the vanishing gradient issue — even LSTMs aren't perfectly immune to it over long sequences). By reversing the input, the **first word of the source sentence ends up right next to the beginning of the target sequence generation**, shortening the effective "distance" the gradient/ information has to travel between corresponding words. This introduces many short-range dependencies between input and output even though the sentence overall is long, which made optimization notably easier and improved translation results in practice — especially for language pairs where corresponding words/structure tend to align near the *start* of both sentences.

Your note that it "works well in only certain languages where there is more context in the start words" is a fair nuance — the benefit of reversing is most pronounced when sentence structure across the two languages roughly aligns front-to-front; it's a practical trick rather than a universal law, and later architectures (attention, Transformers) made this trick largely unnecessary since attention lets the decoder directly access *any* encoder timestep, not just the most recent one.

11. Evaluation Metric — BLEU Score

Your question: *"Metric used to check efficiency of translation is BLEU score — tell me more."*

BLEU (Bilingual Evaluation Understudy) is the standard metric for evaluating machine-generated translations against one or more human reference translations.

Core idea: Compare **n-grams** (overlapping sequences of n consecutive words) in the model's output against the reference translation(s), and measure how much they overlap.

- **1-gram precision:** what fraction of individual words in the model's output also appear in the reference?
- **2-gram, 3-gram, 4-gram precision:** same idea but for pairs/triples/quadruples of consecutive words — this captures whether the model got not just the right words, but the right *word order/phrasing*.

Example:

- Reference: "the cat is on the mat"
- Model output: "the cat is on the rug"

Most unigrams match ("the," "cat," "is," "on," "the" — high 1-gram precision), but several 4-grams won't match exactly ("cat is on the" ✓, but "is on the rug" X) — so BLEU score drops as n increases, correctly reflecting that word choice/order is imperfect even though individual words are largely right.

Two important adjustments BLEU includes:

1. **Clipping:** prevents a model from gaming the score by repeating a common word many times (e.g., outputting "the the the the" wouldn't get credit for 4 "the"s if the reference only has 2).
2. **Brevity penalty:** penalizes outputs that are *too short* — otherwise a model could get artificially high precision just by outputting very few, "safe" words.

Final BLEU score: typically a combination (geometric mean) of 1-gram through 4-gram precision, scaled 0 to 1 (or 0–100), multiplied by the brevity penalty. Higher = better match to human reference translations.

Limitation worth knowing (commonly asked in interviews): BLEU only checks surface-level word overlap — it doesn't understand meaning, so a perfectly valid paraphrase with different wording can score poorly, and BLEU doesn't capture fluency or grammatical correctness directly either. It's a useful, fast, automatic proxy — not a perfect judge of translation quality.

12. Interview Q&A — Core Concepts

Q1: Why can't a single LSTM be used directly for Seq2Seq tasks like translation? A: A single LSTM produces one hidden state/output per input timestep — it's built to *encode* a sequence, not to *generate* an independently-lengthed new sequence. Seq2Seq needs two separate roles: one network to compress/understand the input (encoder), and another to generate output word-by-word, conditioned on that understanding and its own prior outputs (decoder) — hence the encoder-decoder split.

Q2: What exactly is passed from the encoder to the decoder? A: The encoder's final hidden state and cell state (h, c) — collectively called the context vector — become the decoder's *initial* hidden and cell state. In a stacked LSTM, this handoff happens per layer.

Q3: What's the difference between the context vector and attention (if you know it)? A: The vanilla Sutskever architecture compresses the *entire* input sequence into a single fixed-size context vector, which becomes a bottleneck for long sentences (information gets lost/diluted). Attention mechanisms (introduced later, e.g., Bahdanau 2014) let the decoder look back at *all* encoder hidden states at every decoding step, weighting them by relevance — removing the single-vector bottleneck.

Q4: Why use teacher forcing during training? A: It stabilizes and speeds up training by always feeding the ground-truth previous word into the decoder instead of the model's own (possibly wrong) prediction, preventing early mistakes from compounding across timesteps.

Q5: What is exposure bias, and how does it relate to teacher forcing? A: Exposure bias is the train/inference mismatch caused by teacher forcing: during training the decoder always sees correct previous words, but at inference it only has its own predictions to condition on — so it may behave differently (worse) than during training, especially if early errors occur.

Q6: Why is categorical cross-entropy the right loss function here, and not MSE? A: Each decoder timestep is a multi-class classification problem — picking one word out of the entire vocabulary via a softmax probability distribution. Cross-entropy directly measures how far the predicted probability distribution is from the true one-hot distribution, whereas MSE would treat word indices as if they were continuous numeric values, which makes no sense for categorical labels.

Q7: Why reverse the input sequence in the original Sutskever et al. paper? A: To reduce the effective distance between corresponding words at the start of the input and start of the output, making it easier for gradients/information to connect related words, especially for sentence pairs with front-aligned structure. It's a practical optimization trick, later made largely obsolete by attention mechanisms.

Q8: What's the fundamental limitation of the vanilla (no-attention) encoder-decoder architecture? A: Compressing an entire (possibly long) input sequence into one fixed-size context vector is a bottleneck — performance degrades noticeably as input sentences get longer, since a lot of detail simply can't be preserved in one fixed-length vector. This is the core motivation for the attention mechanism and, eventually, Transformers.

Q9: Why does adding more LSTM layers (going deep) help, and what's the trade-off? A: More layers let the network build hierarchical representations (lower layers = more local patterns, higher layers = more abstract/long-range patterns), increasing model capacity to capture subtle patterns. Trade-off: more parameters means more compute, slower training, and higher risk of overfitting without enough data.

Q10: What are the practical limitations of BLEU score as an evaluation metric? A: It only measures n-gram surface overlap with reference translations — it penalizes valid paraphrases, doesn't directly assess grammaticality/fluency, and can be gamed without truly capturing meaning-level translation quality. It's a fast, automatic, widely-used proxy, not a perfect measure.

Quick Recap Cheat-Sheet

- **ANN** → good for tabular, no memory of order → fails on sequences
- **CNN** → good for spatial (image) data → fails on order-dependent text
- **RNN/LSTM/GRU** → good for single sequences with order → can't generate a new, independently-lengthed output sequence
- **Encoder-Decoder** → encoder compresses input into a context vector; decoder generates output word-by-word starting from that context, using `<START>/<END>` tokens and a softmax over the vocabulary at every step
- **Timestep** = one position in a sequence (left→right on the diagram)
- **Layer/depth** = stacking multiple LSTM rows (bottom→top on the diagram) for richer, hierarchical representations
- **Teacher forcing** = feed the *correct* previous word during training, regardless of what the model actually predicted
- **Loss** = categorical cross-entropy, applied per timestep, since each timestep is a multi-class word-prediction problem
- **Key improvements**: pretrained/learned embeddings, deep stacked LSTMs, reversed input ordering
- **Evaluation**: BLEU score, based on n-gram overlap with reference translations

Day : Attention Mechanism — Fixing the Encoder-Decoder Bottleneck

Continuation of your Seq2Seq Encoder-Decoder notes

1. The Problem Attention Solves

In the vanilla encoder-decoder architecture (previous notes), the entire input sentence — no matter how long — gets compressed into **one single fixed-size context vector** (the encoder's final hidden + cell state). The decoder then has to generate the *entire* output relying only on that one vector.

Why this is a problem:

- A long sentence carries a lot of information (many words, clauses, relationships). Squeezing all of that into one fixed-size vector means a lot of detail simply gets lost or diluted — like trying to summarize an entire chapter in a single sentence and then writing a book report using *only* that one sentence.
- This puts **unnecessary pressure on the encoder** — it has to anticipate and pack in everything the decoder might ever need, all at once, before the decoder has even started generating anything.

- The decoder, in turn, is stuck reusing the *same* summary at every single output timestep — even though different output words logically depend on different parts of the input.

Key insight: At any *given* decoder timestep, you don't actually need the meaning of the *whole* input sentence — **you need just the relevant subset of words** for that specific word you're generating right now.

Example: Translating "turn off the lights" → "لائیٹ بند کرو" (Urdu).

- When generating "لائیٹ" (lights), the decoder mainly needs to focus on the input word "lights."
- When generating "بند" (off), it mainly needs to focus on "off."
- When generating "کرو" (do/turn), it mainly needs "turn."

Forcing the decoder to rely on one single blended summary for *all three* of these decisions wastes information and makes translation less accurate, especially for longer sentences. It would be far better if the decoder could **dynamically shift its focus** to different parts of the input at each output step — that's exactly what attention does.

2. The Reading Analogy

Think about how you read a book: even though the whole page is in front of your eyes, your **attention** is only on one word or sentence at a time — you don't process the entire page equally at once. You focus on a small relevant chunk, extract meaning, then move your focus to the next chunk.

The attention mechanism gives the decoder this same ability: instead of being handed one static summary of the entire input, it can **look back at the whole input at every single output step** and decide **how much weight to give each part of it**, based on what it's trying to generate right now.

3. What Is the Attention Mechanism, Concretely?

At its core, attention produces a vector often called the **context vector** (here, c_i for decoder timestep i) — built as a **weighted combination of all the encoder's hidden states**, not just the final one. here we send separate context vector at each decoder timestep not a final one.

- Every encoder timestep produces its own hidden state: $h_1, h_2, h_3, h_4, \dots$ (one per input word).
- Instead of throwing away h_1, h_2, h_3 and keeping only the last one (h_4), like the vanilla architecture does, attention **keeps all of them** available to the decoder.
- At each decoder timestep, attention computes a **weight (alpha)** for each encoder hidden state, saying "how relevant is this input word right now?"
- The context vector for that decoder timestep is then the **weighted sum** of all encoder hidden states, dominated by whichever ones got the highest weight.

- The dimensionality of this context vector matches the dimensionality of a single encoder hidden state h_i — because it's built as a weighted *sum* of vectors that all share that same shape.

4. Three Inputs to the Decoder (Instead of Two)

In the **vanilla** encoder-decoder, the decoder at each timestep effectively receives:

1. Its own previous hidden/cell state
2. The previous output word (via teacher forcing or its own prediction)

In the **attention-based** decoder, there's a **third input added**: 3. A **dynamically computed context vector c_i** — different at every single decoder timestep, built specifically to highlight the most relevant encoder states for generating *this particular* output word.

Purpose of this third input: it tells the decoder **which encoder timestep(s) are most useful right now**, so the decoder isn't relying on one generic, blended summary for the entire sentence — it gets a fresh, targeted summary at every step.

5. How the Decoder Decides Which Encoder Timesteps Matter

This happens by assigning a **weight (alpha)** to each encoder hidden state, at every decoder timestep. Whichever encoder state's weight **dominates** (is largest) is effectively the one the model is "paying attention to" for that output word. The context vector c_i is then just the weighted sum:

$$c_i = \sum (\alpha_{ij} \times h_j) \quad \text{for all encoder timesteps } j$$

So if α_{i2} (weight on encoder hidden state h_2) is much larger than the others, c_i will be dominated by h_2 's information — meaning the decoder is mostly "looking at" input word 2 right now.

6. Calculating the Alphas — Step by Step

Your question: "What quantities does alpha depend on — h_i (hidden state) and s_i (previous hidden state of decoder)? Why does it depend on s_i ?"

Alpha depends on **two things**:

1. **The encoder's hidden state at a given input timestep (h_j)** — what that specific input word "means."
2. **The decoder's *previous* hidden state (s_{i-1})** — what the decoder has generated/understood *so far* in the output sequence.

Why does it need s_{i-1} at all? Because "relevance" isn't fixed — it changes depending on what the decoder is currently trying to produce. Which input words matter **changes at**

every output step. At decoder step 1 (generating "لائيٽ", the word "lights" matters most. At decoder step 2 (generating "بند", the word "off" matters most. The *only* way the model can know "where the decoder currently is" in this generation process is by looking at the decoder's own current state (s_{i-1}), carrying forward everything it has generated up to this point). Without s_{i-1} in the picture, the attention weights would have no way to *shift* across timesteps — they'd be static, which defeats the whole purpose.

The Formula

For decoder timestep i and encoder timestep j :

Step 1 — Alignment score (raw, un-normalized relevance score):

$$e_{ij} = a(s_{i-1}, h_j)$$

Here, a is **not** a fixed mathematical formula like a plain dot product (though simpler "multiplicative" attention variants do just use a dot product) — in the original Bahdanau attention design, a is itself a **small feedforward neural network** (a Dense/fully-connected layer) that takes s_{i-1} and h_j as input and outputs a single scalar score. A common concrete version:

$$e_{ij} = V^T \cdot \tanh(W_s \cdot s_{i-1} + W_h \cdot h_j)$$

where W_s , W_h , and V are all **learned weight matrices/vectors** — trained jointly with the rest of the network via backpropagation, exactly like every other weight in the model. This is why it's a *neural network* acting as the scoring function, not a hardcoded formula — the model learns *for itself* what "relevance" should look like, rather than us defining it by hand.

Step 2 — Normalize into alpha (softmax across all encoder timesteps):

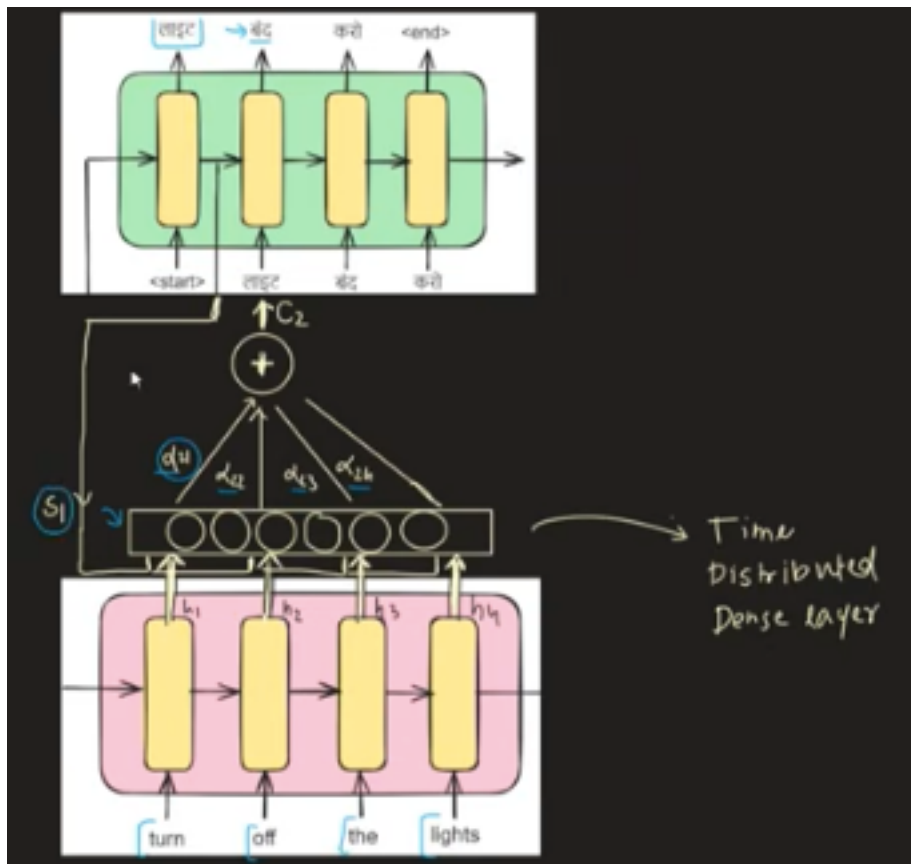
$$\alpha_{ij} = \text{softmax}(e_{ij}) = \exp(e_{ij}) / \sum_k \exp(e_{ik})$$

This ensures all the alpha weights for a given decoder timestep i **sum to 1** — turning raw scores into a clean probability-like distribution over "how much attention to pay to each input word."

Step 3 — Weighted sum to get the context vector:

$$c_i = \sum_j (\alpha_{ij} \times h_j)$$

In plain words: *alpha is an alignment (similarity) score that tells us how strongly a specific encoder timestep matters for producing the output at a specific decoder timestep* — computed by a small trainable neural network that looks at both "what this input word meant" (h_j) and "what the decoder currently understands/has produced so far" (s_{i-1}), then normalized with softmax so the weights are comparable and sum to 1 across the whole input sentence.



7. Explaining the Diagram, Step by Step

Reading the picture from **bottom to top** (this mirrors the actual data flow):

- ① **Encoder (pink block, bottom):** The input sentence "**turn off the lights**" is fed word by word into the encoder LSTM. Each timestep produces its own hidden state: h_1 (for "turn"), h_2 (for "off"), h_3 (for "the"), h_4 (for "lights"). Unlike the vanilla architecture, **all four** hidden states are kept — none are thrown away after encoding finishes.
- ② **Time Distributed Dense Layer (the small block with d_{21} , d_{22} , d_{23} , d_{24}):** This is the **alignment scoring network** — the neural network $a(s_{i-1}, h_j)$ described above. It's applied to **every encoder hidden state** (hence "time distributed" — the same small dense network is reused across all timesteps), combined with the decoder's previous hidden state s_1 . For decoder timestep 2, this produces one raw alignment score per encoder timestep: d_{21} , d_{22} , d_{23} , d_{24} (these are the e_{2j} values — one per input word, showing how relevant each of "turn / off / the / lights" is for generating the *second* output word, which will be "بند"). These scores then get passed through softmax to become the normalized α_{2j} weights.
- ③ **The "+" circle producing c_2 :** This is the **weighted sum step** — each encoder hidden state h_j gets multiplied by its corresponding attention weight α_{2j} , and all four results are summed together into a single vector: c_2 , the **context vector specifically for decoder timestep 2**. Since "off" is the most relevant word for this particular output step, α_{22} would dominate, meaning c_2 is mostly built from h_2 .

④ **s1 feeding into the attention block:** s_1 is the decoder's hidden state from the **previous** timestep (timestep 1, which generated "لائيٹ"). It's fed *into* the attention scoring network alongside each encoder hidden state — this is exactly the "why does alpha depend on s_{i-1} " mechanism from Section 6: it lets the attention weights *shift* depending on where the decoder currently is in generating the output.

⑤ **Decoder (green block, top):** The decoder LSTM receives, at each timestep: (a) its previous hidden state, (b) the previous output word (starting with `<start>`, then feeding back its own generated words — "کرو" → "بند" → "لائيٹ"), and (c) the freshly-computed context vector (c_2 for this timestep, and a differently-weighted context vector at every other timestep). Combining these, it generates the next output word. The sequence continues left to right — `<start>` → "کرو" → "بند" → "لائيٹ" → `<end>` — with the **attention weights being recalculated fresh at every single decoder timestep**, letting the model "re-focus" on a different part of "turn off the lights" each time.

In short: the diagram shows one full decoder timestep's worth of attention — take the decoder's previous state (s_1), score it against every encoder hidden state using a small trainable dense network, normalize those scores into attention weights, blend the encoder states accordingly into c_2 , and feed that dynamic context vector into the decoder LSTM to help produce the next word. This entire block repeats at every decoder timestep, each time producing a *new* context vector tailored to what's being generated at that moment.

Quick Recap Cheat-Sheet

- **Problem fixed:** one static context vector forces the encoder to over-compress long sentences and forces the decoder to reuse the same summary for every output word.
- **Fix:** keep *all* encoder hidden states, and let the decoder dynamically build a fresh, weighted context vector at every output timestep.
- **Alpha (attention weight):** an alignment/similarity score between a decoder's previous state and each encoder hidden state, computed by a small trainable neural network, then normalized via softmax.
- **Why alpha depends on s_{i-1} :** relevance of input words changes as the decoder progresses — without the decoder's own current state as input, attention couldn't shift focus across output timesteps.
- **Context vector c_i :** the weighted sum of all encoder hidden states, using that timestep's alpha weights.
- **Decoder now gets 3 inputs per timestep:** previous hidden state, previous output word, and the fresh context vector c_i .

=====

Day Type of attention:

Types of Attention — Additive (Bahdanau) vs Multiplicative (Luong)

Research reference: Neural machine translation by jointly learning to align and translate

Continuation of your Attention Mechanism notes

1. Where This Fits In

From the previous notes, you already know the core attention formula:

$$c_i = \sum_j (\alpha_{ij} \times h_j)$$

and that α_{ij} comes from normalizing an **alignment score** e_{ij} via softmax. What we hadn't covered yet is: **how exactly is that alignment score e_{ij} calculated?** This is the "word-to-word similarity score" you're asking about — *how much does encoder word j matter for decoder step i ?* There are two competing ways to compute it, and they lead to two named variants of attention.

2. Additive Attention (Bahdanau, 2014)

Formula:

$$e_{ij} = a(s_{(i-1)}, h_j)$$

where a is a **small feedforward neural network (FFN)** — typically something like:

$$e_{ij} = V^T \cdot \tanh(W_s \cdot s_{(i-1)} + W_h \cdot h_j)$$

How it works: The decoder's *previous* hidden state $s_{\{i-1\}}$ and each encoder hidden state h_j are both passed through learned weight matrices (W_s, W_h), added together, squashed with $\tanh$, and then projected down to a single scalar score via another learned vector v . This whole scoring function has its **own trainable parameters**, separate from the encoder and decoder LSTMs — which is why it's called "additive," since the two inputs are combined by addition inside the network before being scored.

Key detail: it uses $s_{\{i-1\}}$ — the decoder's state from the *previous* timestep (before the current word has been generated).

3. Multiplicative Attention (Luong, 2015)

Formula:

$$e_{ij} = a(s_i, h_j) = s_i \cdot h_j \quad (\text{dot product})$$

How it works: Instead of running the two vectors through a neural network, Luong attention simply takes the **dot product** of the decoder's hidden state and each encoder hidden state

directly. The dot product naturally produces a single scalar — and geometrically, a dot product is high when two vectors point in a similar direction (i.e., are "similar" in meaning), which is exactly what you want a similarity/relevance score to capture. That scalar goes straight into softmax to get α_{ij} , no extra learnable weights involved like FFN in bahdanau (in the simplest "dot" variant).

Key detail: it uses s_i , the decoder's **current** hidden state, not the previous one. This matters because it means Luong attention is scoring relevance using the *most up-to-date* information the decoder has, dynamically adjusting as soon as new information is available, rather than scoring based on slightly older state.

Why use a dot product instead of a neural network? Because adding a full feedforward network (with its own trainable w_s, w_h, v parameters) adds extra parameters that need to be learned which means more computation and slower training. A dot product needs **no extra trainable parameters at all**, so it's computationally cheaper and trains faster, while still giving a mathematically sound similarity measure.

4. Additive vs Multiplicative — Side-by-Side

	Additive (Bahdanau)	Multiplicative (Luong)
Decoder state used	Previous state s_{i-1}	Current state s_i
Scoring mechanism	Small feedforward neural network	Direct dot product
Extra trainable parameters	Yes (w_s, w_h, v)	No (in the basic "dot" version)
Computation cost	Higher (extra matrix multiplications + tanh)	Lower (single dot product)
Training speed	Slower	Faster
Info freshness	Slightly "older" (uses state before current step)	More "current"/dynamic
Experimental performance	Good, but generally outperformed	Generally better in practice

Why is Luong experimentally better despite being simpler? Two reinforcing reasons:

1. Fewer parameters to learn means less risk of overfitting and faster, more stable convergence, the model isn't spending capacity learning an extra scoring network when a dot product already captures similarity well.
2. Using the **current** decoder state s_i (rather than the previous one) means the relevance scoring is based on more up-to-date information about what the decoder is about to generate, making the attention slightly more precise.

5. Step-by-Step Execution — Worked Example

Using the same example as before: "turn off the lights" → "لائٹ بند کرو" (Urdu).

Encoder hidden states after processing each input word: h_1 (turn), h_2 (off), h_3 (the), h_4 (lights)

We're computing the context vector for **decoder timestep 1** (generating "لائيٽ"), so the decoder's relevant hidden state here is s_1 (or s_0 , depending on indexing convention — the diagram uses s_1).

Step 1 — Compute raw alignment scores (using whichever attention type — here shown as the dot-product/Luong style, matching the diagram's $s_1 \cdot h_1$ notation):

```
 $s_1 \cdot h_1 \rightarrow e_{11}$   
 $s_1 \cdot h_2 \rightarrow e_{12}$   
 $s_1 \cdot h_3 \rightarrow e_{13}$   
 $s_1 \cdot h_4 \rightarrow e_{14}$ 
```

Each of these is just a **single number** — how well the decoder's current state "matches" that particular encoder hidden state.

Step 2 — Normalize with softmax:

```
 $[e_{11}, e_{12}, e_{13}, e_{14}] \rightarrow \text{softmax} \rightarrow [\alpha_{11}, \alpha_{12}, \alpha_{13}, \alpha_{14}]$ 
```

These four alpha values now sum to 1, representing a clean "how much attention to pay to each input word" distribution.

Step 3 — Weighted sum to get the context vector:

```
 $c_1 = (\alpha_{11} \times h_1) + (\alpha_{12} \times h_2) + (\alpha_{13} \times h_3) + (\alpha_{14} \times h_4)$ 
```

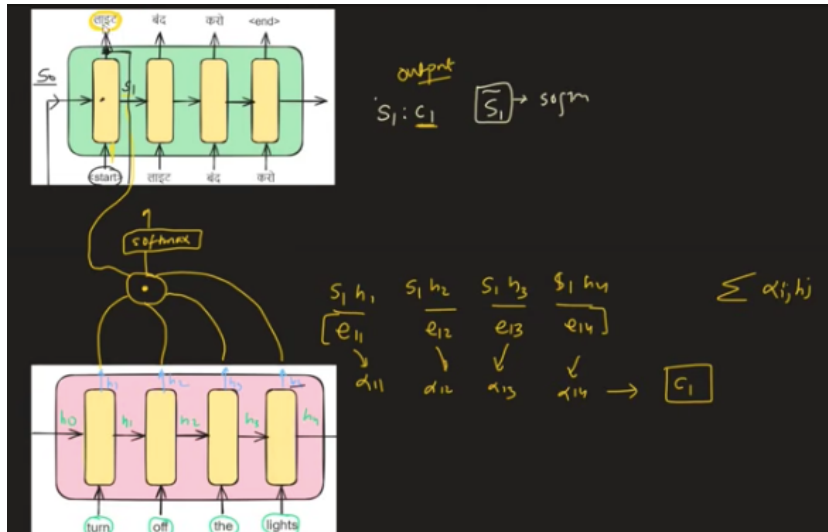
Step 4 — Combine with the decoder's own output pathway: c_1 gets **concatenated** with the decoder's previous output (or hidden state, depending on the exact architecture variant), and this combined vector is what actually gets used to produce the prediction — typically passed through a final dense softmax layer over the vocabulary to output the probability distribution for the next word.

This entire 4-step process repeats fresh at **every decoder timestep** — a new c_i , built from a new set of alphas, computed using that timestep's decoder state.

Why concatenate instead of also just adding/blending them? Because c_1 and s_1 represent two *different kinds* of information — c_1 is "what's relevant from the input sentence right now," s_1 is "what the decoder itself has understood/generated so far." Concatenating **keeps both pieces of information fully intact and separate**, and then a learned Dense layer (with its own weights) figures out *how* to combine them best — rather than us forcing a fixed blend ourselves. That combined vector is then passed through a Dense + Softmax layer to actually predict the next word probability distribution.

One-line summary of the distinction:

- **Weighted sum** ($\alpha_{ij} \times h_j$) $\rightarrow$ blends multiple vectors of the *same* meaning-type into one, same size out.
- **Concatenation** ($[C_1 ; s_1]$) $\rightarrow$ stitches two *different* vectors together, output is *longer*, and a later layer learns how to use both pieces.



6. Explaining the Diagram

Bottom (pink) — Encoder: Same as before — "turn," "off," "the," "lights" are fed in one at a time, producing hidden states h_1 through h_4 (the diagram also shows h_0 as an initial/starting hidden state before any word is processed).

Middle (the small circle with "softmax" label and curved arrows): This is the **attention scoring + weighting block**. The curved arrows sweeping from each encoder hidden state (h_1 to h_4) into the circle represent each h_j being combined with the decoder's state s_1 (dot product, per Luong-style scoring in this diagram) to produce e_{11} through e_{14} . These get passed through **softmax** (explicitly labeled in the diagram) to produce α_{11} through α_{14} , which are then combined (weighted sum) into the context vector C_1 .

The written-out math beside the diagram:

$S_1 h_1 \rightarrow e_{11} \rightarrow \alpha_{11}$
 $S_1 h_2 \rightarrow e_{12} \rightarrow \alpha_{12}$
 $S_1 h_3 \rightarrow e_{13} \rightarrow \alpha_{13}$
 $S_1 h_4 \rightarrow e_{14} \rightarrow \alpha_{14} \rightarrow C_1$

This is exactly Steps 1–3 above, laid out explicitly — each encoder hidden state paired with the decoder state, scored, normalized, then combined into C_1 .

Top (green) — Decoder: The decoder box shows the output sequence being generated: $\langle \text{start} \rangle \rightarrow$ "کرو" $\rightarrow$ "بند" $\rightarrow$ "لائٹ" $\rightarrow \langle \text{end} \rangle$. The $S_i: C_i$ and $[S_i] \rightarrow \text{softmax}$ notation on the right side of the image is showing that at each decoder timestep, the decoder's hidden state S_i together with the context vector C_i gets passed into a final softmax layer to produce the actual predicted output word — matching Step 4 above. The general summation formula

$\sum \alpha_{ij} \times h_j$ shown on the right is just the compact way of writing the weighted-sum step (Step 3) for any timestep i .

In short, the diagram is showing one complete cycle: encoder hidden states (bottom) $\rightarrow$ scored against the decoder's current state via dot products (middle) $\rightarrow$ normalized into attention weights via softmax $\rightarrow$ combined into a context vector c_i $\rightarrow$ fed upward into the decoder to help generate the next output word (top).

7. Quick Recap Cheat-Sheet

- **Both** additive and multiplicative attention compute the same *final* thing an alignment score e_{ij} , normalized into α_{ij} , combined into c_i they just differ in **how** e_{ij} is computed.
- **Additive (Bahdanau):** $e_{ij} = \text{FFN}(s_{(i-1)}, h_j)$ uses the **previous** decoder state, needs extra trainable parameters, more computation.
- **Multiplicative (Luong):** $e_{ij} = s_i \cdot h_j$ (dot product) uses the **current** decoder state, no extra trainable parameters, cheaper and faster.
- **Luong tends to perform better experimentally** fewer parameters to overfit, and scoring is based on more up-to-date decoder information.
- Execution order is always: **score** $\rightarrow$ **softmax** $\rightarrow$ **weighted sum** $\rightarrow$ **concatenate with decoder output** $\rightarrow$ **final softmax layer** $\rightarrow$ **predicted word**.

8. Interview Q&A

Q1: What's the fundamental difference between additive and multiplicative attention?

A: They differ only in how the alignment score e_{ij} is computed. Additive (Bahdanau) uses a small feedforward neural network combining the decoder's previous state and the encoder hidden state. Multiplicative (Luong) uses a direct dot product between the decoder's current state and the encoder hidden state — no extra learned parameters.

Q2: Why does Luong attention use s_i while Bahdanau uses s_{i-1} ? A: Luong's design intentionally uses the decoder's most current state so the attention scoring reflects the freshest information available, adjusting dynamically at each step, whereas Bahdanau's original formulation scores based on the state before the current word is generated.

Q3: Why is multiplicative attention generally faster to train? A: It has no additional trainable weight matrices — the score is just a dot product, which is computationally cheap and adds zero new parameters, unlike additive attention's feedforward scoring network, which must be learned alongside everything else in the model.

Q4: Is additive attention ever preferable to multiplicative attention? A: Additive attention's extra learnable parameters can, in principle, capture more complex, non-linear relationships between decoder and encoder states than a plain dot product can, which can matter when hidden state dimensions are large or the relationship between states is complex — but in practice, Luong-style attention has generally shown comparable or better results with less computational cost, which is why it's more commonly used as a base for later architectures (including scaled dot-product attention in Transformers).

Q5: How does this connect to the attention used in Transformers? A: The Transformer's "Scaled Dot-Product Attention" is a direct descendant of multiplicative (Luong) attention — same dot-product scoring idea, just extended into Query/Key/Value matrices and scaled by the square root of the key dimension for numerical stability, then used to build the full self-attention mechanism.



Day : Transformers — From Attention to a New Architecture

Reference: "Attention Is All You Need" (Vaswani et al., 2017) Also: "A Comprehensive Survey on Application of Transformers for Deep Learning Tasks"

Problem

A Transformer is a deep learning architecture designed to understand relationships between different parts of a sequence, such as words in a sentence. It was created to overcome the limitations of older sequential models like RNNs and LSTMs, which process one word at a time and struggle with long-range dependencies.

Example: **"The animal didn't cross the street because it was too tired."**

To understand what **"it"** refers to, the model needs information from earlier in the sentence. Older models could forget this information over long sequences.

Step 2: Explain what it actually does

A Transformer takes an input sequence and produces contextual representations for every element in that sequence. Instead of looking at words one by one, it lets every word directly look at every other word to determine which ones are important for understanding its meaning.

For example: Sentence: **The bank approved the loan.** The word **bank** should mean a financial institution.

Sentence: **The fisherman sat on the bank of the river.** Now **bank** should mean the side of a river.

The Transformer figures this out automatically from context

A Transformer is a neural network architecture designed to process sequential data, such as text, by learning the relationships between all elements in the sequence simultaneously. Its core mechanism is self-attention, which allows each word to consider every other word when building its representation. This produces contextual embeddings, enabling the model to understand meaning based on surrounding words. Unlike older models such as RNNs and LSTMs, Transformers process all tokens in parallel, making them faster to train and more effective at capturing long-range dependencies. Because of these advantages, Transformers form the foundation of modern AI systems like GPT, BERT, Claude, and many vision and speech models.

It outputs better representations (contextual embeddings) of the input.

For example,

Input: The bank approved the loan.

Output:

```
The      → contextual vector
bank     → contextual vector
approved → contextual vector
...
```

These vectors contain much richer information than the original embeddings.

1. Why Transformers Had to Happen

You already know attention fixed the "one static context vector" bottleneck. But attention, as covered so far, was still **bolted onto an LSTM-based encoder-decoder**. That leftover LSTM backbone quietly kept two old problems alive:

1. **Sequential processing.** An LSTM must read a sentence **word by word, in order** — word 2 can't be processed until word 1's hidden state is computed, and so on. This is inherently a *loop*, not something you can split across many processors at once. For huge datasets, this made training painfully slow.
2. **No real transfer learning culture yet.** Because there wasn't a clean, scalable, pretrain-once-reuse-everywhere architecture, most serious NLP work meant training a model **from scratch** for each task — which needs huge amounts of task-specific data and compute. That's expensive and limits who can actually build good models.

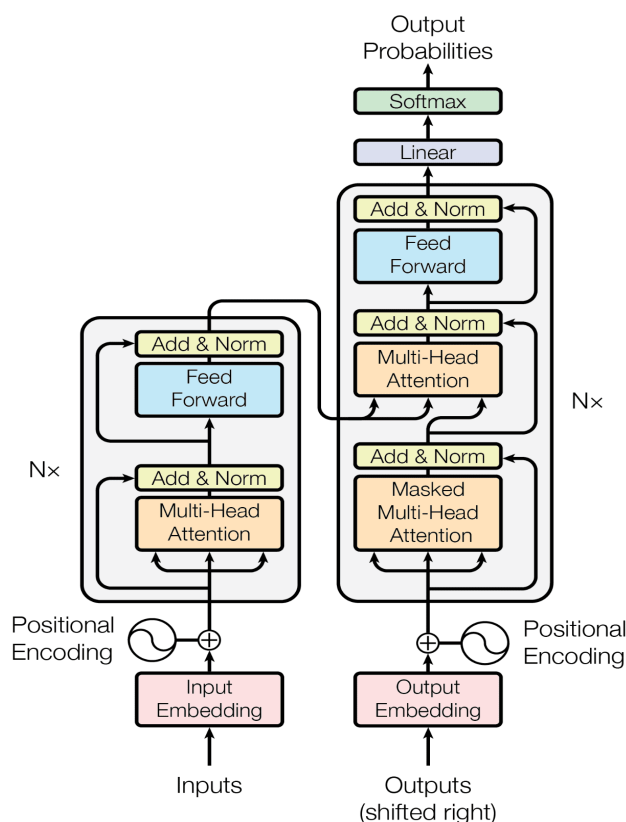
The fix: drop recurrence entirely. Instead of reading one word at a time, look at the **whole sentence at once** and let every word relate directly to every other word simultaneously — this is **self-attention**, and it's the core (only) engine in a Transformer. No LSTM/RNN cells anywhere.

A Transformer isn't just attention.

It is made of several components:

- Input embeddings
- Positional encoding
- Multi-head self-attention
- Feed-forward neural networks
- Residual connections
- Layer normalization

These components work together to produce contextual representations.



2. What Is Self-Attention? (Building the Intuition)

Self-attention = the relationship of one word with every other word in the same sentence, computed all at once, not step-by-step.

Analogy: Compare two ways of reading a sentence to understand it:

- **LSTM (relay race):** Runner 1 reads "The," passes a baton (hidden state) to Runner 2, who reads "animal," updates the baton, passes it to Runner 3, and so on. Everyone waits their turn — strictly sequential.
- **Self-attention (group discussion):** All the words sit in a room together *at the same time*, and each word simultaneously asks every other word, "how relevant are you to me?" — instantly weighing all relationships in parallel, no waiting in line.

Concrete example — why this actually matters, not just for speed:

"The animal didn't cross the street because **it** was too tired."

To understand what "it" refers to, you need to relate "it" to "animal" (not "street"). Self-attention computes exactly this: a **relevance score between "it" and every other word in the sentence** — "it" $\leftrightarrow$ "animal" gets a high score, "it" $\leftrightarrow$ "street" gets a low one. This is the *same* alignment-score-then-weighted-sum idea from your attention notes ($e_{ij} \rightarrow \text{softmax} \rightarrow \alpha_{ij} \rightarrow \text{weighted sum}$), except now it's happening **within a single sentence, between every pair of words**, not just between encoder and decoder.

Why this enables parallel processing: Since every word's relationship to every other word is computed independently (they don't depend on some *previous* word's result finishing first, unlike an LSTM's chain), **the entire sentence can be fed into the encoder all at once**, and all these relationship computations can be done simultaneously on a GPU. This is the single biggest reason Transformers train so much faster than LSTM-based models on large datasets — it's not a minor optimization, it's a structural removal of the sequential bottleneck.

3. Why It Changed So Much (Impact, Briefly)

- **Revolution in NLP** — became the default architecture almost overnight (BERT, GPT, T5, translation, search).
- **Democratizing AI** — pretrained models can be fine-tuned cheaply by anyone, instead of needing to train from scratch.
- **Multimodal capability** — self-attention just needs sequences of vectors, so it extends naturally to image patches, audio, etc. — same mechanism, different tokens.
- **Acceleration of GenAI** — scales well with more data/parameters, so progress compounded fast.
- **Unification across fields:**
 - NLP $\rightarrow$ BERT, GPT
 - Vision $\rightarrow$ ViT (image split into patches, treated like "words")
 - RL $\rightarrow$ **Decision Transformer**: treats a trajectory of (state, action, reward) as a sequence and predicts the next action the way GPT predicts the next word.

Common thread: any problem framed as "predict/relate elements in a sequence" fits.

4. Transfer Learning & Scalability (Why It Trains Well at Scale)

- **Pretrain once, fine-tune many times:** BERT/GPT are pretrained on massive general text, then cheaply fine-tuned on smaller domain-specific data — no need to train from scratch each time.
 - **Parallel training** (from self-attention) + **small, repeatable stacked blocks** + stabilizing design choices (layer norm, residual connections) = training stays fast and stable even at huge scale.
-

5. Advantages / Disadvantages (Quick List)

Advantages: scalability · transfer learning · multimodal input/output · flexible architecture (encoder-only = BERT for understanding, decoder-only = GPT for generation, encoder-decoder = T5 for seq2seq) · strong ecosystem/tooling.

Disadvantages: high compute cost (self-attention cost grows ~quadratically with sequence length) · needs lots of data · overfitting risk · high energy use · hard to interpret · can absorb bias from training data.

8. Clearing Up the Terminology Confusion: Transformer vs LLM vs GPT vs ChatGPT vs BERT

This is genuinely one of the most commonly confused areas, so let's build a clean mental hierarchy:

Transformer (an ARCHITECTURE – a blueprint for a neural network using self-attention)

- ├─ Encoder-only Transformers → e.g., BERT (good at understanding/classifying text)
- ├─ Decoder-only Transformers → e.g., GPT family (good at generating text)
- └─ Encoder-Decoder Transformers → e.g., T5, original translation model (sequence-to-sequence tasks)

LLM ("Large Language Model" – a CATEGORY/description, not a specific architecture)

- Any language model that is very large (billions of parameters) and trained on huge amounts of text. In practice, nearly all modern LLMs are built using the Transformer architecture – but "LLM" describes the *scale and purpose* (a big model that understands/generates language), while "Transformer" describes the *underlying architecture* making that possible.

GPT (a SPECIFIC FAMILY of models – "Generative Pretrained Transformer")

- A decoder-only Transformer architecture, trained to predict the next word, released in versions: GPT-2, GPT-3, GPT-4, etc. GPT models ARE Transformers, and (given their size) ARE LLMs. GPT is a specific brand/family within the broader Transformer/LLM space – not a synonym for either.

ChatGPT (a PRODUCT – an application built on top of a GPT model)

- OpenAI took a GPT model, fine-tuned it specifically for conversational behavior (using techniques like instruction tuning and RLHF – reinforcement learning from human feedback), and wrapped it in a chat interface. ChatGPT is NOT itself a distinct architecture – it's a fine-tuned, product-ified version of an underlying GPT model, packaged for conversational use.

Put plainly, in one line each:

- **Transformer** = the architecture (the engine design).

- **LLM** = a description of scale + purpose (a very large model trained on language) — not tied to one specific architecture, though virtually all current LLMs happen to be Transformers.
- **GPT** = a specific product/model family built using the decoder-only Transformer architecture — one particular kind of LLM.
- **ChatGPT** = a specific fine-tuned, chat-oriented application built on top of a GPT model — the thing you actually talk to.

Analogy: Transformer is like "internal combustion engine" (a general engineering design). LLM is like "a large, powerful car" (a category based on scale/capability). GPT is like a specific car *model line*, e.g., "a Toyota Corolla" (a specific engineered product using that engine design). ChatGPT is like "a Corolla configured and marketed specifically as a taxi, with a driver-assist package added" — a specifically fine-tuned, product-packaged version of that model line for one purpose (conversation).

Other common confusions worth clearing up while we're here:

- **Is BERT an LLM?** Technically, if it's large enough and trained on large text data, yes it fits the "large language model" description — but people usually reserve the *casual* term "LLM" for large **generative** decoder-style models (GPT, Claude, Llama, etc.) in everyday conversation, even though BERT-style models technically qualify by the literal definition too. This is a soft convention, not a hard rule.
- **Is every Transformer an LLM?** No — Transformers are also used for small-scale tasks (a small classification model, a Vision Transformer for image tasks, a Decision Transformer for RL) that aren't "large language models" at all. Transformer = architecture choice; LLM = a specific large-scale, language-focused *use* of that architecture (or, loosely, any architecture at that scale).
- **Is "Claude" (this assistant) the same relationship as GPT/ChatGPT?** Yes, the same pattern applies generally across the industry: Anthropic trains a large Transformer-based language model, and what you interact with is a fine-tuned, product-packaged version of it — same conceptual layering as GPT → ChatGPT, just a different company/model family.

7. Interview Q&A (Core Only)

Q1: Why did Transformers replace LSTM+attention? A: LSTMs process sequentially — can't parallelize, slow to train at scale. Self-attention removes recurrence entirely, so the whole sequence processes in parallel, making training far faster and more scalable.

Q2: Self-attention vs. encoder-decoder attention — what's the difference? A: Self-attention relates words *within the same sequence* (e.g., "it" ↔ "animal"). Encoder-decoder attention relates decoder state to encoder states *across two different sequences*. Same scoring mechanism, different pairing.

Q3: Encoder-only vs decoder-only vs encoder-decoder — when to use each? A: Encoder-only (BERT) → understanding/classification. Decoder-only (GPT) → text generation. Encoder-decoder (T5) → sequence-to-sequence tasks like translation.

Q4: Is LLM an architecture? A: No — it describes scale/purpose, not design. Almost all current LLMs happen to be Transformers, but the terms answer different questions.

Q5: GPT vs. ChatGPT? A: GPT = the underlying decoder-only model family. ChatGPT = a GPT model further fine-tuned (RLHF, instruction tuning) and packaged as a chat product — an application layer on top, not a separate architecture.

Q6: Why is self-attention's quadratic cost a real problem? A: Cost grows roughly with the square of sequence length (every token compares to every other token), so very long documents get expensive fast — a major active research area (efficient/long-context attention variants).

write blog on this below pics timeline.

The Timeline

28 January 2024 00:55



Day : Self-Attention — From Static Embeddings to Task-Specific Contextual Meaning

Continuation of your Transformer notes

1. The Starting Problem: Turning Words Into Numbers

Before any neural network can process text, words must become numbers (**vectorization**). The field went through several stages:

- **One-Hot Encoding (OHE):** each word = a huge sparse vector with a single 1. No notion of meaning — "cat" and "dog" are just as "different" as "cat" and "car."
- **Bag of Words (BoW) / TF-IDF:** represent text by word frequency/importance. Still no real semantic understanding — just counting.
- **Word Embeddings (Word2Vec, GloVe):** each word becomes a **dense, low-dimensional vector** that actually captures *meaning* — semantically similar words end up close together in vector space (e.g., "king" and "queen" are near each other; "king – man + woman $\approx$ queen").

How does an embedding "learn" meaning just from being passed through a neural network? The trick isn't magic — it comes from the **training objective**. Models like Word2Vec are trained on a simple task: *predict a word from its surrounding context* (or vice versa) across millions of sentences. Words that tend to **appear in similar contexts** ("king ruled the ...", "queen ruled the ...") get pushed toward similar vectors, because the network is forced to represent them similarly to succeed at that prediction task. This is called the **distributional hypothesis**: *"a word is characterized by the company it keeps."* The network never explicitly gets told what a word means — it *infers* similarity purely from co-occurrence patterns across huge amounts of data.

Pretrained vs. training your own embeddings — what do people actually do?

- **Pretrained embeddings** (Word2Vec, GloVe, or embeddings from a pretrained Transformer): trained on massive general text already. Best when you have **limited** task-specific data — you piggyback on general language knowledge already learned.
- **Training your own embeddings from scratch:** best when you have a **large, domain-specific** dataset (e.g., medical or legal text) where general-purpose embeddings might not capture domain-specific word usage well.
- **In practice, most modern systems don't even use standalone Word2Vec/GloVe anymore** — they use embeddings learned *inside* a Transformer (BERT, GPT, etc.), trained end-to-end as part of the whole model, which brings us to the real point of this topic.

2. The Real Problem: Static Embeddings Have a Fixed, "Average" Meaning

Word2Vec/GloVe-style embeddings are **static** — one word always maps to the exact **same** vector, no matter what sentence it appears in. That vector ends up being a kind of *blended average* of every meaning/usage the word had across the huge training corpus.

Why is this a problem? Words are often ambiguous — their *actual* meaning depends entirely on context.

Example:

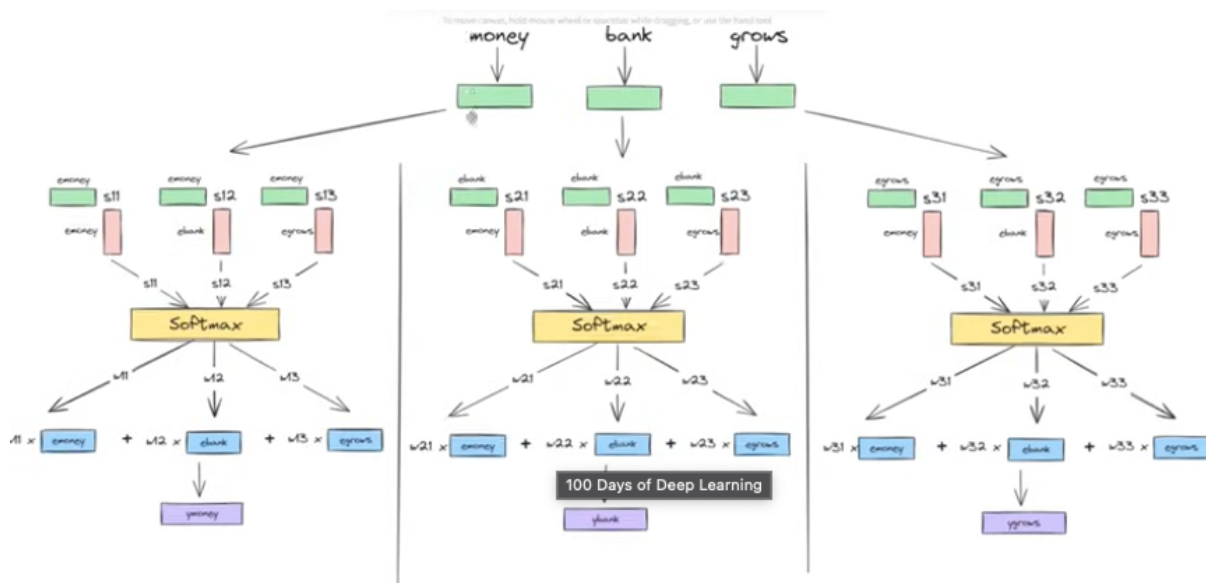
- "I deposited money in the **bank**." → *bank* = financial institution
- "We sat by the river **bank**." → *bank* = edge of a river

A static embedding gives the word "bank" the **same** vector in both sentences — some kind of statistical average of "financial institution" and "riverbank" meanings, which is accurate for *neither* specific usage. Ideally, we want a **contextual embedding** — a representation of "bank" that *changes* depending on the words around it, correctly leaning toward "financial" when near "money," and toward "geography" when near "river."

Self-attention exists specifically to fix this.

3. Self-Attention (Basic Version) — Turning Static Into Contextual

Core idea: self-attention takes a sentence's static embeddings and, for each word, produces a **new, contextual embedding** built as a **weighted blend of all the words' static embeddings in that sentence** — where the weights reflect how relevant each other word is to this one.



Step-by-step, using your diagram's example: "money bank grows"

(Think of this as a company/finance sentence — "bank" here should lean toward the financial meaning, and self-attention should reflect that.)

Step 1 — Start with static embeddings. e_money , e_bank , e_grows — one fixed vector per word (from Word2Vec/GloVe-style embeddings).

Step 2 — For each word, compute similarity (dot product) with every word in the sentence (including itself):

For "money": $s11 = e_money \cdot e_money$ $s12 = e_money \cdot e_bank$ $s13 = e_money \cdot e_grows$

For "bank": $s_{21} = e_{\text{bank}} \cdot e_{\text{money}}$ $s_{22} = e_{\text{bank}} \cdot e_{\text{bank}}$ $s_{23} = e_{\text{bank}} \cdot e_{\text{grows}}$
For "grows": $s_{31} = e_{\text{grows}} \cdot e_{\text{money}}$ $s_{32} = e_{\text{grows}} \cdot e_{\text{bank}}$ $s_{33} = e_{\text{grows}} \cdot e_{\text{grows}}$

A dot product between two vectors is a natural **similarity measure** — bigger when vectors point in similar directions.

Step 3 — Softmax each row to turn raw similarity scores into normalized weights (summing to 1):

$[s_{21}, s_{22}, s_{23}] \rightarrow \text{softmax} \rightarrow [w_{21}, w_{22}, w_{23}]$

Step 4 — Weighted sum of the *original static embeddings*, using these weights:

$\text{contextual_bank} = w_{21} \times e_{\text{money}} + w_{22} \times e_{\text{bank}} + w_{23} \times e_{\text{grows}}$

If "bank" and "money" are semantically closer than "bank" and "grows" (which they are, in a financial sense), w_{21} ends up larger, so contextual_bank is pulled toward e_{money} 's "direction" — producing a bank-embedding that now specifically reflects the *financial* sense, purely because of the words around it in *this* sentence.

This exact process (Steps 2–4) is repeated **independently for every word** — this is why your diagram shows **three separate softmax panels**, one per word ("money," "bank," "grows"), each producing its own contextual embedding from a weighted mix of all three static embeddings.

Important properties of this basic version:

- **Fully parallel** — every word's contextual embedding can be computed independently and simultaneously (no waiting on any previous word, unlike an LSTM).
- **No learnable parameters** — it's pure math (dot products + softmax + weighted sum) on the *existing* static embeddings. Nothing here is "trained."

4. The Next Problem: General Contextual Embeddings Aren't Task-Specific

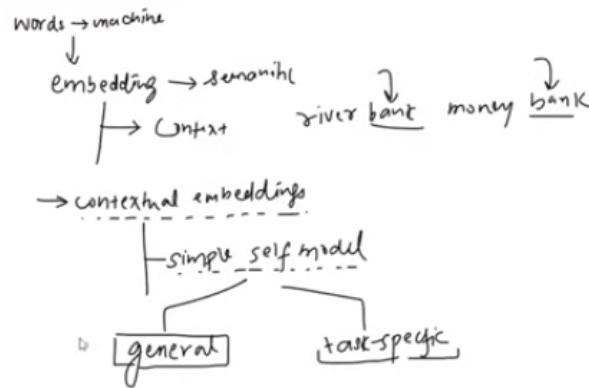
The version above gives you *some* notion of context — but it's completely **generic**, because there's nothing in the process that's aware of *what task* you're actually trying to solve. The same contextual embedding gets used whether you're doing translation, sentiment analysis, or anything else.

Example — why this matters: The English idiom "**piece of cake**" means "an easy task." A general contextual embedding might just represent "cake" in the context of "piece of," still leaning toward the literal food meaning — because nothing in the process was ever taught "this phrase is idiomatic and should be treated specially for translation." If you're building a machine translation model into Urdu, a *literal* translation would produce something meaning "a piece of cake" (the dessert), completely losing the idiomatic "easy" meaning — a mistranslation. What's "needed" from the embedding is different depending on the task: translation needs idiom-awareness, sentiment analysis might need emotional-tone-awareness, and so on. **A one-size-fits-all contextual embedding can't specialize for this** — we need the model to be able to *learn*, from labeled task data, how words should be related to each other specifically for that task.

The fix: introduce **learnable weights and biases** into the self-attention process itself, so the model can adjust *how* it blends words together based on feedback from real task data (via backpropagation) — instead of using a fixed, untrainable formula.

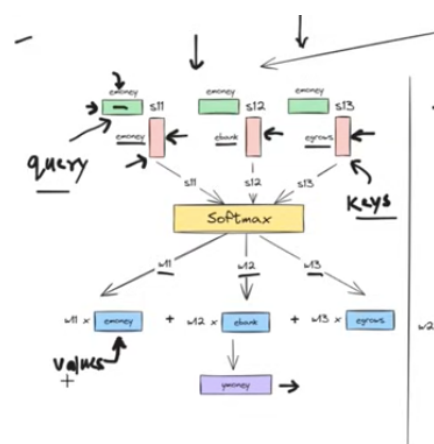
Progress

06 February 2024 00:42



In general self-attention we did only three steps, dot product of matrices (to show the similarities of one word with others) then applying SoftMax and again dot product, no weight introduction in this whole process

So now we will introduce weight in task specific contextual embedding



5. Query, Key, Value — Making Self-Attention Learnable

To make self-attention trainable, we stop using the raw static embedding directly for everything, and instead derive **three separate, specialized vectors** from each word's embedding:

- **Query (Q):** "What am I looking for?" — represents the current word's request: "which other words are relevant to me?"
- **Key (K):** "What do I have to offer, for others to check against?" — represents each word in a way that's specifically built to be *compared against* queries.

- **Value (V):** "*Here's my actual content/information,*" — represents each word in a way that's specifically built to be *combined together* once relevance has been decided.

A small but important correction to keep straight: Value is **not** the *result* of the weighted sum — it's one of the three **input** roles, just like Query and Key. The actual output (the new contextual embedding) is obtained by taking a weighted sum **of the Value vectors** (using attention weights computed from Query and Key) — the result of that weighted sum is the **contextual embedding/output**, not "the Value" itself.

Why not just reuse the same embedding vector for all three roles?

Analogy — a library search: Imagine searching a library.

- Your **search terms** are the Query — what you're looking for.
- Each book's **index tags/catalog entry** is the Key — what gets compared against your search terms to judge relevance.
- The **actual book content** is the Value — what you walk away with, once a good match is found.

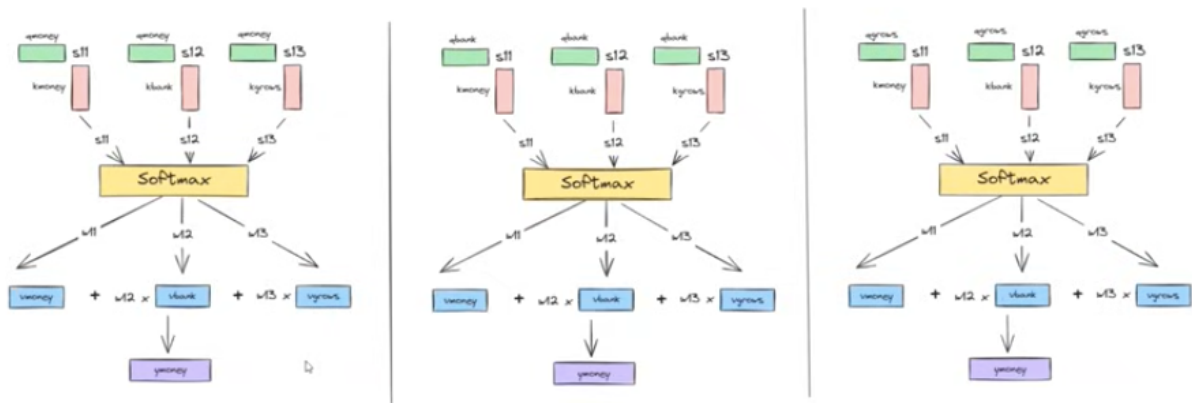
You *could*, in principle, use the same representation for your search terms, the catalog tags, and the book contents — but each of these jobs benefits from being represented *differently*, optimized for its own purpose. A good "search term" representation and a good "actual content to retrieve" representation don't need to look the same. **Separating Q, K, V lets the network learn a different, specialized transformation for each role**, rather than forcing one vector to awkwardly serve three different purposes at once — this is a classic **separation of concerns** design choice, and it gives the model much more flexibility to learn better relationships.

How Q, K, V vectors are actually built

Through a **linear transformation** — multiplying the word's embedding vector by a **learned weight matrix**:

$$\begin{aligned} Q_i &= e_i \cdot W_Q \\ K_i &= e_i \cdot W_K \\ V_i &= e_i \cdot W_V \end{aligned}$$

W_Q, W_K, W_V are **trainable matrices**, shared across all words in the sentence, and they get updated via backpropagation based on the task's loss — this is exactly what makes the whole process *learnable* and *task-specific* now, unlike the basic (parameter-free) version in Section 3.



6. Full Worked Example — Scaled Dot-Product Self-Attention

Using **"money bank grows"** again, now with learnable Q/K/V:

Step 1 — Get static embeddings for each word: e_{money} , e_{bank} , e_{grows} .

Step 2 — Project each into Query, Key, and Value vectors using the *same* three learned weight matrices for every word:

$$\begin{aligned} Q_{\text{money}} &= e_{\text{money}} \cdot W_Q & K_{\text{money}} &= e_{\text{money}} \cdot W_K & V_{\text{money}} &= e_{\text{money}} \cdot W_V \\ Q_{\text{bank}} &= e_{\text{bank}} \cdot W_Q & K_{\text{bank}} &= e_{\text{bank}} \cdot W_K & V_{\text{bank}} &= e_{\text{bank}} \cdot W_V \\ Q_{\text{grows}} &= e_{\text{grows}} \cdot W_Q & K_{\text{grows}} &= e_{\text{grows}} \cdot W_K & V_{\text{grows}} &= e_{\text{grows}} \cdot W_V \end{aligned}$$

Step 3 — Compute raw alignment/attention scores for each word's Query against every word's Key (dot product):

$$\begin{aligned} \text{score}(\text{bank}, \text{money}) &= Q_{\text{bank}} \cdot K_{\text{money}} \\ \text{score}(\text{bank}, \text{bank}) &= Q_{\text{bank}} \cdot K_{\text{bank}} \\ \text{score}(\text{bank}, \text{grows}) &= Q_{\text{bank}} \cdot K_{\text{grows}} \end{aligned}$$

(and the same for "money" and "grows" as the query word, each against all three keys)

Step 4 — Scale the scores by dividing by $\sqrt{d_k}$ (d_k = dimension of the Key vectors) — explained fully in Section 7.

Step 5 — Softmax each row of scaled scores → normalized attention weights (α) summing to 1.

Step 6 — Weighted sum of Value vectors (not the original embeddings this time!):

$$\text{contextual_bank} = \alpha_1 \times V_{\text{money}} + \alpha_2 \times V_{\text{bank}} + \alpha_3 \times V_{\text{grows}}$$

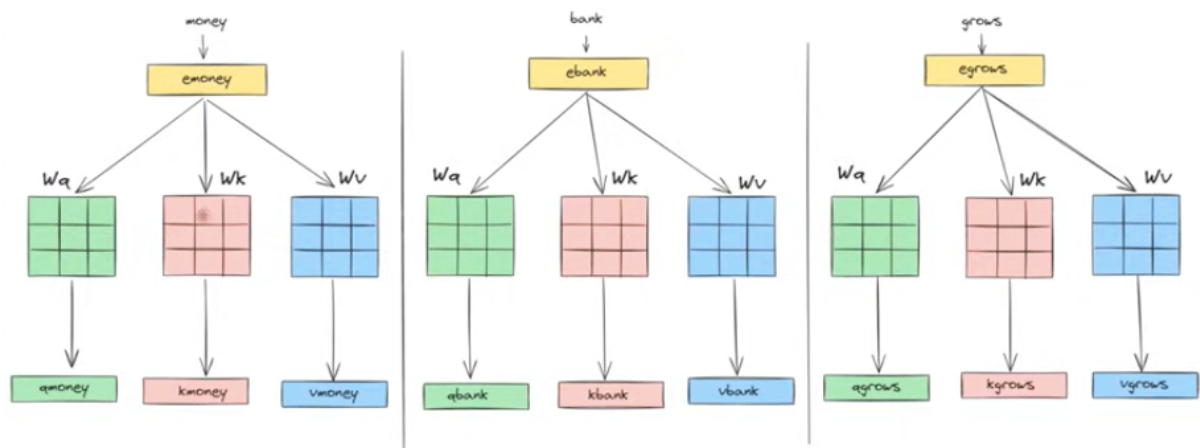
In matrix form (how it's actually implemented), for the whole sentence at once:

$$\begin{aligned} Q &= X \cdot W_Q & (X &= \text{matrix of all word embeddings stacked as rows}) \\ K &= X \cdot W_K \\ V &= X \cdot W_V \end{aligned}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

This single formula computes **every word's contextual embedding simultaneously** — one matrix multiplication handles the whole sentence at once, which is *why* this is fully parallelizable on a GPU.

Because w_Q, w_K, w_V are learned from real labeled task data, the model gradually learns *for that specific task* which relationships between words actually matter — fixing the "piece of cake" problem from Section 4, since the weights can be trained to recognize idiom-like patterns, sentiment-carrying words, or whatever actually matters for the downstream job.



7. Why We Scale: $\sqrt{d_k}$

Short intuition: as the dimensionality of Q and K vectors (d_k) grows, dot products between them tend to grow larger in magnitude too (you're summing more terms). Very large values fed into softmax push it toward extreme, near-one/near-zero outputs — the softmax becomes **saturated**, and its gradient becomes **very close to zero** in those regions. Near-zero gradients mean the model barely learns anything from backpropagation at that point — training becomes slow or unstable.

The fix: divide every score by $\sqrt{d_k}$ before applying softmax. This keeps the scale of the dot products roughly constant regardless of how large d_k is, keeping softmax's output (and its gradients) in a healthy, well-behaved range — hence "**scaled** dot-product attention."

8. Why Is It Called "Self"-Attention?

Because the **Query, Key, and Value all come from the exact same sequence** — the model is computing how each word in a sentence relates to *other words in that same sentence*. This is called **intra-sequence attention** ("intra" = within).

Compare this to **encoder-decoder attention** (Bahdanau/Luong, from your earlier notes) — that was **inter-sequence attention**: the Query came from the *decoder* (one sequence), while the Key/Value came from the *encoder* (a *different* sequence). Self-attention drops that split entirely — everything happens **within one single sequence**, relating its own words to each other, which is exactly why "self" is the right word for it.

9. How Is Self-Attention Different From Bahdanau/Luong Attention?

They share the *same underlying mathematical skeleton*: compute an alignment score $\rightarrow$ softmax $\rightarrow$ weighted sum. But they differ in three important ways:

	Bahdanau / Luong Attention	Self-Attention
Sequences involved	Two different sequences (decoder $\leftrightarrow$ encoder)	One single sequence, related to itself
What's compared	Decoder hidden state vs. encoder hidden states	Every word's Query vs. every word's Key, <i>within the same sentence</i>
Vectors used	Raw RNN/LSTM hidden states directly	Specially learned Query, Key, Value projections of embeddings
Needs recurrence?	Yes — built on top of an LSTM/RNN backbone	No — works directly on embeddings, no recurrence at all
Parallelizable?	No — decoder states are computed sequentially	Yes — all words processed simultaneously

In short: self-attention is a **generalization and formalization** of the same "attention" idea — Luong's dot-product scoring is essentially the direct ancestor of self-attention's $Q \cdot K$ scoring — but self-attention (a) applies it *within* one sequence instead of between two, (b) introduces the explicit Query/Key/Value separation for more learnable flexibility, and (c) removes any dependency on a recurrent backbone, making it fully parallel. This last point is precisely what let Transformers replace LSTMs entirely.

10. Explaining the Diagrams, Step by Step

Image 1 & Image 3 (three-panel "money / bank / grows" diagrams): These show the **basic, parameter-free self-attention process** from Section 3 — one full pass per word:

- Each panel corresponds to computing the contextual embedding for **one specific word** (left panel = "money," middle = "bank," right = "grows").
- At the top: the static embeddings of all three words are shown feeding in.
- The colored boxes (green/pink) below represent each word's embedding being compared against every other word's embedding — producing raw similarity scores (s_{11} , s_{12} , s_{13} , etc., shown as small labeled boxes).
- These scores flow into the yellow **Softmax** box — converting raw scores into normalized attention weights.
- Below softmax: the weighted combination step — each attention weight multiplies its corresponding word's static embedding, and these are summed together (the $w \times \text{money} + w \times \text{bank} + w \times \text{grows}$ row) to produce the final purple box: the **contextual embedding** for that panel's word (e.g., the final purple box in the "bank" panel is the new, context-aware "bank" vector).
- All three panels run **independently and in parallel** — this is exactly Section 3's process, done visually, once per word.

Image 2 (Query/Key/Value diagram): This shows the **upgraded, learnable version** from Sections 5–6:

- The green and pink boxes at the top represent the word embeddings being linearly transformed into **Query** and **Key** vectors (via W_Q and W_K).
- These feed into the **Softmax** block — this is the $QK^T / \sqrt{d_k}$ scoring + normalization step from Section 6.
- The labels "**Query**" and "**Keys**" on either side mark exactly which transformed vectors are playing which role in this comparison.
- Below, the weighted combination ($w \times \text{money} + w \times \text{bank} + w \times \text{grows}$, similar to before) produces the **Value**-based output — except now these weights came from *learned* Query/Key projections rather than raw static embeddings directly, and the vectors being blended are the **Value** vectors, producing a task-adapted contextual embedding (the purple output box).

11. Worth Knowing (Beyond What Was in Your Content)

A few things that naturally come next in this topic, useful to know even briefly:

- **Multi-head attention:** in practice, Transformers don't run self-attention just once — they run it **multiple times in parallel** ("heads"), each with its own separate W_Q , W_K , W_V matrices, letting different heads specialize in capturing different types of relationships (e.g., one head might focus on grammatical structure, another on long-range topical relevance). The outputs of all heads are concatenated and combined at the end.
- **Positional information is missing by default:** self-attention treats a sentence as a *set* of words being compared to each other — it has no inherent sense of *word order* on its own (swap the word order and the math above still "works" the same way). This is why Transformers add **positional encodings** to the embeddings before self-attention, injecting order information back in.

12. Core Summary (Read This Alone and You'll Understand the Whole Topic)

- **Words must first be converted into numbers (embeddings)** so a neural network can process them. Traditional embedding methods like **Word2Vec** and **GloVe** learn word meanings from co-occurrence patterns, but they assign **one fixed vector to every word**, regardless of the sentence it appears in. This creates problems for ambiguous words such as "**bank**" (river bank vs. financial bank), because the embedding cannot change based on context.
- **What Self-Attention's job:** Self-attention solves this limitation by transforming **static embeddings into dynamic, contextual embeddings**. Instead of treating each word independently, every word looks at **all the other words in the sentence** to determine which ones are most relevant to understanding its meaning. It computes these relevance scores using **dot-product similarity**, converts them into probabilities with **softmax**, and then combines information from the entire sentence using those weights. As a result, the embedding for a word like "**bank**" changes depending on whether nearby words are *money*, *loan*, and *ATM*, or *river*, *water*, and *shore*. This basic form of self-attention has **no learnable parameters** and processes **all words in parallel**, making it much faster than sequential models such as RNNs.

- **Limitation:** Although self-attention produces contextual embeddings, this basic version is still **generic**. It has no learnable parameters to adapt its behaviour for a particular task, so it cannot automatically learn specialised patterns such as treating "**piece of cake**" as an idiom during translation or focusing on task-specific information. Or "the man saw the astronomer with a telescope".it figure out only single perspective from sentence or paragraph etc, self attention fails where there is ambiguity.it just form a table by calculating the similarity of one word with another so it cant capture hidden meaning or multiple perspective sometimes. This is why Transformers introduce **learnable Query (Q), Key (K), and Value (V) matrices**, allowing the attention mechanism to learn what information is important for different tasks.
- **Fix — Query, Key, Value:** instead of using raw embeddings directly, we derive three specialized vectors per each word via **learned linear transformations** ($Q = e \cdot W_Q, K = e \cdot W_K, V = e \cdot W_V$). Query = "what am I looking for," Key = "what do I offer to be matched against," Value = "the actual content to blend together." Separating these three roles (rather than reusing one vector for everything) gives the model much more flexibility, the same way a library separates search terms, catalog tags, and book contents.
- **Scaled Dot-Product Attention formula:** $\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$ — compute similarity scores between queries and keys, scale by $\sqrt{d_k}$ to avoid unstable/vanishing gradients in softmax, normalize with softmax, then use those weights to blend the Value vectors into the final contextual, **task-adapted** embedding.
- **"Self"** because Query, Key, and Value all come from the *same* sequence(same sentence) (intra-sequence attention) — unlike Bahdanau/Luong attention, which relates *two different* sequences (decoder ↔ encoder) and depends on a recurrent (LSTM) backbone. Self-attention keeps the same alignment-score → softmax → weighted-sum lineage but removes recurrence entirely and adds learnable Q/K/V projections, making it fully parallel and the foundation of the entire Transformer architecture.

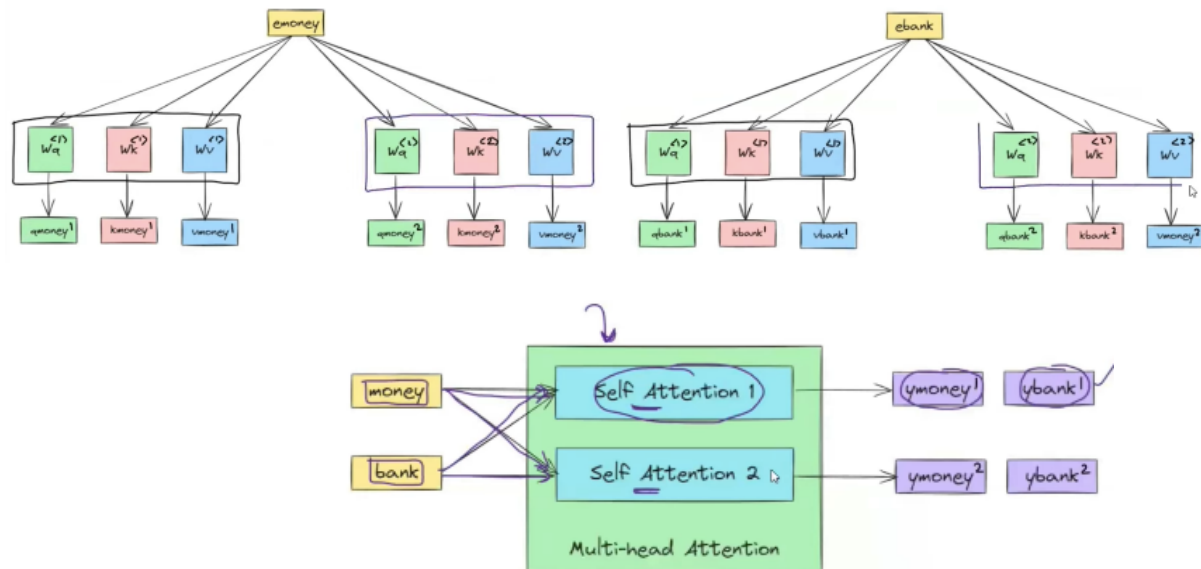
Day : Multi head attention

Multi head attention is also done through the self-attention

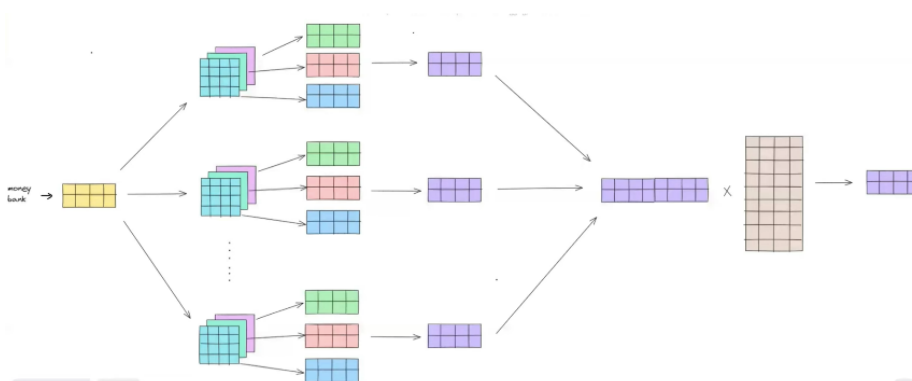
Limitation: Although self-attention produces contextual embeddings, this basic version is still **generic**. It has no learnable parameters to adapt its behaviour for a particular task, so it cannot automatically learn specialised patterns such as treating "**piece of cake**" as an idiom during translation or focusing on task-specific information. Or "the man saw the astronomer with a telescope".it figure out only single perspective from sentence or paragraph etc, self attention fails where there is ambiguity.it just form a table by calculating the similarity of one word with another so it can't capture hidden meaning or multiple perspective sometimes

Multi head attention: if we have sentence that can have two meaning we can't be able to figure out it multiple meaning with just one self-attention so we had to use many self-attention that is why it is called multi head attention.

If we using single self attention we using one set of w_q , w_k and w_v if it became two then we use two set of these parameter in last two set of contextual embedding. Everything will get double what will get double? and how whole process will be then?



Instead of one set of matrices we use multiple set of matrixes and thus we get multiple contextual embedding, in last we concatenate the two contextual representations to draw balance and find the final representation.



Day : Positional encoding:

Self-attention made contextual embedding from the static embedding and can everything in parallel. But we can't capture the order of the sentence or paragraph with the self-attention.

Solution is positional encoding.

Bounded continuous and relative position (periodic)

We have to calculate positional encoding for each word, it will be vector (set of numbers)

The dimensionality of any word positional encoding vector would be equal to the dimension of its static embedding

We add embedding vector and positional and then send it to the self-attention, we don't concatenate it so that we don't increase the dimension and parameter

Formula:

Do one word or sentence positional encoding formula explaining each step



Day : Layer normalization:

We used layered normalization here

Why normalization in deep learning?

- Improved training stability to avoid vanishing or exploding gd
- Faster convergence
- Mitigating internal covariate shift
- Give regularization effect

We keep our data in some range. New transformed data has mean of zero and variance of one.

We can apply normalization on the input data and on hidden layer activations.

Normalizing:

Batch normalization:

Layer normalization

Why we use layer in transformer instead of batch one , and how their working happen and differ from each other. How is it applied on data



Day : Transformer Encoder Architecture:

Transformer have multiple no of encoder and decoder

Each encoder consists of feed forward NN and self-attention.

Input before getting into encoder goes through three steps

Tokenizer : senetence is divided into token

Embedding: each word is transformed into number embedding , is it static or contextual embedding at this point?

Positional encoding: to keep information of order we do positional encoding which generate each vector for each word and then this vector got added to the embedding vector

Encoder block:

In encoder these vector are given to multi head attention to apply self attention so that we can get contextual embedding that can distinguish word and give it separate embedding for each context river bank , money bank

Add and norm block : we have two set of vector one our original input and second that got as output of multi head attention. We add these two set of vector and then we apply layer normalization on it and get set of vector that are normalized , we do normalized to stabilize the training process by bringing them in some small range.

Feed forward block: we pass these set of vectors into the feed forward , we feed these set of vectors as a one matrix to the neural network, why the purpose of PASSING it to FNN, BECAUSE OF Relu we introduce non linearity , we again got matrix in output which is just set of vector

We again add the previous output of self attention and and these output set vector of the FNN

Parameter(weight and baieses) of each encoder will be different from other encoder block. They are same in architecture but these parameter will be different from each other

Why use residual connection in it? No one knows exact reason but maybe for two reason like stable training (resnet) or maybe to correct multi head attention if it fails

Why use a FFNN? With help of fnn we add non linearity as this whole process is linear in nature, so to catch nonlinear complexities we use it

Why use 6 encoder blocks? Language are complex so to understand it we had to use multiple encoder to understand it fully



Day : Masked Self Attention

The transformer decoder is autoregressive at inference time and non-autoregressive at training time.

This difference in behavior is because of masked self-attention.

Autoregressive model: came from time series, they are class of model that generate data point in a sequence by conditioning each new point on the previously generated points.

Autoregressive example: predicting some stock price on any particular day it depends, what the stock price was on the previous day, new data points depend on the previous one

There are two phases of a Transformer's life:

1. **Training**
 - The model learns from millions or billions of examples.
 - It updates its weights using backpropagation.
2. **Inference**
 - The model has already learned.
 - We only use it to make predictions.
 - **No weights are updated.**

Example: Suppose you ask ChatGPT:

"The capital of France is"

At inference time, GPT's job is to predict the next word.

It predicts `Paris`

No learning happens. It is simply using what it already learned during training.

During inference

Initially only

I

exists.

Predict

love

Now

I love

exists.

Predict deep

Now

I love deep exists.

Predict learning

This requires **multiple forward passes**, one for each generated token.

Inference time **is the phase where a trained Transformer is used to generate predictions without updating its weights. In a decoder-only Transformer like GPT, inference is autoregressive: the model predicts one token at a time, appends that token to the input, and repeats the process until the response is complete. This sequential generation is why inference is inherently slower than training, even though each forward pass is highly parallel internally.**

In these notes I didn't understand what masked self attention is , make me understand that in a better way, how it work , what its purpose , why we need it

Difference between masked multihead attention and multi head self attention?



Day : Cross attention:

It is a mechanism used in transformer architecture, particularly in task involving sequence to sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.

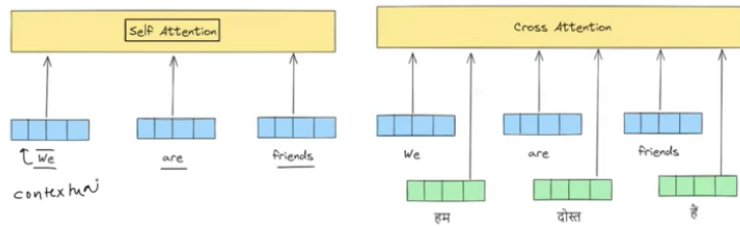
Make me understand this definition with some example ?

Same sequence me similarity find karta ha -> self attention

Two alag sequence me similarity/relationship find krna -> cross attention

Self attention vs cross attention

The input: we are sending single sequence embedding in self attention while in cross it need both sequences in input



The processing difference:

The output difference: we are doing calculation for output sequence in terms of input sequence

Cross attention vs attention badduanua/lyong:

We there used to find context vectors

Use cases : image captioning, text to image generation, text to speech



Day : Decoder architecture in training process:

Right Shifting We add token in the start of input

Second step is tokenization in which we will get multiple token

Third we do embeddings in which we got set of vectors of same dimensions

To keep track of token order we do positional embedding in which we generate vector for each token and then we position encoding vector with the embedding vecor

Do rest of step do each of step what we do with those set of vectors next



Day : Transformer inference:

Encoder behavior is same both in training and inference. While decoder in training time act as non-autoregressive and inference decoder act as autoregressive

At the end of encoder step, we get vector for each corresponding token

Inside the Transformer: Multi-Head Attention → Full Encoder-Decoder

Continuation of your Self-Attention notes

1. Multi-Head Attention

The limitation it fixes

Basic self-attention builds one similarity "table" per sentence — for every word, it computes how similar it is to every other word, using **one single set** of w_Q , w_K , w_V . But some sentences are genuinely **ambiguous** and support more than one valid interpretation *at the same time*.

Classic example: *"The man saw the astronomer with the telescope."*

- Interpretation A: the *man* is holding the telescope, and used it to see the astronomer.
- Interpretation B: the *astronomer* is the one holding the telescope.

A single self-attention pass produces **one** similarity table — it can lean toward one interpretation, but it can't hold *both* relational patterns simultaneously. That's the core limitation: **one head = one perspective**, and language is often genuinely multi-perspective.

The fix: run several self-attentions in parallel

Instead of one set of w_Q , w_K , w_V , we use **multiple independent sets** — each set is called a **head**. Each head has its *own* learned w_Q , w_K , w_V matrices, so each head can specialize in picking up a *different kind* of relationship (one head might lean toward "who's doing the action," another toward "which object is being modified," etc. — the model figures out these specializations on its own through training, they're not manually assigned).

What exactly doubles (or multiplies)? If you go from 1 head to 2 heads:

- You now have **2 separate sets** of w_Q , w_K , w_V (previously 1 set).
- Each head independently runs the full scaled dot-product attention process ($Q \cdot K^T / \sqrt{d_k} \rightarrow \text{softmax} \rightarrow \text{weighted sum of } V$) $\rightarrow$ so you get **2 separate contextual embedding outputs** per word instead of 1.
- **Final step — concatenate:** the 2 (or more) heads' output vectors for each word are **concatenated** together into one longer vector, then usually passed through one more learned linear layer (w_O) to blend the combined information back down to the model's standard working dimension.

In practice (as in the original Transformer), if the model's embedding dimension is d_{model} and you use h heads, each individual head is usually sized d_{model} / h , so that after concatenating all h heads back together, you land back at exactly d_{model} — no dimension explosion, just the *same total space* divided up and processed from h different "angles" in parallel.

Analogy: imagine handing the same ambiguous sentence to several different critics — one focused purely on grammar, one on tone, one on literal meaning. Each gives their own independent judgment (their own contextual embedding). At the end, you don't just pick one critic's opinion — you **combine all of their perspectives** into one final, richer verdict. That combining step is the concatenation + final linear layer.

💡 **Interview angle:** "*Why not just use one very large single-head attention instead of multiple smaller heads?*" — A single head is mathematically forced to compress all relational information into one similarity pattern per word. Multiple heads let the model represent **several independent types of relationships simultaneously**, which a single (even larger) head cannot do — it's not about capacity alone, it's about the model being able to hold multiple *distinct* relational "views" at once.

2. Positional Encoding

The problem

Self-attention compares every word to every other word using dot products — this process is **order-agnostic**: shuffle the words in a sentence and the *math* would treat it identically (attention has no built-in notion of "this word came before that one"). But word order obviously changes meaning ("dog bites man" vs. "man bites dog"). We need to inject order information back in.

The fix

For every word position, compute a **positional encoding vector** — a set of numbers that uniquely represents *where* that word sits in the sentence — and **add** it (element-wise) to that word's embedding vector before it enters self-attention.

- The positional vector's dimensionality **matches** the embedding's dimensionality exactly (so they can be added together).

- We **add**, not concatenate — this keeps the vector size unchanged (no extra dimensions, no extra parameters needed downstream).

Why the encoding needs to be bounded, continuous, and periodic

- **Bounded:** values shouldn't grow unboundedly large for long sentences (that would destabilize training) — so a function like sine/cosine, which stays within $[-1, 1]$ no matter how large the position number gets, is ideal.
- **Continuous:** neighboring positions should have smoothly related encodings, not jump around erratically.
- **Periodic (relative position):** we don't just want the model to know "this is word #5" in isolation — we want it to be able to infer **relative** distances too ("this word is 3 positions after that one"). Sine/cosine waves at different frequencies have a useful mathematical property: a shifted sine/cosine wave can be written as a **linear combination** of the original sine/cosine (via standard trigonometric addition formulas) — meaning the model can learn to compute *relative* offsets between positions through simple linear operations on these encodings, rather than needing to memorize every absolute position independently.

The formula, explained piece by piece

```
PE(pos, 2i)    = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )
```

- **pos** — the word's position in the sentence (0, 1, 2, 3, ...).
- **i** — index of the dimension pair within the embedding vector (each pair of dimensions gets one sine and one cosine).
- **d_model** — the total embedding dimension.
- $10000^{(2i/d_model)}$ — this term controls the **frequency/wavelength** of the wave for that particular dimension. Early dimensions (small *i*) oscillate very fast (high frequency, short wavelength) — good for capturing fine-grained, short-range position differences. Later dimensions (large *i*) oscillate very slowly (low frequency, long wavelength) — good for capturing coarse, long-range position differences. Combining many different frequencies across all the dimensions gives every position a **unique overall signature/pattern**.

Tiny worked example (using $d_model = 4$, just to see the mechanics):

For **position 0** ($pos = 0$): every $\sin(0/\dots) = 0$ and every $\cos(0/\dots) = 1$, so $PE(0) = [0, 1, 0, 1]$.

For **position 1** ($pos = 1$, with $i = 0$ and $i = 1$):

```
PE(1, 0) = sin(1 / 10000^(0/4)) = sin(1)           ≈ 0.841
PE(1, 1) = cos(1 / 10000^(0/4)) = cos(1)           ≈ 0.540
PE(1, 2) = sin(1 / 10000^(2/4)) = sin(1/100)       ≈ 0.010
PE(1, 3) = cos(1 / 10000^(2/4)) = cos(1/100)       ≈ 0.99995
```

So $PE(1) \approx [0.841, 0.540, 0.010, 0.99995]$ — a completely different pattern than $PE(0)$, and every future position gets its own unique combination too, generated by a fixed formula (no training required for this part — it's computed directly, not learned).

This vector then gets **added** directly to that word's static embedding:

```
final_input_vector = word_embedding + positional_encoding
```

This combined vector is what actually enters the self-attention layers.

3. Layer Normalization

Why normalize at all, in deep learning generally

- **Training stability** — keeps values in a reasonable, consistent range, avoiding vanishing or exploding gradients as signals pass through many layers.
- **Faster convergence** — well-scaled inputs help gradient-based optimization converge more quickly.
- **Mitigates internal covariate shift** — without normalization, the distribution of activations at each layer keeps shifting as earlier layers' weights update during training, forcing later layers to constantly "re-adapt" to a moving target; normalization keeps that distribution more stable.
- **Mild regularization effect** — normalization can add a small amount of noise/constraint that slightly discourages overfitting.

Mechanically: subtract the mean and divide by the standard deviation, so the resulting values have **mean 0 and variance 1** — then usually apply a small learned scale (γ) and shift (β) so the network can still adjust the normalized range if that's better for learning.

Batch Normalization vs. Layer Normalization

	Batch Normalization	Layer Normalization
Normalizes across	All examples in the same batch , per feature	All features within a single example (e.g., one token's vector)
Depends on batch size?	Yes — needs a reasonably sized, stable batch	No — works even with a batch size of 1
Handles variable-length sequences well?	Poorly — padding and varying lengths make batch statistics unstable/misleading	Well — each token is normalized independently of any other token or example
Behavior differs at inference?	Yes — needs tracked running averages from training	No — computed identically at both training and inference

Why Transformers use Layer Norm, not Batch Norm: text sequences have **variable length**, and are often padded to align batch shapes — batch statistics computed across a batch of padded, variable-length sequences become unreliable and inconsistent. Layer normalization sidesteps this entirely by normalizing **within each individual token's own**

vector, completely independent of batch size, sequence length, or other examples in the batch — a much more natural fit for sequential, variable-length data like text.

Analogy:

- **Batch Norm** = comparing everyone's height in the room to each other, to figure out a "normal" range — you need the whole room (batch) present to do this.
- **Layer Norm** = normalizing one person's own internal body measurements relative to *themselves* — no need for anyone else to be present at all.

4. Transformer Encoder Architecture

Before the encoder: three preprocessing steps

1. **Tokenizer** — splits the sentence into tokens (words or sub-words).
2. **Embedding** — each token becomes a numeric vector. At *this* stage, the embedding is **static** — it's a plain lookup, with no context factored in yet. It only *becomes* contextual after passing through the self-attention layers inside the encoder block.
3. **Positional Encoding** — a positional vector is generated per token and **added** to its embedding, preserving order information (Section 2).

Inside one encoder block

```
Input vectors
|
Multi-Head Self-Attention    -> produces contextual embeddings
|
Add & Norm                  (residual connection + layer normalization)
|
Feed Forward Network (FFN)  -> adds non-linearity
|
Add & Norm
|
Output vectors -> passed to the next encoder block
```

Multi-Head Self-Attention: exactly as covered in Section 1 — produces a contextual embedding for every token, capable of distinguishing meanings based on surrounding context (e.g., correctly separating "river bank" from "money bank").

Add & Norm: take the block's *original input* vectors and **add** them to the *output* of the sublayer just computed (multi-head attention, or later the FFN) — this is a **residual/skip connection**. Then apply **layer normalization** to the summed result, keeping values in a stable, consistent range before passing to the next stage.

Feed Forward Network (FFN): the set of contextual vectors (treated as one matrix, one row per token) is passed through a small feedforward network — typically two linear layers with a **ReLU** activation in between. **Why this step exists:** the attention mechanism itself is fundamentally a set of **linear operations** (dot products, weighted sums) — without ReLU's non-linearity, stacking more attention layers alone couldn't model complex, non-linear

relationships in language. The FFN is what actually introduces that non-linear expressive power. It's applied **independently to each token's vector** (same small network, reused per token), and outputs a same-shaped set of vectors.

Then **another Add & Norm**: the FFN's output is added back to *its own* input (the output of the first Add & Norm block), then normalized again.

Stacking encoder blocks

The original Transformer stacks **6 identical encoder blocks**. "Identical" means same *architecture* — but **each block has its own separately learned parameters** (different w_Q , w_K , w_V , different FFN weights). **Why stack several**: language is complex and hierarchical — like the "deep LSTM" reasoning from earlier notes, stacking more encoder blocks lets the model build progressively richer, more abstract representations layer by layer, rather than trying to capture everything in one pass.

Why residual connections specifically? Honestly, there isn't one single universally agreed-upon reason — but two commonly cited justifications:

1. **Training stability at depth** (the same reasoning behind ResNet in computer vision) — residual connections give gradients a more direct path backward through the network during backpropagation, helping avoid vanishing gradients in deep stacks.
2. **A safety net if a sublayer underperforms** — if multi-head attention on a particular token doesn't produce anything especially useful for that position, the residual connection ensures the original input information isn't lost entirely — the network can effectively "skip past" a sublayer that isn't helping.

💡 **Interview angle**: be ready to explain **why FFN is necessary despite attention already being powerful** — the clean answer is: attention = linear weighted combination; FFN (with ReLU) = non-linearity, which is what actually gives the network the capacity to model complex functions, not just weighted averages.

5. Masked Self-Attention

Why it's needed — the core idea

The decoder's job is to **generate output one token at a time**, and at inference time, when generating token 5, it genuinely does not have access to tokens 6, 7, 8... (they don't exist yet). This is the **autoregressive** constraint: *each new token can only depend on the tokens that came before it, never on tokens that come after*.

Autoregressive models, defined generally: a class of models that generate each new data point *conditioned on previously generated points* — the same idea used in time series forecasting (e.g., predicting tomorrow's stock price based on previous days' prices; each new value depends on the ones before it, never on future ones).

The training vs. inference mismatch this creates

- **At inference:** generation really is one token at a time — predict a word, append it, feed the whole sequence back in, predict the next word, and so on. This is **sequential and genuinely autoregressive**.
- **At training:** for efficiency, we don't want to loop token-by-token (that would be painfully slow to train on millions of examples) — instead, we feed the **entire target sequence at once**, in parallel, and let the model make predictions for *every* position simultaneously in a single forward pass.

But here's the danger: if the whole target sequence is visible at once, the model could "cheat" — when predicting word 3, it could just peek at word 3 directly (since it's sitting right there in the input) instead of actually learning to predict it from words 1–2. **Masked self-attention exists to prevent exactly this.**

How masking actually works

Inside the decoder's self-attention step, after computing the raw attention scores ($Q \cdot K^T$), a **mask** is applied *before* the softmax step: for every position, the scores corresponding to **future** positions are set to **negative infinity** (or a very large negative number). After softmax, $\exp(-\infty) \approx 0$ — meaning the attention weight assigned to any future position becomes exactly **zero**. Each position can therefore only "attend to" itself and everything **before** it — never anything after.

Analogy: it's like taking an exam where you're only allowed to look at what you've already written on your own answer sheet — never allowed to peek ahead at questions or answers further down the page.

Net effect: even though training processes the whole sequence in one parallel pass (efficient), the masking forces the *computation* at each position to behave **as if** it only had access to previous tokens — faithfully simulating the autoregressive constraint during training, without needing a slow token-by-token loop. This is exactly what your notes meant by "*training is non-autoregressive [in execution/parallelism], inference is autoregressive [in actual token-by-token generation]*" — masking is the mechanism that makes the parallel training version behave consistently with the truly sequential inference version.

Masked Multi-Head (Self-)Attention vs. regular Multi-Head Self-Attention

They use the **exact same underlying mechanism** (multiple heads, each with their own w_Q , w_K , w_V , scaled dot-product attention) — the *only* difference is the **masking step** inserted before softmax:

- **Regular multi-head self-attention** (used in the encoder, and in decoder cross-attention): every position can attend to **every** other position — full visibility, no restrictions.
 - **Masked multi-head self-attention** (used in the decoder's *first* sublayer, over its own previously generated tokens): each position can only attend to **itself and earlier positions** — future positions are blocked out via the mask.
-

6. Cross-Attention

What it means, with an example

"It allows a model to focus on different parts of an input sequence when generating an output sequence."

Example — translation: Suppose the decoder is generating the French translation of "The cat sat on the mat," and it's currently generating the French word for "cat." Cross-attention lets the decoder, at this exact generation step, **look back at the encoder's representation of the entire input sentence** and figure out that the input word "cat" is the most relevant one right now — pulling in that specific information to help generate the correct output word. At the next step (generating the word for "sat"), cross-attention recomputes and shifts its focus toward the input word "sat" instead. This is precisely the same underlying goal as Bahdanau/Luong attention from your earlier notes — it's just now implemented with the Query/Key/Value mechanism.

Self-attention vs. cross-attention

	Self-Attention	Cross-Attention
Intuition	Finds similarity within the same sequence	Finds relationship between two different sequences
Input	One single sequence's embeddings	Two sequences: one for Query, a different one for Key & Value
Query comes from	Same sequence	The decoder (target/output sequence)
Key & Value come from	Same sequence	The encoder's final output (source/input sequence)
Output	Contextual embedding of a sequence, in terms of itself	Contextual embedding of the <i>output</i> sequence, expressed in terms of the <i>input</i> sequence

Cross-attention vs. Bahdanau/Luong attention

Both are solving the exact same underlying problem — letting the decoder dynamically find relevant context vectors from the encoder at each generation step. Cross-attention is essentially the **Transformer-native version of the same idea**: instead of a small feedforward alignment network (Bahdanau) or a raw dot product between recurrent hidden states (Luong), cross-attention uses the same learned Query/Key/Value projection mechanism as self-attention, computed via $\text{softmax}(QK^T/\sqrt{d_k}) \cdot V$ — with Query built from decoder representations and Key/Value built from encoder representations — and, like self-attention, this is computed for **all decoder positions in parallel** (during training) rather than needing a sequential loop.

Use cases beyond text translation

Cross-attention generalizes naturally to **any task combining two different sequences/modalities**:

- **Image captioning** — the text-generating decoder attends to encoded regions/ patches of an image (Key/Value from the image encoder).
 - **Text-to-image generation** — the image-generating side attends to the text prompt's embeddings to decide what to draw where.
 - **Text-to-speech** — the audio-generating decoder attends to the input text's encoded representation to align speech generation with the correct words/timing.
-

7. Decoder Architecture — Training Process, Full Walkthrough

Step 1 — Right-shifting: prepend a `<start>` token to the beginning of the target sequence, shifting everything one position to the right. This ensures that at every position, the decoder's *input* is the *previous* correct token — setting up the standard teacher-forcing arrangement, where the model learns to predict "current token" from "everything before it."

Step 2 — Tokenization: split the (now right-shifted) target sentence into tokens.

Step 3 — Embedding: convert each token into a numeric vector (again, static at this point — no context factored in yet).

Step 4 — Positional Encoding: generate a positional vector per token position and **add** it to the token's embedding — preserving order information, exactly as in the encoder.

Step 5 — Masked Multi-Head Self-Attention: the decoder's own (shifted) target sequence attends to *itself*, but with masking applied (Section 5) so each position can only see previous positions, never future ones — producing contextual embeddings of the target sequence that respect the autoregressive constraint.

Step 6 — Add & Norm: residual connection (add the block's input to this sublayer's output) + layer normalization, same purpose as in the encoder.

Step 7 — Cross-Attention: now the decoder's vectors (Query) attend to the **encoder's final output vectors** (Key & Value) — this is where the decoder actually pulls in relevant information from the *source* sentence to inform what it should generate (Section 6).

Step 8 — Add & Norm: again, residual connection + layer normalization applied to the cross-attention sublayer's output.

Step 9 — Feed Forward Network: same purpose as in the encoder — adds non-linearity, applied independently per-token.

Step 10 — Add & Norm: final residual + normalization for this decoder block.

(Repeat): this entire decoder block (Steps 5–10) is stacked **N times** (6, in the original paper) — each block with its own separately learned parameters, just like the encoder stack.

Final step — Linear + Softmax: the final decoder block's output vectors are passed through one more linear layer that projects them to the size of the **vocabulary**, followed by **softmax**, producing a probability distribution over all possible next tokens — **for every position in the sequence simultaneously**, since training processes the whole (masked) sequence in one parallel pass. The loss (categorical cross-entropy, same as in the pre-Transformer Seq2Seq notes) is then computed by comparing these predicted distributions against the actual next tokens at every position.

8. Transformer Inference

Encoder — identical behavior in training and inference: the encoder always processes the **entire input sequence at once**, since the input is fully known upfront in both phases — there's never anything to "hide" on the encoder side, so no masking is needed there, and the encoder is *never* autoregressive, regardless of phase. At the end of the encoder stack, you get one contextual output vector **per input token** — and critically, these encoder outputs are computed **once** and then reused at every single decoder generation step that follows.

Decoder — different behavior in training vs. inference:

- **Training:** processes the whole (right-shifted, masked) target sequence in one parallel forward pass — computationally "non-autoregressive" in *execution*, even though masking enforces the *logical* autoregressive constraint (Section 5).
- **Inference:** truly sequential/autoregressive — there is no full target sequence available yet, since we're generating it. The process looks like this:
 1. Start with just the <start> token as decoder input.
 2. Run a full forward pass through the decoder (using the encoder's already-computed outputs for cross-attention) → predict the first output token.
 3. **Append** that predicted token to the decoder's input sequence.
 4. Run another full forward pass over this now-longer sequence → predict the next token.
 5. Repeat, appending and re-running, until an <end> token is generated or a maximum length is reached.

Why inference is slower than training, despite internal parallelism: each individual forward pass through the decoder is itself highly parallel (all current positions computed together, same as training) — but generating a full response requires **one entire forward pass per new token**, run sequentially, one after another. Training processes millions of *complete* examples with one parallel pass each; inference has to loop, forward-passing repeatedly, once per generated token, for every single response — that repeated, sequential looping is the actual source of inference's extra latency, not any lack of parallelism within a single pass.

Quick Summary — Everything in This Note, Compressed

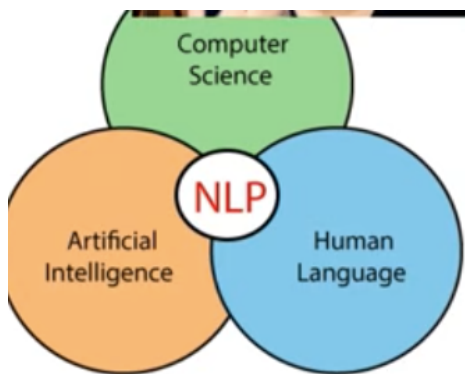
- **Multi-Head Attention:** one self-attention head only captures one relational "view" of a sentence, failing on genuinely ambiguous sentences. Multiple heads, each with their own W_Q , W_K , W_V , run in parallel to capture multiple relationships simultaneously; their outputs are concatenated and linearly combined into one final representation.
- **Positional Encoding:** self-attention has no built-in sense of word order, since it's fundamentally a set-based comparison. A sine/cosine-based positional vector (bounded, continuous, periodic — enabling relative-position reasoning) is generated per position and **added** to each token's embedding before attention.
- **Layer Normalization:** stabilizes training by normalizing each token's own feature vector to mean 0 / variance 1, independent of batch size — unlike Batch Norm, which normalizes across a batch and struggles with variable-length, padded sequences. This makes Layer Norm the natural fit for Transformers.
- **Encoder block:** Multi-Head Self-Attention → Add & Norm → Feed Forward Network (adds non-linearity via ReLU) → Add & Norm, stacked N times (6 in the original paper), each block with independently learned parameters, to build increasingly rich, hierarchical language understanding.
- **Masked Self-Attention:** blocks each decoder position from attending to future positions (via setting future attention scores to $-\infty$ before softmax), enforcing the autoregressive constraint even while training processes the full sequence in one parallel pass — resolving the "training is parallel, inference is sequential" distinction.
- **Cross-Attention:** the Transformer-native version of Bahdanau/Luong attention — decoder Query attends to encoder Key/Value, letting the decoder dynamically pull relevant information from the *input* sequence while generating each *output* token; used far beyond translation (image captioning, text-to-image, text-to-speech).
- **Decoder training pipeline:** right-shift target → tokenize → embed → add positional encoding → masked self-attention → add & norm → cross-attention (with encoder output) → add & norm → feed forward → add & norm → (repeat N times) → linear + softmax over the vocabulary, for every position at once.
- **Inference:** encoder behaves identically to training (always full, parallel, no masking needed). Decoder becomes genuinely sequential — one new token generated per full forward pass, appended, and fed back in repeatedly — which is why inference is slower than training despite each individual pass being internally parallel.

DL ENDED HERE, MAKE DL ONLY UPTO THIS POINT

=====

Natural Language Processing (NLP)

1. Introduction to NLP



A natural language is language that has evolved naturally in humans through use and repetition without conscious planning or premeditation

Need for NLP

Real world application:

- Contextual advertisements like targeted ads
- Email clients spam filtering smart reply
- Social media – removing adult media, opinion mining on twitter etc
- Search engine like today google summarize what we ask, we don't go inside web
- Chatbots

Common nlp tasks

- Text/documents classification
- Sentiment analysis
- Information retrieval,
- Part of speech tagging, we assign part of speech to each word
- Language detection and machine translation
- Conversational agents
- Knowledge graph and qa systems
- Text summarization
- Topic modelling use ml model LDA
- Text generation in keyboards
- Spell checking and grammar correction
- Text parsing
- Speech to text

Approaches to NLP

1. Heuristic methods: jugar lagana , rule based approaches , regular expression regex(They allow you to search, parse, modify, and validate text patterns efficiently), wordnet(lexical dictionary that give us relationship of word with another, running and shoes), open mind common sense (store all fact of common sense), these are quick approaches and mostly error free as human design it
2. Machine learning models: rules are made by models looking at data and finding patterns and relationship in it , naïve based, logistic regreesion , svm , lda , hidden Markov models
3. Deep learning model: big advantage is retain sequential information, feature gerneation is done automatically by dl models , architechtures used are RNNS(good for time series sequential data, it cannot retain conext in long sentences or inputs), LSTM(retain context, memory better than the rnn), GRU(used mostly for text generation), Transformers (boosted everything , models like BERT are transformer based,), Autoencoders

Challenges in NLP:

- **Ambiguity:** one sentence can have many meanings (saw a boy with glasses)
- Contextual words (I ran into store because we ran out of milk)
- Colloquialism and slang (piece of cake mean easy we know it meaning, machine don't)
- Synonyms:
- Irony sarcasm and tonal diff (that just what I needed today in two scenarios)
- Spelling errors
- Creativity
- Diversity (so many langauges understanding them grammers etc is hard)

2. NLP PIPELINE END TO END

This pipeline is not universal

Deep learning piepline are slightly diffferent

Pipeline is non linear

STEPS:

- **Data acquisition**
- **Text preparation**
 1. Text cleanup
 2. Basic preprocessing like tokenization, stop words ,
 3. Advanced preprocessing pos tagging , chunking , core resolution
- **Feature engineering**

1. **Bag of words**
2. **Tf-idf**
3. **Word to vec**
- **Modelling**
 1. **Model building**
 2. **Evaluation**
- **Deployment**
 1. **Deployment**
 2. **Monitoring**
 3. **Model update**

DATA acquisition

- Data available
 1. If less data then we do data augmentation like bi gram , back translate ,add noise , symo
 2. Database
 3. Csv file, tables
- Other sources:
 1. Api
 2. Web scraping
 3. Pdf
 4. Images
 5. Audio
- If data not available:
 1. Surveys , forms , questionare

Text preparation

1. **Basic cleaning:** removing html tag etc from scraped data, we use regular expression here, removing emojis by encoding into utf-8 , spell checking,
2. **Basic text preprocessing**
 - Basic: sentence or character tokenization
 - Optional: stemming (bringing word to it root form), stop word removal (it, is, the ,and), lemmatization, lower case or upper case, , removing digits or punctuation, language detection for language translation tasks
3. **Advanced preprocessing**
 - **Pos tagging** (assigning parts of speech for chatbots, no need to remove stop word for this step)
 - **Parsing** (understating sentence structure)
 - **Coreference resolution**
4. **Feature engineering:** converting text into numbers
 - **Bag of words**
 - **Tf idf**
 - **Ohe**
 - **Word2vec**
 - Which technique to follow depend on the problem
 - In dl feature engineering is done automatically

- While in ml we do it by ourselves manually using domain knowledge,

5. Modelling:

- **Heuristic approach:**
- **ML algorithm:**
- **Deep learning:** need a lot of data to train, transfer learning (BERT) model trained on lot of data already we use that and apply it on our problem
- **Cloud APIs**
 - Which technique to use depend on data size and nature of problem

Evaluation

Intrinsic evaluation: accuracy confusion matrix

Extrinsic evaluation: after deployment evaluating it in business setting

6. Deployments

- Deploy:
- Monitoring: checking time to time how ml model working
- Update: product changes or not , with other data etc, many possibilities

Perplexity tell us our nlp model confusion at some point

3.NLP Text Preprocessing

Lowercasing:

Change all data to the lowercase

Remove html tags : we had to remove the html tags through the regular expression

Regex101.com is a tool to check and test different patterns

Pattern is '<.*?>'

Import re

```
def removeTag(text):
    pattern=re.compile('<.*?>')
    Return pattern.sub('r',text)
```

Remove URLs: remove through regular expression

Remove punctuation:

import string,time

exclude= string.punctuation it will give us all punctuation

now replace those punctuation with ''

```
def remove(text)
    return text.translate(str.maketrans("", "",exclude))
```


chat word treatment: like lmoa, lol, wth, etc

REMOVING short words: like i, am ,

Spell correction: libraries like textblob etc do this

Removing stop words: stop words are, the, my is nltk libraries

Handling emojis: either remove it or replace it

Removing emojis

```
import re
def remove_emoji(text):
    emoji_pattern = re.compile("[
        u"\U0001F600-\U0001F64F" # emoticons
        u"\U0001F300-\U0001F5FF" # symbols & pictographs
        u"\U0001F680-\U0001F6FF" # transport & map symbols
        u"\U0001F1E0-\U0001F1FF" # flags (iOS)
        u"\U00002702-\U000027B0"
        u"\U000024C2-\U0001F251"
        "]+", flags=re.UNICODE)
    return emoji_pattern.sub(r'', text)
```

replacing emoji with their meaning

```
import emoji
print(emoji.demojize('Python is 🔥'))
```

TOKENIZATION: breaking words into characters and sentences into words. why we do tokenization

We can do splitting through the split function, splitting through dot function

Splitting through the regular function

Many libraries do this like NLTK, SPACY they are more better that handle most of the cases

SPACY LIBRARY IS THE MOST PREFERRED ONE

Calling `nlp()` once per row is slower.

spaCy provides `nlp.pipe()`, which processes many texts efficiently.

Stemming:

```
playing
played
plays
player
```

Although these words are different, they all come from the root idea "play". A machine learning model treats them as **different words**:

playing ≠ played ≠ plays ≠ play

This increases the vocabulary size.

Stemming reduces words to a common root (called a **stem**).used mostly in information retrieval system would use NLTK library. **Stemming simply cuts off the end of a word using rules.**

Porter Stemmer: (English) It removes common suffixes like: Ing, ed, ly, es, s

playing

play

Playing -> play

study → studi : It isn't a real word. That's okay. Porter only cares about reducing similar words to the same stem.

2. Lancaster Stemmer

Think of Lancaster as **Porter's aggressive cousin**.It removes much more of the word.

Example

Original	Lancaster
maximum	maxim
happiness	happy
running	run
organization	org
general	gen

Sometimes it removes **too much**.

Example : connection becomes connect

which is okay, but general becomes gen which loses information.

3. Snowball Stemmer (Porter2)

Snowball is basically an improved Porter. People sometimes call it **Porter2**.

Examples

Original Snowball

Original Snowball

studies studi
playing play
happiness happi
running run

Compared to Porter:

- Better linguistic rules
- Supports many languages
- Usually produces slightly better stems

Most NLP practitioners prefer Snowball over Porter.

4. Regexp Stemmer

This one is completely different. There is **no built-in linguistic intelligence**. You write your own rule.

Example: `RegexpStemmer("ing$")` . Means Remove **"ing"** only if it appears at the end.

Examples

Word	Output
playing	play
swimming	swimm
singer	singer

Notice `singer` doesn't change because it doesn't end in "ing".

Comparison

Let's use one sentence.

The players were playing happily in the playground.

Porter: `the player were play happili in the playground`

Lancaster: `the play wer play happy in the playground`

Notice `were` became `wer` Much more aggressive.

Snowball: the player were play happili in the playground

Similar to Porter but uses improved rules.

Regexp (remove ing, ed, s)

the player were play happily in the playground

Only the endings you specify are removed.

Lemmatization:

Lemmatization converts a word to its actual dictionary (base) form, called a lemma.

Example:

Original Lemma

playing play

played play

every output is a **real English word**.

lemmatization and stemming both work the same but stemming sometime give word which is not even English words and lemmatization is slow

if we don't have to show output to user we will use stemming as it fast

while if we to give show some output to user then we will use lemmatization

in lemmatization the root word is called lemma.

Wordnet is a lexical dictionary that has all English word relation with each other in it , we have to search in this dictionary while looking for lemma

While doing lematimmzation we have to provide where we want to do lemmatization in which part of speech

Difference

Stemming	Lemmatization
Cuts words using rules	Uses vocabulary + grammar
Faster	Slower

Stemming	Lemmatization
Can produce non-words	Produces real words
Less accurate	More accurate

Example

Sentence:

The children were running better than the adults.

Stemming

the children were run better than the adult
(or sometimes adult → adult, children stays unchanged depending on the stemmer)

Lemmatization

the child be run good than the adult

Notice: children → child, were → be, better → good

A stemmer usually **cannot** do this.

Why use Stemming if Lemmatization is better?

Because stemming is:  Much faster  Simpler  Uses less computation

For older ML models like: Naive Bayes, Logistic Regression, SVM stemming often performs almost as well as lemmatization.

Do we use both together?

No. Normally you choose **one**.

Typical pipeline:

```

Lowercase
↓
Remove punctuation
↓
Remove stopwords
↓
Tokenize
↓
Stemming OR Lemmatization ← Choose one
↓
TF-IDF
↓
Model

```

You don't do:

```
Stem
↓
Lemmatize
```

or

```
Lemmatize
↓
Stem
```

because one transformation is enough, and applying both can distort the text.

2. NLP Text Preprocessing

Why preprocessing matters: Raw text is messy — inconsistent casing, HTML tags, URLs, punctuation, slang, typos, emojis. Before any text representation technique (BoW, TF-IDF, Word2Vec) can work well, we clean the text so the model focuses on *meaningful* words instead of noise.

1. Lowercasing

Convert all text to lowercase so "Pizza" and "pizza" aren't treated as two different words.

```
text = text.lower()
```

Without this, the vocabulary would double-count the same word in different cases — wasteful and hurts pattern matching.

2. Remove HTML Tags

Text scraped from websites/reviews often contains leftover HTML like `<br>`, `<p>`. These carry no meaning and must be stripped using **regex**.

```
import re
def removeTag(text):
    pattern = re.compile('<.*?>')
    return pattern.sub('', text)
```

`<.*?>` = match anything between `<` and `>`, non-greedy (`?` stops it from matching too much at once). Tool: **regex101.com** — great for testing regex patterns interactively before using them in code.

3. Remove URLs

Same idea — URLs (`http://...`) carry no semantic value for most NLP tasks and are removed via regex.

4. Remove Punctuation

Punctuation (`!`, `,`, `.`) usually adds no useful signal for classic text representation techniques.

```
import string
exclude = string.punctuation    # gives all punctuation characters

def remove_punc(text):
    return text.translate(str.maketrans('', '', exclude))
```

5. Chat Word Treatment

Convert internet slang/abbreviations into their full form so the model understands them: `lol` → laughing out loud, `lmao` → laughing my ass off, `wth` → what the hell. Usually done via a lookup dictionary mapping slang → full phrase.

6. Removing Short Words

Very short words (`i`, `am`, `a`) often carry little standalone meaning and are sometimes filtered out to reduce noise (similar reasoning to removing stopwords).

7. Spell Correction

Fixes typos (`"pizzza"` → `"pizza"`) so misspelled variants aren't treated as unique, unrelated words. Commonly done using the **TextBlob** library.

```
from textblob import TextBlob
corrected = TextBlob("pizzza is amazingg").correct()
```

8. Removing Stopwords

Stopwords = extremely common words (`the`, `is`, `my`, `a`, `and`) that appear everywhere and carry little distinctive meaning. Removing them reduces vocabulary size and lets the model focus on meaningful/content words. Commonly handled via **NLTK's** built-in stopwords list.

Analogy: Stopwords are like "um," "uh," "so" in spoken conversation — grammatically necessary for humans, but if you're trying to figure out *what a conversation is about*, you can safely ignore them.

9. Handling Emojis

Two approaches: **remove** emojis entirely, or **replace them with their textual meaning** (preserves sentiment/information).

Removing emojis (regex on Unicode emoji ranges):

```
import re
def remove_emoji(text):
    emoji_pattern = re.compile("[
        u"\U0001F600-\U0001F64F" # emoticons
        u"\U0001F300-\U0001F5FF" # symbols & pictographs
        u"\U0001F680-\U0001F6FF" # transport & map symbols
        u"\U0001F1E0-\U0001F1FF" # flags
        u"\U00002702-\U000027B0"
        u"\U000024C2-\U0001F251"
    ]+", flags=re.UNICODE)
    return emoji_pattern.sub(r'', text)
```

Replacing emoji with meaning (better — preserves sentiment info):

```
import emoji
print(emoji.demojize('Python is 🔥'))
# Output: "Python is :fire:"
```

Replacing is usually better than removing when sentiment matters (e.g., sentiment analysis) — 🔥 carries real emotional signal that shouldn't just be thrown away.

10. Tokenization

What it is: Breaking text into smaller units — either **sentences** → **words**, or **words** → **characters** — because models need to process language piece by piece, not as one giant blob of text.

Why we do it: Every text representation technique (BoW, TF-IDF, Word2Vec) operates on individual *tokens* (words), not whole sentences. Tokenization is the mandatory first step that produces these units.

Ways to do it:

- `split()` function — naive, splits on whitespace, breaks on edge cases (e.g., "Mr. Smith" incorrectly splits at the period).
- Splitting via regex — more control, still limited.
- **NLTK / spaCy** — proper NLP libraries that handle punctuation, abbreviations, contractions, and edge cases correctly.

spaCy is the most preferred library — faster, more robust, production-grade.

Efficiency tip: Calling `nlp()` once per row (one document at a time) is slow. spaCy provides `nlp.pipe()`, which processes many texts in a batched, optimized way — much faster for large datasets.


```
import spacy
nlp = spacy.load("en_core_web_sm")

# Slow way
docs = [nlp(text) for text in texts]

# Fast way
docs = list(nlp.pipe(texts))
```

11. Stemming

The problem it solves: Words like `playing`, `played`, `plays`, `player` all share the same root idea — "play." But to a machine, these are **completely different strings/words**, unnecessarily inflating vocabulary size.

What stemming does: Chops off word endings using fixed rules to reduce a word to its **stem** (root form) — no grammar or dictionary knowledge involved, just pattern-based suffix removal. Commonly done via **NLTK**. Mostly used in **information retrieval systems** (like search engines) where exact grammatical correctness doesn't matter — just matching similar words.

Analogy: Stemming is like a lawnmower — it just chops off the top of anything sticking up past a certain length, without caring whether what's left still "looks nice" or makes grammatical sense.

Types of Stemmers

1. Porter Stemmer (most common, English-only) Removes common suffixes: `ing`, `ed`, `ly`, `es`, `s`.

- `playing` → `play` ✓
- `study` → `studi` — not a real word, but that's fine; Porter only cares that similar words map to the *same* stem, not that the output is valid English.

2. Lancaster Stemmer — "Porter's aggressive cousin." Cuts much more aggressively.

Original	Lancaster
maximum	maxim
happiness	happy
running	run
organization	org
general	gen

Sometimes too aggressive — `general` → `gen` loses meaningful information.

3. Snowball Stemmer (Porter2) — an improved, more refined version of Porter.

Original	Snowball
studies	studi

Original Snowball

playing play
happiness happi

Better linguistic rules, supports multiple languages, generally produces slightly better stems than Porter. **Most practitioners prefer Snowball over Porter.**

4. Regexp Stemmer — no linguistic intelligence at all; you manually define the rule.

```
RegexpStemmer("ing$") # remove "ing" only if it's at the end
```

Word

Output

playing play
swimming swimm
singer singer (unchanged — doesn't end in "ing")

[Side-by-Side Comparison](#)

Sentence: "The players were playing happily in the playground."

Stemmer	Output
Porter	the player were play happili in the playground
Lancaster	the play wer play happy in the playground (<i>very aggressive</i> — "were" → "wer")
Snowball	the player were play happili in the playground (<i>similar to Porter, improved rules</i>)
Regexp (remove ing,ed,s)	the player were play happily in the playground (<i>only specified endings removed</i>)

12. Lemmatization

What it does: Converts a word to its actual **dictionary/base form**, called a **lemma** — using real vocabulary and grammar knowledge, not blind rule-chopping.

Original Lemma

playing play
played play

Unlike stemming, **every lemmatization output is a real English word.**

How it works internally: Lemmatization looks up words in **WordNet** — a lexical dictionary containing relationships between English words (synonyms, base forms, etc.). You must also specify the **part of speech (POS)** (noun, verb, adjective) for accurate lemmatization, since the correct base form depends on grammatical role (e.g., "better" as an adjective → "good", but as a verb might lemmatize differently).

Analogy: Stemming is a lawnmower — fast, rough, chops blindly. Lemmatization is a skilled gardener who actually knows what each plant should look like and prunes it back to its proper natural form — slower, but the result always looks "correct."

Stemming vs Lemmatization

	Stemming	Lemmatization
Method	Rule-based suffix cutting	Vocabulary + grammar-based lookup
Speed	Faster	Slower
Output	Can be non-words	Always real words
Accuracy	Less accurate	More accurate

Example: Sentence: "The children were running better than the adults."

- **Stemming:** the children were run better than the adult (often can't properly handle irregular forms like "children" or "better")
- **Lemmatization:** the child be run good than the adult — notice children → child, were → be, better → good. A stemmer typically cannot make these grammar-aware transformations.

When to Use Which

- **No output shown to the user** (internal processing only, e.g., feeding into TF-IDF for a classifier) → use **stemming** — it's fast, and classic ML models (Naive Bayes, Logistic Regression, SVM) perform nearly as well with stemmed text.
- **Output IS shown to the user** (e.g., chatbot responses, summaries, search snippets) → use **lemmatization** — because it produces grammatically correct, readable real words.

Do We Use Both Together?

No. You pick one, never both — because each is already a *complete* transformation of the word. Applying stemming after lemmatization (or vice versa) can actually distort the text further and provides no added benefit.

Typical Preprocessing Pipeline

```
Lowercase
↓
Remove punctuation
↓
Remove stopwords
↓
Tokenize
↓
Stemming OR Lemmatization ← choose one, not both
↓
TF-IDF (or other vectorization)
↓
Model
```

NLP LECTURE 4 TEXT REPRESENTATION

To convert text into number is complicated while converting images and audio to number is straight forward

Machine learning models **cannot understand text, images, or audio directly**. Everything must eventually become **numbers**.

Images => Each pixel already has a number representing its brightness. The image is **already just a matrix of numbers**.

Audio → Audio is simply a wave. Imagine recording someone saying:

"Hello"

The microphone stores it like: 0.12 0.45 -0.30 0.18 ...

These are the sound amplitudes at different times. Again, **audio is already a sequence of numbers**.

Text → Hard

- Suppose we have I love pizza
- A computer sees 'I' 'love' 'pizza'
- These are **letters**, not numbers.

Why text is difficult

Words have meaning, context, synonyms, different spellings, grammar

For example: car automobile vehicle

All mean almost the same thing.

That's why NLP uses feature extraction

Instead of assigning random numbers, we use techniques like

- One hot encoding
- Bag of Words
- N grams
- CountVectorizer
- TF-IDF

- Word2Vec (**embedding**)
- GloVe
- FastText
- BERT embeddings

These methods convert words into numbers **while trying to preserve their meaning or importance**.

Related Terms that use in these topics:

1. **Corpus:** Corpus is the collection of all documents (all text) in the dataset. can have repeated words
 - If we combine all the reviews:
 - I love pizza
 - Pizza is delicious
 - I love burgers
 - The corpus contains all the words:
 - I, love, pizza, Pizza, is, delicious, I, love, burgers
 - Notice that **repeated words are still part of the corpus**.
2. **Vocabulary:** Vocabulary is the set of all unique words present in the corpus.
 - From the corpus:
 - I, love, pizza, Pizza, is, delicious, I, love, burgers
 - The vocabulary is:
 - I, love, pizza, is, delicious, burgers
3. **Document:** A document is one individual piece of text (one review, one tweet, one email, etc.).
 - Example:
 - Document 1: I love pizza
 - Document 2: Pizza is delicious
 - Document 3: I love burgers
 - Each row is one document.
4. **Word:** A word (or token) is one individual word inside a document.
 - Example:
 - Document:
 - I love pizza
 - Words (tokens):
 - I
 - love
 - pizza

Imagine you have a book.

- 📖 **Corpus** → The entire book (all pages together).
- 📄 **Document** → One page (or one review/tweet).
- 📖 **Word/Token** → One word on the page.
- 📖 **Vocabulary** → The dictionary of all **unique** words used in the book.

One hot encoding:

Each word should be converted into V dimensional vector. V dimensional vector dimension should be equal to the no of words in vocabulary, not used nowadays because of many flaws

Pros : very intuitive easy to implement

cons: sparsity(multiple zeros in each vector), no fixed size of input , out of vocabulary, capture no semantics meaning

Bag of words:

Monday, December 13, 2021 6:22 PM

Text Classification

V=5

	people	watch	campusx	write	comment
D1	1	1	1	0	0
D2	0	1	2	0	0
D3	1	0	0	1	1
D4	0	0	1	1	1

Order of the word doesn't matter, no context of sentence matter, what matter is words occurring and their occurring frequency

In sciket learn we can use the class of CountVectorizer for its implementation

```
cv= CountVectorizer()  
cv=fit_transform(df['text'])
```

solve the problem of out of vocabulary problem, we only show the frequency of word already present in vocabulary

pros : simple and intuitive , solve fixed input size problem , vector closeness capture little bit semantic relationship

cons: sparsity as we move to large datasets

out of vocabulary words are ignored

order doesnt matter here which is important factor in conveying meaning

N -grams:

Our vocabulary was mostly made up of single words so in n-grams we take multiple words combine to capture the semantic meaning

```
cv= CountVectorizer(ngram_range=(2,2))  
cv=fit_tranform(df['text'])
```

ngram_range=(2,2) this parameter change bag of words from uni gram to bi grams

pros: n gram make two vector different from each other like most of their values are different instead of bag of words case where they are mostly similar, we are able to capture semantic meaning

dimensionality increases with increasing the n number of grams
no out of vocab problem solution

TF – IDF: tell us about how much any word is frequent in doc and how much doc is rare
if some word coming many times in one sentence but rarely in corpus or other sentences so this word value will be higher in that document
it assign separate weightage to each word

tf formula = (no of occurrences of term in doc d/ total no of terms in the doc), always between 0 and 1

IDF: this term tells how common or rare any terms in corpus
Formula is $\log(\text{total no of doc in corpus} / \text{no of docs with term t in them})$
If term is frequent in corpus then its idf would be low
If terms is rare then idf value would be higher

Tf*IDF:
We take their multiplication while making vector

Class TfidfVectorizer
And then fit_Transform any desired column

Sparsity , out of vocab , dimensionality , no semantic capturing

NLP Lecture 4: Text Representation

Why Converting Text to Numbers is Hard

Machine learning models only understand numbers — everything (text, images, audio) must eventually become numeric.

- **Images** → Easy. Each pixel is already a number (brightness/intensity value). An image is literally already a matrix of numbers.
- **Audio** → Easy. Audio is a wave — a microphone records amplitude values over time: 0.12, 0.45, -0.30, 0.18, ... Already a sequence of numbers.
- **Text** → Hard. "I love pizza" → computer sees 'I' 'love' 'pizza' — letters/symbols, not numbers, with no obvious numeric meaning.

Why specifically text is hard: Words carry meaning, context, synonyms, spelling variations, and grammar. For example, *car*, *automobile*, *vehicle* all mean nearly the same thing — but as raw text/characters, a computer has no way to know that. Unlike pixels or sound waves, there's no "natural" numeric representation for meaning.

Analogy: An image is like a paint-by-numbers grid — the numbers were already there. Text is like handing someone a poem in a language they've never seen — you have to build an entire translation system from scratch to give the symbols numeric meaning.

This is why **NLP relies on feature extraction techniques** — instead of assigning arbitrary/random numbers to words, we use techniques that try to *preserve meaning or importance* while converting words to numbers:

- One-Hot Encoding
- Bag of Words
- N-grams
- CountVectorizer
- TF-IDF
- Word2Vec (embeddings)
- GloVe
- FastText
- BERT embeddings

Key Terminology (Foundation Before Any Technique)

Term	Meaning	Example
Corpus	The entire collection of all text/documents in the dataset (repeated words allowed)	All reviews combined: I love pizza. Pizza is delicious. I love burgers.
Document	One individual unit of text (one review, tweet, email, sentence, etc.)	Document 1: "I love pizza"
Word / Token	One individual word inside a document	"I", "love", "pizza"
Vocabulary	The set of all unique words across the whole corpus	{I, love, pizza, is, delicious, burgers}

Book Analogy:

- 📖 **Corpus** = the entire book (all pages combined)

-  **Document** = one page (or one review/tweet)
-  **Word/Token** = one word on a page
-  **Vocabulary** = the unique dictionary of all words used across the whole book (no duplicates)

Important distinction: Corpus counts every word occurrence (with repeats). Vocabulary counts each unique word only once. This distinction matters because most text representation techniques build a vector whose length/dimensions = size of the **vocabulary**, not the corpus.

1. One-Hot Encoding

How it works: Every word gets converted into a vector of length V (V = vocabulary size). The vector is all zeros, except a single 1 at the position corresponding to that word.

Example: Vocabulary: [I, love, pizza, is, delicious, burgers] $\rightarrow V = 6$

```
I           → [1, 0, 0, 0, 0, 0]
love        → [0, 1, 0, 0, 0, 0]
pizza       → [0, 0, 1, 0, 0, 0]
delicious   → [0, 0, 0, 0, 1, 0]
```

Pros:

- Very intuitive, extremely easy to implement.

Cons:

- **Sparsity** — vectors are mostly zeros; wasteful in memory and computation.
 - **No fixed input size** — every sentence becomes a different number of vectors (one per word), and sentences have different lengths, causing problems for models expecting fixed-size input.
 - **Out of Vocabulary (OOV)** — a new word not seen during training has no valid representation.
 - **No semantic meaning** — "happy" and "joy" are just as "different" (mathematically) as "happy" and "table" — the vectors carry no notion of similarity. Not used in practice anymore.
-

2. Bag of Words (BoW)

Core idea: Ignore grammar and word order entirely. What matters is **which words appear** and **how often (frequency)** they appear in a document. Each document becomes one vector of length V , where each position holds the count of that vocabulary word in the document.

Example:

- Doc1: "I love pizza"
- Doc2: "I love burgers"

Vocabulary: [I, love, pizza, burgers]

Doc1 → [1, 1, 1, 0]

Doc2 → [1, 1, 0, 1]

Scikit-learn implementation:

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
X = cv.fit_transform(df['text'])
```

Solves the OOV problem partially — new/unseen words are simply ignored (not counted) rather than crashing the model; the vector still stays valid, just doesn't reflect the unseen word.

Pros:

- Simple, intuitive.
- Fixed-size input solved (every document → same-length vector = V, regardless of sentence length).
- Documents with similar word usage end up with vector directions somewhat close together → captures a small amount of semantic similarity.

Cons:

- Still sparse for large vocabularies.
- Out-of-vocabulary words are simply dropped/ignored (information loss).
- **Word order is discarded** — "food was good, not bad" and "food was bad, not good" can produce very similar or identical BoW vectors despite opposite meaning. This loses a critical part of how meaning is conveyed.

3. N-grams

Core idea: Instead of treating single words (unigrams) as vocabulary items, group **N consecutive words together** as one "unit," to partially recover the word-order/context information that BoW loses.

- N=1 → unigram (regular BoW): I, love, pizza
- N=2 → bigram: I love, love pizza
- N=3 → trigram: I love pizza

Scikit-learn implementation:

```
cv = CountVectorizer(ngram_range=(2,2))    # bigrams only
X = cv.fit_transform(df['text'])
```

`ngram_range=(2,2)` shifts CountVectorizer from unigram-based BoW to bigram-based BoW.

Example of why this helps:

- "food was good not bad" → bigrams: [food was, was good, good not, not bad]
- "food was bad not good" → bigrams: [food was, was bad, bad not, not good]

Now the two vectors look **noticeably different** (most bigram values differ), whereas plain BoW (unigram) would have made them look nearly identical. This is the key advantage — N-grams **better capture local context and semantic ordering** than plain BoW.

Pros:

- Makes semantically different sentences produce clearly different vectors (unlike BoW where they often looked similar).
- Captures some word-order/context information BoW misses entirely.

Cons:

- **Dimensionality increases rapidly** as N increases (many more possible word-combinations than single words) → more sparsity, more memory/compute.
- Still **no solution for out-of-vocabulary** n-grams — an unseen combination is simply not represented.

4. TF-IDF (Term Frequency – Inverse Document Frequency)

Core idea: Not all words are equally important. TF-IDF assigns a **higher weight** to a word that appears **frequently in one specific document** but **rarely across the rest of the corpus** — because such a word is likely more distinctive/informative about that document's content. Common words that appear everywhere (like "the," "is") get **low weight**, since they don't help distinguish documents.

Analogy: Imagine reading restaurant reviews. The word "food" appears in almost every review — it doesn't tell you anything special about a particular review. But the word "undercooked" appearing in only one review out of a thousand is a strong, distinctive signal about that specific review. TF-IDF mathematically rewards rare-but-locally-frequent words like "undercooked," and downweights generic words like "food."

Term Frequency (TF)

Measures how frequently a term appears **within one document**.

$$TF(t, d) = \frac{\text{number of times term } t \text{ appears in document } d}{\text{total number of terms in document } d}$$

- Always between 0 and 1.
- Higher TF $\rightarrow$ word is prominent within that specific document.

Inverse Document Frequency (IDF)

Measures how **rare or common** a term is **across the whole corpus**.

$$IDF(t) = \log\left(\frac{\text{total number of documents in corpus}}{\text{number of documents containing term } t}\right)$$

- If a term appears in **almost every document** (common, e.g., "the," "is") $\rightarrow$ IDF is **low** (close to 0).
- If a term appears in **very few documents** (rare, distinctive) $\rightarrow$ IDF is **high**.

TF-IDF Score

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

We compute this score for every word in every document — the final vector for a document holds the TF-IDF score of each vocabulary word.

Worked Example

Corpus (3 documents):

- Doc1: "I love pizza"
- Doc2: "Pizza is delicious"
- Doc3: "I love burgers"

Consider the word "**pizza**":

- Appears in Doc1 and Doc2 $\rightarrow$ in 2 out of 3 documents.
- TF in Doc1 = $1/3$ (appears once, 3 total words)
- $IDF(\text{"pizza"}) = \log(3/2) \approx 0.176 \rightarrow$ moderate, since it's somewhat common
- $TF\text{-}IDF(\text{pizza}, \text{Doc1}) = (1/3) \times 0.176 \approx 0.059$

Consider the word "**delicious**":

- Appears only in Doc2 $\rightarrow$ in 1 out of 3 documents (rarer $\rightarrow$ more distinctive).
- TF in Doc2 = $1/3$
- $IDF(\text{"delicious"}) = \log(3/1) \approx 0.477 \rightarrow$ much higher, since it's rare
- $TF\text{-}IDF(\text{delicious}, \text{Doc2}) = (1/3) \times 0.477 \approx 0.159$

Even though "pizza" and "delicious" both appear once in their respective document, "**delicious**" gets a **higher score** because it's rarer across the corpus — making it more distinctive/informative.

Scikit-learn implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df['text'])
```

Pros:

- Gives meaningful weight to distinctive/informative words instead of treating all words equally (unlike plain BoW).
- Downweights very common, low-information words automatically.

Cons:

- Still **sparse** (most values zero for large vocabularies).
- **Out-of-vocabulary** words still unsupported.
- **Dimensionality** still tied to vocabulary size — grows large with big corpora.
- **No semantic meaning captured** — like BoW/N-grams, TF-IDF has no notion that "happy" and "joy" mean similar things; it only reflects statistical frequency/rarity, not meaning.

What Does It Actually Mean to "Use a Pretrained Embedding Model"?

The Core Idea (Clear It Up First)

When someone trains Word2Vec/GloVe/FastText, what actually happens is:

1. They take a **massive text corpus** (e.g., all of Google News — ~100 billion words, or all of Wikipedia).
2. They run the training algorithm (the CBOW/Skip-gram neural network, or GloVe's matrix factorization) on this huge corpus.
3. After training finishes, they get a **giant lookup table**: every word that appeared in that corpus now has a fixed-length vector (e.g., 300 numbers) assigned to it.
4. This lookup table — literally millions of `word → vector` pairs — is **saved to a file** and released publicly.

So when we say "a pretrained model," we don't mean a live neural network you interact with. We mean a saved file containing a big dictionary of `word: [list of 300 numbers]`.

Analogy: Think of it like a massive, pre-built bilingual dictionary. Someone spent years compiling it (that's the "training"). You don't need to relearn Spanish yourself — you just **open the dictionary and look up the word** whenever you need its translation. Using a pretrained embedding model is exactly this: you look up a word, and it hands you back its vector.

What's Actually Inside the File?

Literally something like this (simplified):

```
king      → [0.23, -0.11, 0.87, ..., 0.04]    (300 numbers)
queen     → [0.21, -0.09, 0.85, ..., 0.02]    (300 numbers)
apple     → [0.55, 0.32, -0.12, ..., 0.71]    (300 numbers)
happy     → [0.44, 0.29, 0.10, ..., -0.18]    (300 numbers)
...
(millions of such rows, one per word in the training vocabulary)
```

That's it. That's the "model." It's essentially a **giant lookup table (dictionary)** mapping words → fixed vectors. Nothing more mysterious than that.

How Do We "Use" It? (Two Very Different Meanings)

Meaning 1: Just look up ready-made vectors (most common usage)

You download this pretrained file, load it into memory, and simply **query it** for whatever words you need — no training happens on your end at all.

```
import gensim.downloader as api

# downloads and loads Google's pretrained Word2Vec (trained on Google News)
model = api.load('word2vec-google-news-300')

# just LOOK UP the vector — no training, instant retrieval
vector = model['king']
print(vector.shape)    # (300,) → 300 numbers representing "king"

# use cosine similarity to find related words
print(model.most_similar('king'))
# [('queen', 0.65), ('monarch', 0.61), ('prince', 0.59), ...]

# famous analogy example: king - man + woman ≈ queen
print(model.most_similar(positive=['king', 'woman'], negative=['man']))
```

This is the most common way people "use" Word2Vec/GloVe — you never train anything yourself. You just borrow the vectors someone else already computed, because they captured rich semantic relationships from a corpus far bigger than what you could ever train on your own.

Practical use case: Say you're building a sentiment classifier and your dataset has the word "fantastic" in the test set, but your own training data never had that word. If you're using pretrained embeddings, "fantastic" already has a meaningful vector (learned from Google News), close to "amazing"/"great" — so your model can still make a sensible decision, even though it never saw that specific word in *your* dataset.

Meaning 2: Use it as the Embedding Layer inside your own neural network

Instead of randomly initializing your embedding layer's weights, you **load the pretrained vectors as the starting weights** — giving your model a huge head start (this is transfer learning, applied to NLP, exactly like using a pretrained CNN's conv base for images).

```
import numpy as np
from tensorflow.keras.layers import Embedding

vocab_size = len(tokenizer.word_index) + 1
embedding_dim = 300

# build an embedding matrix using pretrained Word2Vec vectors
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in tokenizer.word_index.items():
    if word in model:
        embedding_matrix[i] = model[word]    # copy pretrained vector into
our matrix
    # if word not found (OOV) → row stays all zeros

embedding_layer = Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    weights=[embedding_matrix],    # initialize with PRETRAINED vectors
    trainable=False               # freeze it (like feature extraction in
CNNs!)
)
```

This is **exactly the same idea as freezing a CNN's convolutional base** in transfer learning for images:

- Pretrained conv base = generic visual feature extractor → reused for images.
- Pretrained Word2Vec/GloVe = generic semantic feature extractor → reused for text.

You can even set `trainable=True` to let these pretrained vectors adjust slightly during your training — this is the NLP equivalent of **fine-tuning**.

Two Ways to Get These Vectors (Recap From Earlier Notes)

	Pretrained	Self-trained
Who trains it	Someone else (Google, Stanford, Facebook), on a massive general corpus	You, on your own specific dataset
When to use	General-purpose text (news, reviews, social media, everyday English)	Domain-specific text (medical, legal, internal company jargon) where general vectors won't capture specialized meaning
Effort	Just download and load	Requires your own corpus + training time

Applying This to Each Famous Model

Word2Vec (Google)

Download the pretrained `.bin` file (trained on Google News), load with **Gensim**, and query directly — as shown above.

GloVe (Stanford)

Downloaded as plain text files (e.g., `glove.6B.300d.txt`) — each line is word followed by 300 numbers separated by spaces. You parse this file yourself into a Python dictionary.

```
embeddings_index = {}
with open('glove.6B.300d.txt', encoding='utf8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = vector
```

```
print(embeddings_index['pizza'])    # → 300-dim vector
```

FastText (Facebook)

Same lookup idea, but with a bonus: if a word is missing (OOV), FastText can **construct** a vector on the fly using character n-grams, instead of failing.

```
import fasttext.util
fasttext.util.download_model('en', if_exists='ignore')
ft = fasttext.load_model('cc.en.300.bin')

print(ft.get_word_vector('dragonfruit'))    # works even if "dragonfruit"
wasn't in training data!
```

BERT Embeddings (Google)

Key difference — this breaks the "simple lookup table" idea. BERT is not a static dictionary; it's a full neural network you **run text through** every time, because the vector for a word **depends on the surrounding sentence** (contextual).

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

text = "I deposited money in the bank"
inputs = tokenizer(text, return_tensors='pt')
outputs = model(**inputs)    # actual forward pass through the network — not
a lookup!

# vector for "bank" here will differ from "bank" in "river bank"
word_vectors = outputs.last_hidden_state
```

So "using BERT" means **loading the actual trained neural network** (not just a lookup table) and passing your sentence through it every time — the vector is computed fresh based on context, rather than fetched from a fixed dictionary.

ELMo

Same idea as BERT — a trained bidirectional LSTM network you run text through to get contextual vectors, rather than a static lookup table.

Summary — The One Sentence to Remember

- **Word2Vec / GloVe / FastText** = pretrained **lookup tables** (static — one fixed vector per word, computed once, reused forever, just look it up).
- **BERT / ELMo** = pretrained **networks** (dynamic — you feed in a sentence, and it computes a fresh, context-aware vector on each run).

In both cases, "using a pretrained model" means **reusing knowledge someone else already extracted from massive data**, so you don't need your own huge dataset or compute power — exactly the same underlying philosophy as transfer learning in computer vision.

Summary Table

Technique	Order Matters?	Solves OOV?	Captures Semantics?	Main Weakness
One-Hot Encoding	✗	✗	✗	Sparse, no fixed size, no meaning
Bag of Words	✗	Partial (ignores unseen)	Very little	No word order
N-grams	Partial (local order within N)	✗	Little more than BoW	Dimensionality explodes
TF-IDF	✗	✗	✗	Still no true meaning, just weighted frequency

The common thread: None of these frequency-based methods truly understand *meaning* — they're all fundamentally counting/weighting mechanisms. This is exactly the gap that **Word2Vec, GloVe, FastText, and BERT embeddings** (prediction-based / contextual methods, covered in Lecture 5) are designed to solve — by learning dense vectors that actually capture semantic relationships between words.

NLP lecture 5: WORD2VEC

Word Embedding: All text representation techniques are collectively called word embedding,

In word embedding we convert words into vectors.

Types of word embedding:

Frequency based: bag of words , tf - idf , glove (global vectorization) which is based on matrix factorization

Prediction based: word2vec

Word2Vec:

Biggest advantage over previous techniques:

1. Here we can capture semantic meaning for example that happy and joy have similar meaning
2. Here we get low dimension vector so computation would become fast
3. Dense vector -> most of them are non zero so there would not be overfitting

We use deep learning nn to make this word2vec

Two ways

Pre trained model of word2vec.

Self-trained model

Gensim library

We convert word into vector of some n dimension which has number that represent this word

Let say we have words like king queen men women monkey , we will try to capture their semantic meaning , first we will create some features for them like gender, wealth, weight, power , speak and on basis on those features we will assign them some value

Internally word2vec create features on the basis of vocabulary, we use NN to create these features automatically, how do these features are decided by nn, the underlying assumption of word2vec is that two words sharing similar contexts also share a similar meaning and consequently a similar vector representation from the model.

Types of word2vec on basis of architecture:

CBoW (continuous bag of words): we can't directly convert a word into vector , so we solve some dummy fake problem and in that we got our vector as a by product

Skip-gram: given the target words we have to predict the context words , weights would become embeddings

When to use which:

Working with small data : CBow

Large data : skip- gram , give more better data

To improve word2vec embedding

Increase the training data

Increase the dimension of vectors

Increase window size , window size mean how many words to consider on left and right of target words

Give me whole intuition behind it, how these vector number are created and are represented in backend , make me understand more better with analogies along with examples

Famous embedding models that are used mostly ?

NLP Lecture 5: Word2Vec

What is Word Embedding?

Word Embedding is the umbrella term for **any technique that converts text (words) into numerical vectors**, so that machines (which only understand numbers) can process language.

Types of Word Embedding:

1. **Frequency-based** (count how often words appear):
 - Bag of Words (BoW)
 - TF-IDF
 - GloVe (Global Vectors) — based on matrix factorization of word co-occurrence counts
2. **Prediction-based** (train a neural network to predict something, and use the learned weights as vectors):
 - Word2Vec

Why Word2Vec is Better Than Frequency-Based Methods

Old techniques like BoW/TF-IDF just count word occurrences — they have **no understanding of meaning**. "Happy" and "joy" are treated as completely unrelated, just like "happy" and "table" would be.

Word2Vec fixes three major problems:

1. **Captures semantic meaning** — words with similar meaning end up with similar vectors (e.g., "happy" and "joy" will be close together in vector space).
2. **Low dimensional vectors** — instead of a vector as long as your entire vocabulary (which BoW/TF-IDF produce), Word2Vec gives compact vectors (commonly 100–300 dimensions), making computation much faster.
3. **Dense vectors** — most values are non-zero (unlike BoW/TF-IDF which are mostly zeros — "sparse"). Dense vectors generalize better and reduce overfitting, because the model isn't relying on rare, isolated features.

Analogy: BoW/TF-IDF is like describing a person only by which exact words appear on their ID card — very rigid, no notion of similarity. Word2Vec is like describing a person using a rich profile of traits (personality, interests, habits) — so you can tell two people are "similar" even if their ID cards look nothing alike.

The Core Intuition Behind Word2Vec

The underlying assumption (this is the single most important idea):

Words that appear in similar contexts tend to have similar meanings.

This is called the **Distributional Hypothesis** — "a word is known by the company it keeps." If "king" and "queen" often appear surrounded by similar neighboring words (royal, throne, crown, palace), the model learns they must be semantically related — without ever being told what "royal" means.

How are the vector numbers conceptually decided?

Imagine manually describing words like king, queen, man, woman, monkey using human-understandable features:

Word	Gender	Wealth	Power	Speaks
King	+1.0	0.9	0.9	1.0
Queen	-1.0	0.9	0.9	1.0
Man	+1.0	0.3	0.2	1.0
Woman	-1.0	0.3	0.2	1.0
Monkey	0.0	0.0	0.1	0.0

Each word becomes a **point in an n-dimensional space**, defined by these feature values. Words with similar feature values sit close together geometrically.

The catch: we don't manually design these features ourselves — that's impossible to do for every nuance of meaning in language. Instead, **a neural network automatically discovers these features** by solving a prediction task, and the numbers it lands on become each word's vector. We don't know exactly what each dimension "means" (unlike our hand-made gender/wealth example) — the network invents its own abstract features that happen to organize words meaningfully.

How Word2Vec Actually Creates These Vectors (Backend Mechanics)

Word2Vec uses a **shallow neural network** trained on a **fake/dummy task** — the real goal isn't the task itself, but the **byproduct**: the weight matrix learned along the way becomes our word vectors.

Structure of the network:

Input (one-hot word vector) → Hidden Layer (no activation, just weights) → Output Layer (softmax over vocabulary)

- Input layer size = vocabulary size (V) — each word represented as a one-hot vector.
- Hidden layer size = N (the embedding dimension you choose, e.g. 100 or 300).
- The **weight matrix between input and hidden layer** (size $V \times N$) is exactly what becomes the word embeddings — each row of this matrix corresponds to one word's vector.

Once training finishes, we throw away the output layer entirely — we only care about that hidden weight matrix. That's the "byproduct" the notes mention.

Analogy: It's like training someone to guess a person's job by only observing their daily habits (context). We don't actually care about the "job-guessing" skill itself — what we care about is the rich internal understanding of "types of people" that they develop while learning to guess. Word2Vec similarly doesn't care about its literal prediction task — it cares about the internal representation (weights) built while solving it.

Cosine Similarity — How We Measure "Similar" Vectors (Very Important)

Once every word is a vector, a natural question arises: **how do we actually check if two words are similar?**

We don't compare raw numbers directly — instead we measure the **angle between two vectors** using **Cosine Similarity**.

$$\text{cosine similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$
$$\text{cosine similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

- Computes the **cosine of the angle** between vector A and vector B.
- Range: **-1 to 1**
 - **1** → vectors point in the exact same direction → words are highly similar (e.g., "happy" & "joy")
 - **0** → vectors are perpendicular → unrelated words
 - **-1** → vectors point in opposite directions → opposite meaning

Why cosine similarity and not just distance (like Euclidean)?

Cosine similarity only cares about **direction**, not magnitude/length of the vector. Two vectors can point the exact same way but have different lengths (e.g., one word appeared more often in training, giving it a "longer" vector) — cosine similarity correctly still calls them similar, while raw distance metrics could be misled by the length difference.

Analogy: Imagine two arrows drawn on a map, both pointing exactly northeast — one is short, one is long. Cosine similarity says "these point the same direction, so they represent the same idea," ignoring how long each arrow is. Euclidean distance would instead focus on how far apart their tips are, which is misleading here.

In practice: `cosine_similarity(vector("happy"), vector("joy"))` → close to 1
`cosine_similarity(vector("happy"), vector("stone"))` → close to 0

This is the standard way embeddings are compared — used for finding synonyms, nearest neighbors, word analogies (king - man + woman $\approx$ queen), and semantic search.

Two Architectures of Word2Vec

Both use a **sliding window** of context around a target word — this is the "fake problem" being solved.

1. CBOW (Continuous Bag of Words)

Task: Given the surrounding **context words**, predict the **target (center) word**.

Example sentence: "The quick brown fox jumps" with target word "brown":

- Input: context words → quick, fox (and possibly The, jumps depending on window size)
- Output: predict → brown

Analogy: Like a fill-in-the-blank quiz: "*The quick ____ fox jumps*" — you use the surrounding words to guess the missing one.

2. Skip-gram

Task: Given the **target word**, predict the **context words** around it. This is the exact reverse of CBOW.

Example: target word "brown" →

- Input: brown
- Output: predict → quick, fox (and other nearby words)

Analogy: Instead of filling in a blank, you're given one word and asked "what words usually surround this one?" — like being shown "brown" and asked to guess it might appear near "fox," "bear," "hair," etc.

In both cases, once training converges, the learned weight matrix (mapping words to hidden layer activations) becomes the embeddings. The network's actual prediction accuracy isn't the end goal — the embeddings are.

Complete Skip-gram Example (Step-by-Step)

Sentence: "The quick brown fox jumps over the lazy dog"

Window size = 2 (2 words left, 2 words right of target)

Target word: "fox"

Context words within window: quick, brown, jumps, over

Skip-gram creates **training pairs** (input = target, output = one context word each):

(fox → quick)
(fox → brown)
(fox → jumps)
(fox → over)

Each pair becomes one training example fed to the network:

Input: "fox" (one-hot vector, size = vocab size V)

↓

Hidden Layer (size $N=300$, no activation) → this row = fox's embedding

↓

Output Layer (softmax over vocab V) → tries to predict "quick" / "brown" / "jumps" / "over"

The network is trained to maximize the probability of predicting real context words correctly. Words like "fox" that repeatedly co-occur with "quick," "jumps," "animal," "wild," etc. across the entire corpus (millions of sentences, not just one) will gradually get pushed toward similar hidden-layer weight patterns as other animal-related words (e.g., "dog," "cat," "wolf") — because they share similar context distributions.

After training on the full corpus, we discard the output layer. The **input→hidden weight matrix** ($V \times N$) is kept — row for "fox" = fox's final embedding vector.

CBOW vs Skip-gram — When to Use Which

	CBOW	Skip-gram
Predicts	Target word from context	Context words from target
Best for	Small datasets	Large datasets
Training speed	Faster (fewer predictions per	Slower (predicts multiple context words per target)

	CBOW	Skip-gram
	window)	
Quality on rare words	Weaker	Better — skip-gram works well even for less frequent words, because each word gets multiple training examples (one per context word)

Rule of thumb: Small data → CBOW. Large data → Skip-gram (produces richer, better quality embeddings but needs more data/compute to shine).

How to Improve Word2Vec Embeddings

1. **Increase training data** — more text exposure means richer, more reliable context patterns.
2. **Increase vector dimensions** — more dimensions can capture more nuanced relationships (but too high risks sparsity/overfitting again and slower computation — there's a sweet spot, typically 100–300).
3. **Increase window size** — window size = how many neighboring words (on left and right) are considered as "context" for a target word.
 - Small window (e.g., 2) → captures more syntactic relationships (grammar-like closeness).
 - Large window (e.g., 5–10) → captures more topical/semantic relationships (broader thematic similarity).

Pretrained vs Self-trained Word2Vec

- **Pretrained model** — someone already trained Word2Vec on a huge corpus (e.g., Google News dataset, ~100 billion words); you just load and use the vectors directly. Good when your domain is general-purpose (news, everyday language). **If we using this one then it better to match at least 80% between our data vocabulary and pre trained word2vec vocabulary.**
- **Self-trained model** — you train Word2Vec yourself on your own domain-specific corpus (e.g., medical text, legal documents), useful when your vocabulary/context is specialized and general pretrained vectors won't capture domain-specific meaning well. **It requires to have enough data.**

Gensim is the most commonly used Python library for training or loading Word2Vec models easily.

Famous Embedding Models Used in Practice

- **Word2Vec (Google)** — prediction-based, CBOW/Skip-gram, the one covered here.

- **GloVe** (Stanford) — frequency-based, uses global word co-occurrence matrix factorization instead of a neural prediction task.
- **FastText** (Facebook) — extension of Word2Vec that also considers sub-word (character n-gram) information, so it handles rare/out-of-vocabulary words better (e.g., can generate embeddings for misspelled or unseen words by looking at word pieces).
- **BERT embeddings** (Google) — modern contextual embeddings; unlike Word2Vec (which gives one fixed vector per word regardless of context), BERT generates *different* vectors for the same word depending on the sentence it's used in (e.g., "bank" in "river bank" vs "bank account" gets different vectors).
- **ELMo** — another contextual embedding model (character-based, bidirectional LSTM), predecessor to BERT-style contextual embeddings.

Key limitation of Word2Vec to keep in mind: it produces a **static** embedding — the word "bank" always gets the same vector no matter the sentence, so it can't distinguish different meanings of the same word (polysemy). This is exactly the gap that contextual models like BERT were built to solve.

OOV (Out of Vocabulary) Problem

Word2Vec can only produce vectors for words it saw **during training**. If a word never appeared in the training corpus, Word2Vec has **no vector for it at all** — it simply doesn't exist in the model's vocabulary.

Example: If "dragonfruit" never appeared in the training data, asking Word2Vec for `vector("dragonfruit")` fails completely — there's no fallback, no approximation, just nothing.

This is a real limitation in practice — new slang, rare technical terms, typos, or domain-specific words will all break a Word2Vec model.

Why FastText fixes this:

FastText represents each word as a combination of **character n-grams** (sub-word pieces) instead of treating the whole word as one atomic unit.

Example — "dragonfruit" (n-grams of size 3): `<dr, dra, rag, ago, gon, onf, nfr, fru, rui, uit, it>`

Even if "dragonfruit" itself was never seen, FastText can build a reasonable vector for it by combining vectors of these familiar character chunks (many of which likely appeared in other words during training, e.g., "fruit," "dragon"). This lets FastText handle:

- Unseen/rare words
- Misspellings
- Morphological variants (e.g., "running," "runner," "runs" share sub-word info with "run")

Analogy: Word2Vec is like knowing only whole words from a dictionary — if a word isn't in the dictionary, you're stuck. FastText is like knowing common prefixes, roots, and suffixes — even if you've never seen the exact word "dragonfruit," you recognize "dragon" and "fruit" as familiar pieces and can make an educated guess about its meaning.

NLP LECTURE 6: TEXT CLASSIFICATION using Machine learning

What is text classification?

Classification is supervised, classification could be done on tabular data, on images, on textual data like whether email text is spam or not

Types of text classification

Binary 2 classes

Multiclass: having more than 2 classes

Multi label: for input there could be multiple classes, a single new can come into sports, politics

The pipeline:

Data acquisition -> text preprocessing -> text vectorization (bow,tf-idf,word2vec) -> modelling (naïve bayes) -> evaluation (accuracy, confusion matrix) -> deploy

Different Approaches for text classification:

Heuristics – APIs – ML – DL

Tips:

- Use ensemble technique
- Use heuristic technique
- Make sure data is balanced not imbalanced
- Solve as many project

NLP LECTURE 7: PART OF SPEECH TAGGING

If we have sentence then we assign respective part of speech to each word.

Useful in chatbots, information retrieval step

It is preprocessing step in above application

Will use two algorithms hidden Markov algo and

- What POS tagging?
- Application
- Spacy library
- Hidden Markov Viterbi

POS is labeling each word in sentence with its appropriate part of speech.

It's a pre processing step .

Why we do it? what it application:

Named entity recognition

Question answering system

Word sense disambiguation : use of same word in two places conveyinf two different meaning

Chatbots:

Spacy library

Displacy library for displaying

```
Nlp=spicy.load("en_core_web_sm")
```

```
Doc=nlp("I am siffat")
```

```
doc.text
```

```
doc[0].pos_ course grained part of speech
```

```
doc[0].tag_ Fine grained part of speech give more detail information like present past future  
etc
```

how pos tagging works?

algorithm used : hidden Markov models HMM

do one example trthough it

Emission probability: what does it tells us ? divide by column sum

Transition probability : what does it tell? Probability of next noun to be verb or something like this , we mention start and end to each document , divide by row sum

We use Viterbi algo here for optimization ? but optimization of what ?how we do optimization here ?

NLP Lecture 7: Part of Speech (POS) Tagging

What is POS Tagging?

Definition: POS tagging means assigning the correct **grammatical category** (noun, verb, adjective, pronoun, etc.) to every word in a sentence.

Example:

Sentence: "I am Saifat"
Tags: PRON VERB PROPN

It is a **preprocessing step** — meaning it's not usually the final goal by itself, but a foundation that other, more complex NLP applications depend on.

Analogy: Think of POS tagging like labeling every piece of furniture in a house as "chair," "table," "lamp," etc., before you can give someone instructions like "move the chair near the table." You need to know *what category* each word/object belongs to before you can reason about how they relate to each other in a sentence.

Why Do We Need POS Tagging? (Applications)

POS tagging matters because **the same word can behave completely differently depending on its grammatical role**, and many downstream NLP tasks need to know that role to work correctly.

1. **Named Entity Recognition (NER)** — identifying names of people, organizations, locations in text relies heavily on knowing which words are proper nouns (PROPN) vs regular words. E.g., "Apple" (ORG) vs "apple" (fruit, common noun) — POS tagging helps disambiguate.
2. **Question Answering Systems** — to answer "Who wrote this book?" the system needs to identify nouns/proper nouns as candidate answers, and verbs to understand the action being asked about.

3. **Word Sense Disambiguation** — the same word used in different grammatical roles conveys different meanings.
 - o "I will book a flight" → **book** = VERB (to reserve)
 - o "I am reading a book" → **book** = NOUN (an object) POS tagging is often the first clue used to resolve this ambiguity.
4. **Chatbots** — understanding sentence structure (who's doing what to whom) requires knowing each word's grammatical role, which helps the bot correctly parse user intent.

Analogy: Without POS tags, a sentence is just a bag of words to a machine. With POS tags, the machine starts to see *structure* — like the difference between reading random letters versus reading words organized into a sentence.

Using spaCy for POS Tagging

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("I am Saifat")

for token in doc:
    print(token.text, token.pos_, token.tag_)
```

Two levels of granularity spaCy gives you:

- **token.pos_** → **Coarse-grained POS** — the broad category (NOUN, VERB, PRON, ADJ, PROPN, etc.)
- **token.tag_** → **Fine-grained POS** — much more detailed grammatical information, e.g., distinguishing:
 - o VBP (verb, non-3rd person singular present) vs VBD (verb, past tense) vs VBG (verb, gerund/present participle)
 - o So "am" might be tagged coarse = AUX/VERB, fine = VBP (present tense, specific form)

Analogy: Coarse-grained tag = saying "it's a vehicle." Fine-grained tag = saying "it's a 2-door manual sedan." Both are true, but the fine-grained tag gives you much more precise, useful detail.

Visualization: spaCy's companion library **displaCy** lets you visually render POS tags and dependency structure as a diagram instead of reading raw text output.

```
from spacy import displacy
displacy.render(doc, style="dep")
```

How Does POS Tagging Actually Work Internally? — Hidden Markov Model (HMM)

The Core Problem HMM Solves

Given a sentence (a sequence of words), we want to find the **most likely sequence of POS tags** for those words. This is fundamentally a **sequence labeling problem** — the tag for a word doesn't just depend on the word itself, it also depends on **what tags came before it**

(grammar has structure — a determiner like "the" is very likely followed by a noun or adjective, not another determiner).

Analogy: Imagine predicting tomorrow's weather. You don't just look at "what usually happens on this date" (like the word alone) — you also consider "what was the weather yesterday" (like the previous tag), because weather tends to follow patterns (rainy days often follow cloudy days). HMM applies this same "previous state affects next state" logic to grammar.

The Two Key Probabilities in HMM

An HMM for POS tagging is built from a training corpus (a large set of sentences that are **already correctly tagged** by humans). From this corpus, we calculate two types of probabilities:

1. Transition Probability

What it tells us: Given the current POS tag, how likely is the *next* word to have a certain POS tag? This captures **grammar patterns** — e.g., how likely is a NOUN to be followed by a VERB, or a DETERMINER (like "the") to be followed by a NOUN.

We also add special **<START>** and **<END>** markers to every sentence, so we can calculate the probability of which tag typically *starts* a sentence, and which tag typically *ends* one.

How it's calculated: Count how many times tag A is followed by tag B across the training corpus, then **divide by the row sum** (i.e., divide by the total number of times tag A appeared followed by *any* tag).

$$P(\text{tag}_i \mid \text{tag}_{i-1}) = \frac{\text{count}(\text{tag}_{i-1} \text{ followed by tag}_i)}{\text{count}(\text{tag}_{i-1} \text{ followed by anything})}$$

2. Emission Probability

What it tells us: Given a POS tag, how likely is a *specific word* to be generated from that tag? This captures **word-tag association** — e.g., how likely is the word "dog" to appear when the tag is NOUN vs. some other tag.

How it's calculated: Count how many times a specific word appears with a specific tag, then **divide by the column sum** (i.e., divide by the total number of times that tag appeared in the corpus, regardless of which word).

$$P(\text{word}_i \mid \text{tag}_i) = \frac{\text{count}(\text{word}_i \text{ tagged as tag}_i)}{\text{count}(\text{tag}_i \text{ appearing at all})}$$

Analogy for both probabilities together:

- **Transition probability** = "grammar sense" — knowing that determiners are usually followed by nouns/adjectives, regardless of which specific words are involved.

- **Emission probability** = "vocabulary sense" — knowing that if something is tagged as a NOUN, it's much more likely to be a word like "dog" or "book" than a word like "quickly."

Together, HMM combines **"what usually comes next grammatically"** (transition) with **"what word usually shows up for this tag"** (emission) to figure out the best tag sequence.

Full Worked Example

Suppose our tiny training corpus (already tagged) contains these sentences:

```
<START> The/DET dog/NOUN barks/VERB <END>
<START> The/DET cat/NOUN runs/VERB <END>
<START> A/DET dog/NOUN sleeps/VERB <END>
```

Step 1: Build Transition Counts (then convert to probabilities)

Count how often each tag follows another tag:

From \ To DET NOUN VERB END

START	3	0	0	0
DET	0	3	0	0
NOUN	0	0	3	0
VERB	0	0	0	3

Convert to probabilities (divide each row by its row sum):

- $P(\text{DET} \mid \text{START}) = 3/3 = 1.0$
- $P(\text{NOUN} \mid \text{DET}) = 3/3 = 1.0$
- $P(\text{VERB} \mid \text{NOUN}) = 3/3 = 1.0$
- $P(\text{END} \mid \text{VERB}) = 3/3 = 1.0$

(In a real, larger corpus, these would be spread across multiple possible tags rather than being cleanly 1.0 — this toy example is simplified for clarity.)

Step 2: Build Emission Counts (then convert to probabilities)

Count how often each word appears under each tag:

Word \ Tag DET NOUN VERB

The	2	0	0
A	1	0	0
dog	0	2	0
cat	0	1	0
barks	0	0	1
runs	0	0	1
sleeps	0	0	1

Convert to probabilities (divide each column by its column sum):

- $P(\text{The} \mid \text{DET}) = 2/3 \approx 0.67$
- $P(\text{A} \mid \text{DET}) = 1/3 \approx 0.33$
- $P(\text{dog} \mid \text{NOUN}) = 2/3 \approx 0.67$
- $P(\text{cat} \mid \text{NOUN}) = 1/3 \approx 0.33$
- $P(\text{barks} \mid \text{VERB}) = 1/3 \approx 0.33$

Step 3: Tag a New Sentence — "The dog runs"

We now want the most likely tag sequence for a new sentence. For each word, we combine emission probability (how likely is this word given a tag) with transition probability (how likely is this tag given the previous tag).

For the sequence $\text{DET} \rightarrow \text{NOUN} \rightarrow \text{VERB}$:

$$\begin{aligned} & P(\text{DET} \rightarrow \text{NOUN} \rightarrow \text{VERB} \mid \text{"The dog runs"}) = \\ & P(\text{DET} \mid \text{START}) \times P(\text{The} \mid \text{DET}) \times \\ & P(\text{NOUN} \mid \text{DET}) \times P(\text{dog} \mid \text{NOUN}) \times \\ & P(\text{VERB} \mid \text{NOUN}) \times P(\text{runs} \mid \text{VERB}) \end{aligned}$$

$$= 1.0 \times 0.67 \times 1.0 \times 0.67 \times 1.0 \times 0.33 \approx 0.148$$

This is the probability of **one specific path** (one specific tag sequence) through the sentence. In a real sentence, each word could potentially take *multiple* possible tags, creating many possible paths — and we need to find the single best one.

Why We Need the Viterbi Algorithm (What "Optimization" Means Here)

The Actual Problem

For a sentence with **n words**, and a tagset with **T possible tags**, there are **T^n possible tag sequences** to consider (since each word could theoretically take any of the T tags). Trying every single combination and computing its full probability (like we did above) is **computationally explosive** — for a 10-word sentence with 20 possible tags, that's $20^{10} \approx 10$ trillion combinations. This is completely infeasible to brute-force.

This is exactly what "optimization" means here: we need an efficient way to find the **single most probable tag sequence** without literally computing the probability of every possible sequence one by one.

Analogy: Imagine you're trying to find the shortest driving route from City A to City Z, passing through several intermediate cities, where multiple roads connect each pair of cities. Checking *every possible full route* combination is enormously wasteful. Instead, you can be smart: at each intermediate city, only keep track of the *best way to reach that city so far* — discard all the worse paths reaching that same city. This is exactly the idea behind Viterbi.

The **Viterbi algorithm** is a **dynamic programming** algorithm. The key insight: instead of recomputing probabilities for every full path from scratch, it builds up the solution **word by word**, and at each step, for each possible tag, it only keeps the **single best (highest-probability) path** that leads to that tag at that position — discarding all worse alternatives immediately.

Step-by-step logic:

1. **Initialization:** For the first word, compute the probability of each possible tag using $P(\text{tag} \mid \text{START}) \times P(\text{word} \mid \text{tag})$.
2. **Recursion (the core optimization step):** For each subsequent word, and for each possible tag it could have, compute:
$$\text{best_score}(\text{tag}_i) = \max_{\text{prev_tag}} \left[\text{best_score}(\text{prev_tag}) \times P(\text{tag}_i \mid \text{prev_tag}) \right] \times P(\text{word}_i \mid \text{tag}_i)$$
 In plain words: "What's the best possible score to reach this tag at this word, coming from *any* previous tag?" We only keep the **maximum** — all other, worse-scoring paths into this tag are thrown away and never considered again.
3. **Store backpointers:** At each step, remember *which* previous tag gave the best score (this is what lets us reconstruct the full best path at the end, not just know its final probability).
4. **Termination:** At the last word, pick the tag with the highest overall score, then **backtrack** through the stored backpointers to recover the entire best tag sequence.

Why this is efficient: Instead of T^n total paths, Viterbi only ever needs to track T possible "best scores" at each of the n word positions — reducing the complexity from **exponential** (T^n) down to **polynomial** (roughly $n \times T^2$). This is what makes POS tagging on real sentences computationally practical.

Analogy recap: Viterbi is like a GPS navigation system that, instead of evaluating every conceivable full route from start to destination, incrementally builds the best route city by city — at each city, it only remembers the cheapest way to have arrived there, and discards all worse alternatives immediately, then reconstructs the full best route at the very end by tracing back through its choices.

Summary — Putting It All Together

Training Corpus (tagged sentences)

↓

Calculate Transition Probabilities (grammar: tag → tag likelihood, row-normalized)

Calculate Emission Probabilities (vocabulary: tag → word likelihood, column-normalized)

↓

New Sentence to Tag

↓

Viterbi Algorithm (dynamic programming)

- efficiently searches all possible tag sequences
- keeps only the best-scoring path to each tag at each step
- avoids exponential brute-force search

↓

Most Likely POS Tag Sequence (output)

One-line intuition to remember: HMM defines *how likely* any given tag sequence is (using transition + emission probabilities); Viterbi is the *efficient search strategy* that finds the best sequence without checking every possibility.

NLP: NAMED ENTITY RECOGNITION

Use cases:
search for news etc
recommendations (like stories for u)
customer care

spacy library is used here

```
doc = nlp("Michael Bloomberg founded Bloomberg L.P. in 1982")  
  
for ent in doc.ents:  
    print(ent.text, "|", ent.label_, "|", spacy.explain(ent.label_))
```

Approach: simple lookup:

RULE BASED NER (REGEX ETC)

MACHINE LEARNING (CRF, BERT)

NLP LECTURE :TOPIC MODELLING

NLP: Named Entity Recognition (NER)

What is NER?

Named Entity Recognition is the task of finding and classifying "**named entities**" in text — real-world objects like people, organizations, locations, dates, money amounts, products, etc. — and labeling *what type* of entity each one is.

Example:

Sentence: "Elon Musk founded SpaceX in California in 2002."

Elon Musk → PERSON
SpaceX → ORG (organization)
California → GPE (geo-political entity / location)
2002 → DATE

Analogy: Reading a sentence normally, you see just a stream of words. NER is like using a highlighter with different colors — one color for people's names, another for places, another for dates — so you can instantly scan and pull out exactly the "who, where, when" facts from a wall of text, instead of reading every word carefully yourself.

NER builds directly on **POS tagging** — POS tagging tells you a word is a noun/proper noun, but NER goes a step further and tells you *what kind* of real-world thing that noun refers to.

Why NER Matters — Use Cases

1. **Search Engines / News Search** When you search "Elon Musk news," the search engine needs to recognize "Elon Musk" as a specific *person entity*, not just three separate common words, so it can retrieve articles specifically about him rather than any article containing those words scattered randomly.
2. **Recommendations** (e.g., "stories for you," personalized content feeds) If you read several articles mentioning "Tesla" and "SpaceX" (ORG entities), the recommendation system can detect this pattern and suggest more content involving those same organizations — NER helps extract the *key entities* a user cares about, beyond just keywords.
3. **Customer Care / Support Chatbots** If a customer writes "My order #48213 from Amazon hasn't arrived since March 5th," NER can automatically pull out the order number, organization, and date — letting the system route or auto-fill a support ticket without a human manually re-typing these details.

Analogy: Think of NER as an assistant that reads a huge pile of documents for you and pulls out sticky notes labeled "Person," "Company," "Date," "Money" — so you never have to read the whole document just to find the key facts.

Using spaCy for NER

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("Elon Musk founded SpaceX in California in 2002.")

for ent in doc.ents:
    print(ent.text, ent.label_)

# Output:
# Elon Musk    PERSON
# SpaceX      ORG
# California   GPE
# 2002         DATE
```

Visualizing with displaCy:

```
from spacy import displacy
displacy.render(doc, style="ent")
```

This highlights entities directly in colored boxes over the text — much easier to scan visually than reading raw label output.

Approaches to Building NER Systems

1. Simple Lookup (Dictionary-Based)

How it works: Maintain a predefined list ("gazetteer") of known entities — e.g., a list of all country names, a list of known company names — and simply **check if a word/phrase in the text matches something in that list**.

Example: If your list contains ["USA", "India", "Germany"], any exact match in text gets tagged as a location.

Pros: Extremely simple, fast, no training needed. **Cons:** Can't handle new/unseen entities not in your list (e.g., a brand-new startup name), and can't use context — it will incorrectly tag "Turkey" as a country even in the sentence "I'm cooking turkey for dinner."

Analogy: This is like using a fixed phonebook — if a name isn't already written in the book, you have no way of recognizing it, no matter how obviously it looks like a name.

2. Rule-Based NER (Regex, Pattern Matching)

How it works: Define explicit **patterns/rules** using regular expressions or grammatical patterns to catch entities that follow a predictable structure.

Example patterns:

- Dates: `\d{1,2}/\d{1,2}/\d{4}` → catches "05/03/2024"
- Money: `\$\d+(\.\d{2})?` → catches "\$45.99"
- Capitalized word sequences after "Mr./Dr./Mrs." → likely a PERSON name

Pros: Works very well for *structured* entities with predictable formats (dates, phone numbers, money, emails) — doesn't need training data. **Cons:** Breaks easily on messy or unpredictable real-world text; can't detect entities without a clear structural pattern (e.g., recognizing "Zephyra Technologies" as a company name has no fixed regex pattern to catch it), and doesn't generalize to new entity styles.

Analogy: Rule-based NER is like teaching someone "any word right after 'Mr.' is a person's name" — great for consistent formats, but falls apart the moment text doesn't follow the expected pattern.

3. Machine Learning-Based NER (CRF, BERT)

This is the modern, most accurate approach — instead of hardcoded rules, a model **learns from labeled examples** what entities typically look like in context.

CRF (Conditional Random Fields)

- A statistical sequence-labeling model, similar in spirit to HMM (which you covered for POS tagging) but more flexible — it can incorporate many features at once (the word itself, its POS tag, surrounding words, capitalization, prefixes/suffixes) rather than being limited to just "previous tag → next tag" transitions.
- Learns from a labeled training corpus which combinations of features are typically associated with entity boundaries (where an entity starts/ends) and types.
- Good for structured prediction tasks like NER where the *sequence* of labels matters (e.g., knowing "New" + "York" together form one LOCATION entity, not two separate ones).

BERT-based NER

- Uses a pretrained transformer model (BERT) fine-tuned specifically for entity recognition.
- Because BERT produces **contextual embeddings** (recall from earlier notes — the same word gets different vectors depending on sentence context), it can much better distinguish cases like "Apple" the company vs. "apple" the fruit, or "Amazon" the company vs. "Amazon" the river — based on the surrounding sentence, not just the word itself.
- Currently the state-of-the-art approach for NER — significantly outperforms rule-based and CRF approaches on messy, real-world, unstructured text, because it generalizes to unseen entities by understanding context and semantics rather than memorized patterns.

Pros of ML-based (CRF/BERT): Handles unseen entities well, understands context, much more robust to messy real-world text. **Cons:** Requires labeled training data, more computationally expensive than lookup/rule-based methods, less transparent/interpretable than simple rules.

Analogy: A lookup table is like memorizing a fixed list of celebrity names. Rule-based is like knowing "capitalized words after 'Mr.' are names." ML-based (BERT) is like a well-read, experienced person who can correctly identify that "Kai" is a person's name in one sentence and a brand name in another, purely by understanding the *context* of the sentence — without ever having memorized that exact case beforehand.

Quick Comparison Table

Approach	How it Works	Handles New/Unseen Entities?	Understands Context?	Effort to Build
Simple Lookup	Match against a fixed list	✗ No	✗ No	Very low

Approach	How it Works	Handles New/Unseen Entities?	Understands Context?	Effort to Build
Rule-Based (Regex)	Predefined patterns	✗ No	✗ No	Low
CRF	Learns from labeled sequence features	✓ Somewhat	✓ Somewhat	Medium
BERT-based	Fine-tuned contextual transformer	✓ Yes	✓ Yes (deeply)	High (but pretrained models make this easier now)

NLP Lecture: Topic Modeling

What is Topic Modeling?

Topic Modeling is an **unsupervised learning** technique that automatically discovers the abstract "topics" that occur across a collection of documents — **without any labeled data**, and without being told in advance what the topics are.

Example: Feed in thousands of news articles, and topic modeling might automatically discover:

- Topic 1: words like `election`, `government`, `president`, `vote` → clearly about **Politics**
- Topic 2: words like `goal`, `match`, `team`, `player` → clearly about **Sports**
- Topic 3: words like `stock`, `market`, `economy`, `inflation` → clearly about **Finance**

The algorithm never explicitly labels these as "Politics" or "Sports" — it just groups statistically co-occurring words into clusters (topics), and a human interprets what each cluster represents by looking at its top words.

Analogy: Imagine dumping thousands of unsorted newspaper clippings on a table, with no headings or categories. Topic modeling is like an assistant who reads through all of them and starts making piles based on which clippings tend to use similar vocabulary — one pile naturally ends up being "sports" articles, another "politics," without ever being told those category names in advance. You just look at each pile afterward and give it a name based on what's in it.

Why Topic Modeling? (Use Cases)

1. **Organizing large document collections** — automatically categorizing thousands of articles, support tickets, or reviews into themes without manual tagging.

2. **Trend discovery** — seeing what themes are emerging across large volumes of text (e.g., social media posts) over time.
3. **Recommendation systems** — grouping content by discovered topic to recommend similar articles/products.
4. **Summarizing large corpora** — quickly understanding "what is this huge pile of documents mostly about" without reading everything.

Key Difference from Everything Covered So Far

Notice this is fundamentally different from POS tagging and NER:

- **POS tagging / NER = supervised** — trained on labeled examples (humans already tagged the correct answer).
- **Topic Modeling = unsupervised** — no labels at all; the algorithm discovers structure purely from patterns of word co-occurrence across documents.

Analogy: POS tagging/NER is like a student learning from an answer key (supervised). Topic modeling is like a student left alone with a huge stack of books and asked to group them by theme purely by noticing which books share similar vocabulary — no answer key provided at all (unsupervised).

Core Intuition: How Does It Work?

The foundational assumption behind topic modeling: **each document is a mixture of multiple topics, and each topic is a distribution over words.**

This means:

- A single document isn't purely "100% about sports" — it might be 70% Sports, 20% Business (e.g., an article about a player's contract negotiation), 10% something else.
- A "topic" itself isn't one fixed keyword — it's a **probability distribution over the entire vocabulary**, where certain words are much more likely to appear under that topic than others.

Example — Topic "Sports" as a word distribution:

```
goal      → 0.08
match     → 0.07
team      → 0.06
player    → 0.05
election  → 0.0001  (extremely unlikely under this topic)
```

Example — one document's topic mixture:

Document: "The player negotiated a new contract worth millions before the match."

Topic mixture: 60% Sports, 35% Business/Finance, 5% Other

The algorithm's job is to work backward: **given only the raw text of many documents, figure out (a) what the underlying topics (word distributions) probably are, and (b) what mixture of those topics each document is made of** — purely from statistical word co-occurrence patterns.

Most Common Algorithm: LDA (Latent Dirichlet Allocation)

LDA is the most widely used topic modeling algorithm.

- **"Latent"** — the topics are hidden/not directly observed; we only see the words, and must infer the topics that likely generated them.
- **"Dirichlet"** — refers to the type of probability distribution used to model "how topics mix within documents" and "how words mix within topics" (a Dirichlet distribution is well-suited for modeling proportions that must sum to 1, like topic mixture percentages).
- **"Allocation"** — the process of assigning/allocating words to topics.

Simplified intuition of how LDA works (generative story): LDA assumes documents were "generated" in this imagined process (in reality documents are already written, but this is the mental model behind the math):

1. For each document, first randomly decide a topic mixture (e.g., "this document will be 70% Sports, 30% Business").
2. Then, to generate each word in the document, first pick a topic according to that mixture (e.g., roll a dice weighted 70/30 toward Sports), then pick a word from that topic's word-distribution (e.g., from the Sports topic, "match" is likely to be picked).

LDA then works **backward** from real documents to infer the most probable topic distributions and word distributions that would have generated the text we actually observe.

Analogy: Imagine each document is like a smoothie made by blending different fruit purees (topics) in different proportions. LDA is like a food scientist who, given only the finished smoothie (the document), tries to figure out which fruits (topics) were blended and in what proportions — purely by analyzing the flavor profile (word patterns), without ever seeing the blending process directly.

Practical Steps to Do Topic Modeling (e.g., with Gensim / sklearn)

1. **Preprocess text** — lowercase, remove stopwords, remove punctuation, lemmatize (all the preprocessing steps from earlier notes apply here too — clean text matters a lot for good topics).
2. **Convert to a numeric representation** — typically Bag of Words or TF-IDF (from earlier notes) is used as input to LDA — note LDA itself works on raw word counts/frequencies, not embeddings like Word2Vec.
3. **Choose the number of topics (k)** — you must decide in advance how many topics you want LDA to discover (e.g., $k=5$). This is a key hyperparameter — too few topics

merges distinct themes together, too many topics splits a single theme into overly narrow fragments.

4. **Train LDA** — the algorithm iteratively refines its guess of topic-word distributions and document-topic distributions until it converges on a stable solution.
5. **Interpret topics manually** — look at the top-weighted words in each discovered topic and assign a human-readable label (e.g., "this topic's top words are goal/match/team/player → label it 'Sports'").

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

cv = CountVectorizer(stop_words='english', max_df=0.9, min_df=2)
X = cv.fit_transform(documents)  # documents = list of preprocessed texts

lda = LatentDirichletAllocation(n_components=5, random_state=42)  # 5
topics
lda.fit(X)

# print top words for each discovered topic
words = cv.get_feature_names_out()
for idx, topic in enumerate(lda.components_):
    top_words = [words[i] for i in topic.argsort()[-10:]]
    print(f"Topic {idx}: {top_words}")
```

Other Topic Modeling Techniques (Good to Know)

- **LSA (Latent Semantic Analysis)** — an older technique using matrix factorization (SVD) on the TF-IDF matrix, conceptually similar to GloVe's approach for word embeddings, but applied at the document-topic level instead. Generally considered less interpretable and lower quality than LDA.
- **NMF (Non-negative Matrix Factorization)** — another matrix-factorization-based alternative to LDA, often produces more interpretable topics on smaller datasets, and doesn't rely on the probabilistic Dirichlet assumptions LDA uses.
- **BERTopic** — a modern approach that uses BERT-based embeddings (contextual, semantic) combined with clustering, instead of relying purely on word co-occurrence counts like LDA. Generally produces higher-quality, more coherent topics on modern text (like social media, complex documents), because it leverages the deep semantic understanding of transformer embeddings rather than just statistical word-frequency patterns.

Summary Comparison

Aspect	NER	Topic Modeling
Learning type	Supervised (needs labeled entities)	Unsupervised (no labels needed)
Output	Specific entity type per word/phrase (PERSON, ORG, DATE)	Broad thematic clusters across a document collection
Granularity	Word/phrase level	Document/corpus level
Common tools	spaCy, CRF, BERT-NER	LDA, NMF, LSA, BERTopic

Aspect	NER	Topic Modeling
Example use	Extracting "Elon Musk" as PERSON from one sentence	Discovering that 1000 articles cluster into Politics/Sports/Finance themes

RAG (Retrieval-Augmented Generation)

What Problem Does RAG Solve?

LLMs like GPT/Claude are trained on a fixed snapshot of data up to some cutoff date. They:

- **Don't know your private/specific data** (your university's site content, your company's internal docs, live/updated info).
- Can **hallucinate** — confidently generate wrong facts when they don't actually know the answer.
- Can't cite exact sources for what they say.

RAG's core idea: Instead of relying purely on what the LLM "memorized" during training, **retrieve relevant real documents/text at query time, and hand them to the LLM as context**, so it generates its answer grounded in actual facts you gave it — not from memory alone.

Analogy: An LLM without RAG is like a student taking a closed-book exam purely from memory — sometimes brilliant, sometimes confidently wrong. RAG turns it into an **open-book exam** — the student (LLM) is handed the exact relevant textbook pages (retrieved chunks) right before answering, so their answer is grounded in real material instead of guesswork.

The Name Breaks Down Exactly What It Does

- **Retrieval** — find the most relevant pieces of information from a knowledge source (your documents).
- **Augmented** — add that retrieved information into the LLM's input/context.
- **Generation** — the LLM generates a natural-language answer using that added context.

What Knowledge/Foundation is RAG Based On?

RAG combines two separate fields that already existed independently, fused together:

1. **Information Retrieval (IR)** — the decades-old field behind search engines (Google, Elasticsearch). The core problem: given a query, find the most relevant documents from a large collection. Traditionally done via keyword matching (TF-IDF, BM25) —

RAG modernizes this using **semantic search** via embeddings (covered in your earlier notes — Word2Vec/BERT/SBERT lineage).

2. **Text Generation via LLMs** — the ability of transformer-based language models (GPT, Claude, etc.) to take input text and generate coherent, fluent responses.

RAG = **semantic retrieval system** + **LLM generation**, wired together so the retrieval step feeds the generation step.

This directly builds on everything from your embedding notes — RAG *is* essentially the practical application of sentence embeddings + cosine similarity (which we just discussed for your uni chatbot) combined with an LLM at the final step.

The Technologies/Techniques Used at Each Stage

1. Document Loading & Chunking

Raw documents (PDFs, HTML pages, Word docs) are split into smaller **chunks** (e.g., 200–500 words), because:

- LLMs have a limited context window — you can't stuff an entire website into one prompt.
- Smaller chunks give more precise, relevant retrieval (a whole page might mix scholarship info and library hours — you want to retrieve just the relevant paragraph).

Tools: LangChain / LlamaIndex text splitters, custom regex-based chunkers.

2. Embedding Model (Retrieval Foundation)

Each chunk is converted into a dense vector using a **sentence embedding model** (this is exactly what we discussed for your uni chatbot).

Common choices: sentence-transformers (all-MiniLM-L6-v2, all-mpnet-base-v2), OpenAI/Cohere embedding APIs, Google's text-embedding-gecko.

3. Vector Database (Storage + Fast Similarity Search)

All chunk embeddings are stored in a **vector database**, optimized for fast **nearest-neighbor search** (finding the most similar vectors to a query vector) — this would be too slow with brute-force comparison at scale (millions of chunks).

Common choices:

- **FAISS** (Facebook, open-source, runs locally, most popular for prototyping)
- **Pinecone** (managed cloud vector DB)
- **Chroma** (lightweight, easy for small/medium projects — great for your uni chatbot)
- **Weaviate, Milvus, Qdrant** (more production-scale options)

Technique used internally: Approximate Nearest Neighbor search (ANN) — algorithms like **HNSW (Hierarchical Navigable Small World graphs)** make similarity search fast even across millions of vectors, trading a tiny bit of accuracy for huge speed gains.

4. Retrieval Step

When a user asks a question:

- Embed the question using the **same embedding model** used for the chunks.
- Use **cosine similarity** (sometimes dot product or Euclidean distance, depending on the DB) to find the top-k most similar chunks.

5. Augmentation (Prompt Construction)

The retrieved chunks are inserted into the LLM's prompt, typically structured like:

```
Context:
[retrieved chunk 1]
[retrieved chunk 2]
[retrieved chunk 3]

Question: {user's question}
```

Answer using only the information in the context above.

6. Generation (LLM)

The LLM (Claude, GPT, Llama, etc.) reads the context + question and generates the final natural-language answer, grounded in the retrieved material.

Optional Advanced Techniques (Good to Know)

- **Re-ranking:** After initial retrieval (fast but rough), a second, more precise model (a "cross-encoder") re-scores the top candidates for better relevance ordering. Slower but more accurate — often used as a second pass.
- **Hybrid Search:** Combine traditional keyword search (BM25/TF-IDF) **with** semantic embedding search — catches cases where exact keyword matches matter alongside semantic meaning (e.g., matching an exact course code like "CS101").
- **Chunk overlap:** Chunks are often given slight overlapping text at boundaries, so information split across a chunk boundary isn't lost.
- **Metadata filtering:** Attach metadata to chunks (e.g., "department: admissions", "date: 2026") so retrieval can be filtered/narrowed before similarity search.

Full RAG Pipeline (Visual Summary)

```
[Your Documents]
  ↓ chunking
[Text Chunks]
  ↓ embedding model (e.g. SBERT)
[Vectors] → stored in [Vector DB: FAISS/Chroma/Pinecone]
```

--- at query time ---

```

[User Question]
  ↓ embed with SAME model
[Query Vector]
  ↓ cosine similarity search in Vector DB
[Top-K Relevant Chunks]
  ↓ inserted into prompt as context
[LLM: Claude/GPT]
  ↓
[Final Answer, grounded in your actual data]

```

Directly Tying This to Your Uni Chatbot

Everything you'd need for your chatbot **is RAG**:

- Your site content = the document source.
- all-MiniLM-L6-v2 (which we discussed) = the embedding model powering the retrieval step.
- Chroma or FAISS = the vector database to store and search those embeddings.
- Claude/GPT = the generation step that turns retrieved chunks into a natural answer.

So the pipeline you outlined earlier for your uni chatbot **is a complete RAG system** — the term "RAG" is simply the name for that entire architecture.

=====

1. About ANN - https://en.wikipedia.org/wiki/Artificial_neural_network
2. About CNN - https://en.wikipedia.org/wiki/Convolutional_neural_network
3. yann le cunn - https://en.wikipedia.org/wiki/Yann_LeCun
4. About RNN - https://en.wikipedia.org/wiki/Recurrent_neural_network
5. About Autoencoders - <https://en.wikipedia.org/wiki/Autoencoder>
6. About GAN - <https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/>
7. Unsupervised Pre-training - <https://www.jmlr.org/papers/volume11/erhan10a/erhan10a.pdf>
8. ImageNET - <https://www.image-net.org/>
9. Alphago - <https://www.youtube.com/watch?v=WXuK6gekU1Y>
10. Image Colorization - <https://pyimagesearch.com/2019/02/25/black-and-white-image-colorization-with-opencv-and-deep-learning/>
11. Audio gen in mute video - <https://www.youtube.com/watch?v=QZGqLLsNArg>
12. Image Caption gen - <https://www.analyticsvidhya.com/blog/2021/12/step-by-step-guide-to-build-image-caption-generator-using-deep-learning/>
13. Pixel Restoration - <https://www.analyticsvidhya.com/blog/2021/05/deep-learning-for-image-super-resolution/>
14. This person does not exist - <https://thispersondoesnotexist.com/>
15. AI generates story - <https://www.youtube.com/watch?v=LY7x2lhqjmc>
15. Deep Dream - <https://www.youtube.com/watch?v=nelv5kkNxis>